# KaffeeKlatsch

Das Magazin rund um Software-Entwicklung

ISSN 1865-682X
01/2008



Sie können den

KaffeeKlatsch

*das Magazin rund um Software-Entwicklung*

kostenlos und unverbindlich unter

http://www.bookware.de/abo

abonnieren.

Editorial

# Mal ganz offen

Braucht es denn in Deutschland wirklich noch ein Magazin, das sich mit dem Thema Java beschäftigt? Gibt es denn nicht schon genügend Zeitschriften zum Thema .NET? Wird denn nicht das Thema Software-Entwicklung in jeder Fachzeitschrift behandelt? Und ist denn die Objektorientierung nicht schon ein richtig alter Hut?

Obwohl die Hardware um uns herum immer noch dem Moore'schen Gesetz [1] folgt – und neue Technologien, allen gegenteiligen Prognosen zum Trotz, immer wieder rechtzeitig dazu führen, dass sich die Rechnerleistung alle zwei Jahre verdoppelt – dümpelt die Software-Entwicklung immer noch vor sich hin. Das Wirth'sche Gesetz besagt sogar, dass Software deutlich schneller langsamer wird, als Hardware schneller[1].

Anforderungen werden immer komplizierter und nebenläufiger, Software wird immer komplexer und undurchschaubarer. Somit wird es auch immer schwieriger, vernünftige und verlässliche Aufwandsabschätzungen zu machen. Das Rad wird ständig neu erfunden: Frameworks und Bibliotheken sprießen wie Pilze aus dem Boden und versuchen sich gegenseitig den Rang abzulaufen; Immer speziellere Lösungen tauchen für kleinste Varianten ein und desselben Problems auf. Das Internet bietet schier unbegrenzte Hilfe an: keine Dokumentation, die nicht zu finden, und kein Thema, das nicht in mindestens einem Blog vorhanden ist; Es gibt aber praktisch kein Stichwort, zu dem nicht wenigstens hunderttausend Einträge existieren. Es scheint also doch noch Bedarf zu geben, die Software-Entwicklung ein bisschen besser und einfacher zu machen.

Mit dem KaffeeKlatsch wollen wir Abhilfe schaffen. Der KaffeeKlatsch soll eine Kommunikationsplattform sein, die sich ein wenig von dem Zeitgeistigen unterscheidet: kein Blog, kein Feed, keine E-Mail. Aber ausgewählte, redigierte Texte, Erfahrungsberichte, Anleitungen, Meinungen: Alles, was das Entwicklerherz höher schlagen lässt und tatsächlich bei der täglichen Arbeit hilft. Der KaffeeKlatsch soll eine Ergänzung – keine Konkurrenz – zu den etablierten Magazinen sein und die speziellen Möglichkeiten nutzen, die diese nicht haben: den quasi unbegrenzten Platz für den Blick über den technologischen „Tellerrand".

[1] Niklaus Wirth, wohl den meisten als Erfinder von Pascal und Oberon bekannt, schreibt dies in seinem Aufsatz „A Plea for Lean Software" [2] allerdings seinem Kollegen Martin Reiser zu: „Software is getting slower more rapidly than hardware becomes faster".

Der KaffeeKlatsch ist ein kostenloses Magazin rund um die Software-Entwicklung mit Java und .NET als Technologieschwerpunkt. Neben Java und C# soll auch all das zum Zuge kommen, was hilfreich ist, die Probleme bei der Software-Entwicklung besser zu verstehen, Fehler zu vermeiden oder einfach nur zuverlässiger und schneller zum Ziel zu kommen – was immer das für jeden einzelnen bedeuten mag.

Und das nicht jeden Tag oder jede Stunde, sondern ganz banal einmal pro Monat. In einem ansehnlichen Format, als eine Einheit, die sich auch bequem drucken und so auf Wunsch auch mal gemütlich auf dem Sofa, in der Badewanne oder unterwegs beim Essen – ganz ohne Rechner! – lesen lässt.

Das können wir natürlich nicht alleine verwirklichen. Wir sind auf willige Leser angewiesen, die nicht nur die Qualität von eingereichten Beiträgen beurteilen, sondern sie auch verbessern und „druckbar" machen. Wir brauchen aufmerksame Leser, die interessante Nachrichten oder Artikel melden oder über sie berichten. Wir suchen nach Neugierigen, die neue Technologien ausprobieren und ihre damit gemachten Erfahrungen teilen wollen. Wir wünschen uns Lehrwillige, die Anfängern oder Experten noch etwas beibringen wollen. Unser Ziel ist es eine Plattform für Software-Entwickler schaffen, die ihre Ideen, Sorgen und Ängste, aber vor allem ihre Lösungen teilen wollen. Eine Art offenes Wissen von Software-Entwicklern für Software-Entwickler.

Ich hoffe also auf eine interessierte Leserschaft, die sich gelegentlich oder regelmäßig aktiv beteiligt, den Inhalt dieses Magazins mitzubestimmen und durch konstruktive Kritik zu verbessern, um so das Niveau der Software-Entwicklungswelt ein kleines bisschen anzuheben.

Und so wünsche ich Ihnen viel Spaß beim Kaffeeklatschen,

Michael Wiedeking
Herausgeber

Referenzen


[1] Moore, Gordon E. *Cramming more components onto integrated circuits* Electronics, Volume 38, Number 8, 19. April 1965, http://download.intel.com/research/silicon/moorespaper.pdf

[2] Wirth, Niklaus *A Plea for Lean Software*, Computer, 28 (2), Februar 1995: pp. 64-68, http://doi.ieeecomputersociety.org/10.1109/2.348001, siehe auch http://cr.yp.to/bib/1995/wirth.pdf


Weiterführende Literatur

- Intel *Moore's Law* http://www.intel.com/technology/mooreslaw/index.htm
- Wikipedia *Moore'sches Gesetz* http://de.wikipedia.org/wiki/Mooresches_Gesetz
- Wikipedia: *Wirth'sches Gesetz* http://de.wikipedia.org/wiki/Wirthsches_Gesetz

# Inhalt

Editorial ..... 3
Inhalt ..... 4
Beitragsinfo ..... 6
User Groups ..... 44
Werbung ..... 46
Das Allerletzte ..... 59

## Artikel

Alles oder nichts
Verteilte Transaktionen ..... 7

Echt abgefahren!
Einführung in Groovy ..... 11

Das hängt davon ab …
Eine Einführung in Dependency Injection mit Guice ... 21

Super, Packages!
Das Java-Modul-System ..... 27

Vom Unterschied, der den Unterschied macht
Eine neues Programmier-Paradigma ..... 31

Selbststudium für Computer
Skizze einer Theorie zum autonomen, maschinellen Wissenserwerb ..... 33

Destruktive Robustheit
Wie gut gemeinte Fehlervermeidung das Leben erschwert .. 36

## Kolumnen

Über die Halbsumme zweier ganzer Zahlen
Des Programmierers kleine Vergnügen ..... 29

Das Kreuz mit der elektronischen Post
Deutsch für Informatiker ..... 39

Informatik und Gesellschaft
Hex and the City ..... 41

Kaffeesatz ..... 43

## Alles oder nichts

Verteilte Transaktionen ..... 7
von Thomas Haug

In den meisten ernst zu nehmenden IT-Systemen müssen Daten auf die eine oder andere Art dauerhaft gespeichert werden. Hierfür haben sich Transaktionen nach dem so genannten ACID-Prinzip etabliert. Dieser erste von drei Teilen beschäftigt sich mit den Grundlagen von Commit-Protokollen, über die sich verteilte Transaktionen realisieren lassen.

## Echt abgefahren!

Einführung in Groovy ..... 11
von Falk Sippach

Für die Virtuelle Maschine von Java (JVM) gibt es mehr als zweihundert Programmiersprachen. Im Jahre 2003 hat sich auch Groovy aufgemacht sich als sogenannte dynamische Skriptsprache einen Platz in diesem hart umkämpften Markt zu erobern. Dabei glänzt Groovy nicht nur mit einer prägnanten Syntax und dynamischen Eigenschaften, sondern auch mit der perfekten Integration mit Java. Dieser Artikel gibt eine kurze Einführung in die wichtigsten Features dieser modernen Programmiersprache.

## Das hängt davon ab...

Eine Einführung in Dependency Injection mit Guice ..... 21
von Christoph Dietze

Guice ist ein Dependency Injection Framework in Java und entstammt dem Hause Google. Zu den Hauptmerkmalen von Guice gehören seine Leichtgewichtigkeit, der Verzicht auf Fremdsprachen wie XML und die starke Verwendung von Java-5-Features wie Annotations und Generics. Dieser Artikel gibt eine Einführung, wie Dependency Injection mit Guice realisiert werden kann und welche weitergehenden Features Guice unterstützt.

## Super, Packages!

Das Java-Modul-System ..... 27
von Jörn Hahn

In der siebten Version von Java wird ein Modul-System eingeführt, das mit einem zusätzlichen „Superpackage"-Sprachkonstrukt, sowie mit einem neuen Modul-Dateiformat und verschiedenen Tools zum Teil lange erwartete Neuerungen bringt. Vor allem die Möglichkeit, unterschiedliche Versionen der Module zu definieren und bei Bedarf gleichzeitig zu laden, soll Erleichterung im Alltag von größeren Projekten bringen.

## Vom Unterschied, der den Unterschied macht

Eine neues Programmier-Paradigma ..... 31
von Nicole Delong

Vom ersten simplen Rechenknecht bis heute entwickelten sich unsere Computer auf rasante Art und Weise zu schnellen, leistungsfähigen, hochkompatiblen Maschinen. Kompliment, doch eines wurden sie leider nie: schlauer.

## Selbststudium für Computer

Skizze einer Theorie zum autonomen, maschinellen Wissenserwerb ..... 33
von Wolf-Rüdiger Delong

Seit es Maschinen gibt träumt der Mensch davon eine zu bauen, die ihm ähnlich ist und ihm lästige Arbeit abnehmen kann. Seit es Computer gibt scheint die Verwirklichung in greifbare Nähe gerückt zu sein. Auch wenn sich die Prognosen aus den Anfängen der Künstlichen Intelligenz nicht bewahrheitet haben, hat man dennoch große Fortschritte gemacht. Aber trotzdem haben alle Ansätze die Schwäche, dass der Mensch dem Computer den Rahmen vorgibt. Dieser Artikel skizziert ein Modell, in dem sich der Computer seinen Rahmen selbst erarbeiten kann.

## Destruktive Robustheit

Wie gut gemeinte Fehlervermeidung das Leben erschwert ..... 36
von Jan Lessner

Robustheit bezeichnet die Fähigkeit eines Systems, mit Ausnahmesituationen klar zu kommen. In übertriebener Form ist diese Eigenschaft aber hinderlich, weil sie Fehlerfälle verschleiert und die Analyse erschwert. Dieser Artikel zeigt an ein paar einfachen Beispielen wie schnell das passieren kann.

# Beitragsinformation

Der KaffeeKlatsch dient Entwicklern, Architekten, Projektleitern und Entscheidern als Kommunikationsplattform. Er soll neben dem Know-how-Transfer von Technologien (insbesondere Java und .NET) auch auf einfache Weise die Publikation von Projekt- und Erfahrungsberichten ermöglichen.

## Beiträge

Um einen Beitrag im KaffeeKlatsch veröffentlichen zu können, müssen Sie prüfen, ob Ihr Beitrag den folgenden Mindestanforderungen genügt:

- Ist das Thema von Interesse für Entwickler, Architekten, Projektleiter oder Entscheider, speziell wenn sich diese mit der Java- oder .NET-Technologie beschäftigen?
- Ist der Artikel für diese Zielgruppe bei der Arbeit mit Java oder .NET relevant oder hilfreich?
- Genügt die Arbeit den üblichen professionellen Standards für Artikel in Bezug auf Sprache und Erscheinungsbild?

Wenn Sie uns einen solchen Artikel, um ihn in diesem Medium zu veröffentlichen, zukommen lassen, dann übertragen Sie Bookware unwiderruflich das nicht exklusive, weltweit geltende Recht

- diesen Artikel bei Annahme durch die Redaktion im KaffeeKlatsch zu veröffentlichen
- diesen Artikel nach Belieben in elektronischer oder gedruckter Form zu verbreiten
- diesen Artikel in der Bookware-Bibliothek zu veröffentlichen
- den Nutzern zu erlauben diesen Artikel für nicht-kommerzielle Zwecke, insbesondere für Weiterbildung und Forschung, zu kopieren und zu verteilen.

Wir möchten deshalb keine Artikel veröffentlichen, die bereits in anderen Print- oder Online-Medien veröffentlicht worden sind.

Selbstverständlich bleibt das Copyright auch bei Ihnen und Bookware wird jede Anfrage für eine kommerzielle Nutzung direkt an Sie weiterleiten.

Die Beiträge sollten in elektronischer Form via E-Mail an redaktion@bookware.de geschickt werden.

Auf Wunsch stellen wir dem Autor seinen Artikel als unveränderlichen PDF-Nachdruck in der kanonischen KaffeeKlatsch-Form zur Verfügung, für den er ein unwiderrufliches, nicht-exklusives Nutzungsrecht erhält.

## Leserbriefe

Leserbriefe werden nur dann akzeptiert, wenn sie mit vollständigem Namen, Anschrift und E-Mail-Adresse versehen sind. Die Redaktion behält sich vor, Leserbriefe – auch gekürzt – zu veröffentlichen, wenn dem nicht explizit widersprochen wurde.

Sobald ein Leserbrief (oder auch Artikel) als direkte Kritik zu einem bereits veröffentlichten Beitrag aufgefasst werden kann, behält sich die Redaktion vor, die Veröffentlichung jener Beiträge zu verzögern, so dass der Kritisierte die Möglichkeit hat, auf die Kritik in der selben Ausgabe zu reagieren.

Leserbriefe schicken Sie bitte an leserbrief@bookware.de. Für Fragen und Wünsche zu Nachdrucken, Kopien von Berichten oder Referenzen wenden Sie sich bitte direkt an die Autoren.

## Werbung ist Information

Firmen haben die Möglichkeit Werbung im KaffeeKlatsch unterzubringen. Der Werbeteil ist in drei Teile gegliedert:

- Stellenanzeigen
- Seminaranzeigen
- Produktinformation und -werbung

Die Werbeflächen werden als Vielfaches von Sechsteln und Vierteln einer DIN-A4-Seite zur Verfügung gestellt.

Der Werbeplatz kann bei Frau Alexandra Specht via E-Mail an anzeigen@bookware.de oder telefonisch unter 09131/8903-14 gebucht werden.

## Abonnement

Der KaffeeKlatsch erscheint zur Zeit monatlich. Die jeweils aktuelle Version wird nur via E-Mail als PDF-Dokument versandt. Sie können den KaffeeKlatsch via E-Mail an abo@bookware.de oder über das Internet unter www.bookware.de/abo bestellen. Selbstverständlich können Sie das Abo jederzeit und ohne Angabe von Gründen sowohl via E-Mail als auch übers Internet kündigen.

Ältere Versionen können einfach über das Internet als Download unter www.bookware.de/archiv bezogen werden.

Auf Wunsch schicken wir Ihnen auch ein gedrucktes Exemplar. Da es sich dabei um einzelne Exemplare handelt, erkundigen Sie sich bitte wegen der Preise und Versandkosten bei Alexandra Specht via E-Mail unter alexandra.specht@bookware.de oder telefonisch unter 09131/8903-14.

## Copyright

Das Copyright des KaffeeKlatschs liegt vollständig bei der Bookware. Wir gestatten die Übernahme des KaffeeKlatschs in Datenbestände, wenn sie ausschließlich privaten Zwecken dienen. Das auszugsweise Kopieren und Archivieren zu gewerblichen Zwecken ohne unsere schriftliche Genehmigung ist nicht gestattet.

Sie dürfen jedoch die unveränderte PDF-Datei gelegentlich und unentgeltlich zu Bildungs- und Forschungszwecken an Interessenten verschicken. Sollten diese allerdings ein dauerhaftes Interesse am KaffeeKlatsch haben, so möchten wir diese herzlich dazu einladen, das Magazin direkt von uns zu beziehen. Ein regelmäßiger Versand soll nur über uns erfolgen.

Bei entsprechenden Fragen wenden Sie sich bitte per E-Mail an copyright@bookware.de.

## Impressum

KaffeeKlatsch Jahrgang 1, Nummer 1, Januar 2008
Bookware – eine Initiative der MATHEMA Software GmbH
ISSN 1865-682X

Henkestraße 91, 91052 Erlangen
Telefon: 0 91 31 / 89 03-0
Telefax: 0 91 31 / 89 03-55
E-Mail: redaktion@bookware.de
Internet: www.bookware.de

Herausgeber/Redakteur: Michael Wiedeking
Anzeigen: Alexandra Specht
Grafik: Nicole Delong

# Alles oder nichts

## Verteilte Transaktionen

von Thomas Haug

In den meisten ernst zu nehmenden IT-Systemen müssen Daten auf die eine oder andere Art dauerhaft gespeichert werden. Hierfür haben sich Transaktionen nach dem so genannten ACID-Prinzip etabliert.

Dieser erste von drei Teilen beschäftigt sich mit den Grundlagen von Commit-Protokollen, über die sich verteilte Transaktionen realisieren lassen.

### Verteilte Anwendungen

In diesem ersten Teil wird der Grundstein für die kommenden beiden Artikel gelegt und in die Theorie der verteilten Commit[1]-Protokolle eingeführt. Der nächste Teil wird eine konkrete Implementierung verteilter Transaktionen mittels Windows Communication Foundation (WCF) betrachten. Im dritten Teil sollen dann abschließend die Klippen der transaktionalen Interoperabilität von Web-Services sicher umschifft werden können.

### Begriffsklärung

Bevor wir uns den Untiefen verteilter Transaktionen zuwenden, ist es notwendig einige Begriffe festzulegen.

Müssen Änderungen an Ressourcen wie Datenbanken nach dem „Alles-oder-Nichts"-Prinzip durchgeführt werden, so haben sich hierfür Transaktionen nach dem ACID Prinzip etabliert:

- Sie sind atomar (**a**tomic), d.h. falls sie durch einen Fehler unterbrochen werden, werden alle in ihnen vorgenommene Änderungen rückgängig gemacht. Man spricht in diesem Zusammenhang vom Zurückrollen (Rollback) einer Transaktion.
- Sie erzeugen ausschließlich konsistente (**c**onsistent) Ergebnisse.
- Sie sind isoliert (**i**solated), d.h. alle ihre Zwischenzustände sind für andere Transaktionen nicht sichtbar.
- Ihre Ergebnisse sind dauerhaft (**d**urable).

[1] commit (engl.): begehen, festlegen, sich binden, verpflichten, anvertrauen, …

Ist nun mehr als eine transaktionale Ressource in einer solchen ACID-Transaktion involviert, werden sogenannte verteilte Transaktionen eingesetzt.

Ein Beispiel für eine verteilte Transaktion ist das folgende Szenario: In einem Buchungssystem soll nach Speicherung der Bestellung eines Kunden in der Datenbank eine Nachricht (über einen Message-Service) an das ausgelagerte Rechnungssystem erfolgen. Hierbei ist zu beachten, dass ausschließlich eine erfolgreiche Speicherung der Bestellung in die Datenbank die Nachricht an das Rechnungssystem auslösen soll. Schlägt das Speichern der Bestellung in die Datenbank fehl, so darf auf keinen Fall die Nachricht an das Rechnungssystem gesendet werden (welcher Kunde zahlt schon gerne eine Rechnung ohne etwas geliefert zu bekommen). Genauso soll die Speicherung der Bestellung in die Datenbank nur dann abgeschlossen werden, wenn dem Message-Service-Broker die Nachricht übermittelt werden konnte (wir wollen auch keine Waren versenden, ohne hierfür Geld zu bekommen).

Verteilte Transaktionen werden üblicherweise über ein „Two-Phase-Commit"-(2PC)-Protokoll realisiert, z.B. in CORBA, .Net und Java (EE).

### Das Two-Phase-Commit

Das 2PC Protokoll gehört zur Familie der verteilten Commit-Protokolle und wurde erstmalig von Jim Gray beschrieben [1]. Im Gegensatz zu seiner Beschreibung und den Ausführungen in „Advanced Concepts in Operating Systems" [2] werden im Folgenden nicht Begriffe wie Kohorten und Koordinator benutzt, sondern die bekannteren Begriffe aus der XA-Spezifikation der Open Group [3] verwendet.

Um eine verteilte Transaktion zu unterstützen, bedarf es einer Reihe an Komponenten (siehe Klassendiagramm in Abbildung 1).

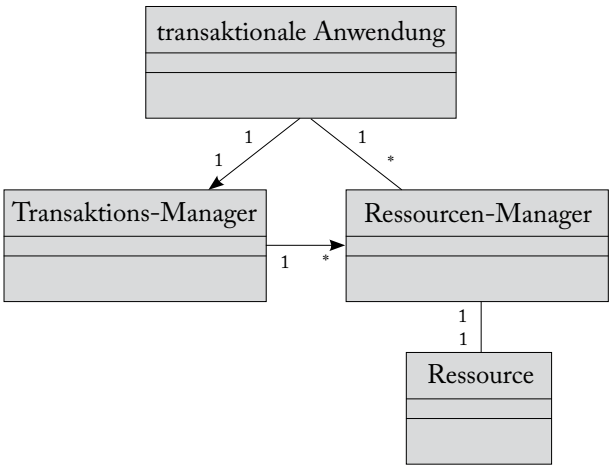



Abb. 1: Komponenten einer verteilten Transaktion

*Transaktionale Anwendung*

Die transaktionale Anwendung veranlasst die verteilte Transaktion, d.h. sie startet und beendet (committet[2]) die Transaktion über ein entsprechendes API am Transaktionsmanager. Änderungen an den Ressourcen führt die Anwendung über die von den Ressourcen-Managern bereitgestellten Schnittstellen durch.

*Transaktionsmanager*

Der Transaktionsmanager fungiert als Koordinator der verteilten Transaktion für die Anwendung und die Ressourcen-Manager, die für die Anwendung auf persistent gehaltenen Daten zugreifen (z.B. Datenbank, persistente Message-Service-Queues).

*Ressourcen-Manager*

Ein Ressourcen-Manager (RM) unterstützt verteilte Transaktionen durch die Implementierung des sog. Transaction Ressource Interfaces (TRI) und kann somit an Transaktionen teilnehmen, die von einem externen Transaktionsmanager gesteuert werden. Das XA-Protokoll ist ein standardisiertes TRI und wird von allen namhaften Datenbankherstellern und Message Brokern (z. B. Microsoft MessageQueue, IBM MQ und vielen JMS Implementierungen) unterstützt. Diese Schnittstelle bietet einen standardisierten Zugriff auf die Transaktionsmechanismen des Ressourcen-Managers, indem von den produktspezifischen Mechanismen abstrahiert wird.

## Durchführen einer verteilten Transaktion

Die transaktionale Anwendung initiiert die verteilte Transaktion, indem sie dem Transaktionsmanager die Nachricht *beginTransaction* zusendet. Im weiteren Verlauf der soeben geöffneten Transaktion ruft die Anwendung verschiedene (insbesondere modifizierende) Operationen an involvierten Ressourcen-Managern auf. Alle an der Transaktion beteiligten Ressourcen-Manager müssen dem Transaktionsmanager bekannt gegeben werden, um beim Beenden der Transaktion durch den Transaktionsmanager benachrichtigt zu werden. Nachdem die transaktionale Anwendung alle ihre Änderungen an den Ressourcen-Managern durchgeführt hat, teilt sie ihren Wunsch die durchgeführten Änderungen dauerhaft zu speichern dem Transaktionsmanager durch die Nachricht *commitTransaction* mit. Dieser initiiert nun das 2PC.

## Die *prepare*-Phase (Phase I)

Der Transaktionsmanager sendet an alle beteiligten Ressourcen-Manager eine *prepare*-Nachricht und wartet dann auf deren Antwort. Sobald ein Ressourcen-Manager eine solche Nachricht erhält, prüft er, ob die lokale Transaktion erfolgreich abgeschlossen werden kann. In diesem Fall wird in den lokalen Transaktions-Log entsprechende Informationen geschrieben, um die Transaktion rückrollen zu können (UNDO) und einen entsprechende Eintrag, um im „Fall der Fälle" (Absturz des Ressourcen-Managers) die Aktion wiederholen zu können (REDO).

Zum Abschluss sendet er eine *voteCommit*-Antwort an den Transaktionsmanager. Nachdem ein Ressourcen-Manager auf die *prepare*-Nachrichten mit *voteCommit* geantwortet hat, hat er sein Anrecht eine Transaktion selbstständig abzuschließen abgegeben und muss auf die endgültig Entscheidung des Transaktionsmanager warten (dies ist ein wichtiger Aspekt, auf den wir im kommenden Abschnitt zurückkommen werden).

Kann der Ressourcen-Manager die lokale Transaktion nicht erfolgreich abschließen, so müssen keine Log-Informationen in das Transaktions-Log geschrieben werden und der Ressourcen-Manager wird mit ein *voteRollback* auf die *prepare*-Nachricht antworten.

## Die *commit*-Phase (Phase II)

Sobald der Transaktionsmanager von allen beteiligten Ressourcen-Managern die Antwort auf die *prepare*-Nachricht eingesammelt hat, kann er eine Entscheidung über den Ausgang der verteilten Transaktion treffen und somit die zweite Phase einleiten. Haben alle Ressourcen-Manager mit *voteCommit* geantwortet, so schreibt der Manager einen commit-Eintrag in seine Log-Datei und sendet dann eine *commit*-Nachricht an alle Ressourcen-Manager. Sollten ein oder mehrere Ressourcen-Manager mit *voteRollback* geantwortet haben, so schreibt der Transaktionsmanager einen Rollback-Eintrag in seine Log-Datei und sendet eine *rollback*-Nachricht an die Beteiligten.

Haben die Ressourcen-Manager die Entscheidung empfangen, müssen sie nur im Fall einer *rollback*-Nachricht ihre Änderungen mittels der gespeicherten UNDO-Information rückgängig machen. Im Falle einer *commit*-Nachricht reicht es, gehaltene Sperren auf die genutzten Ressourcen freizugeben.

Abschließend antworten die Ressourcen-Manager mit eine Bestätigungsnachricht an den Transaktionsmanager. Sobald dieser von allen diese Bestätigung erhalten hat, muss er die Transaktion als abgeschlossen in seiner Log-Datei markieren und das Ergebnis der transaktionalen Anwendung mitteilen.

Dieser Ablauf ist im Sequenzdiagramm (Abbildung 2) dargestellt (ohne das Schreiben von Log-Informationen und das Anmelden der Ressourcen-Manager am Transaktionsmanager zu zeigen).

[2] Das „committen" findet sich tatsächlich schon im Duden als „sich bekennen, verpflichten".

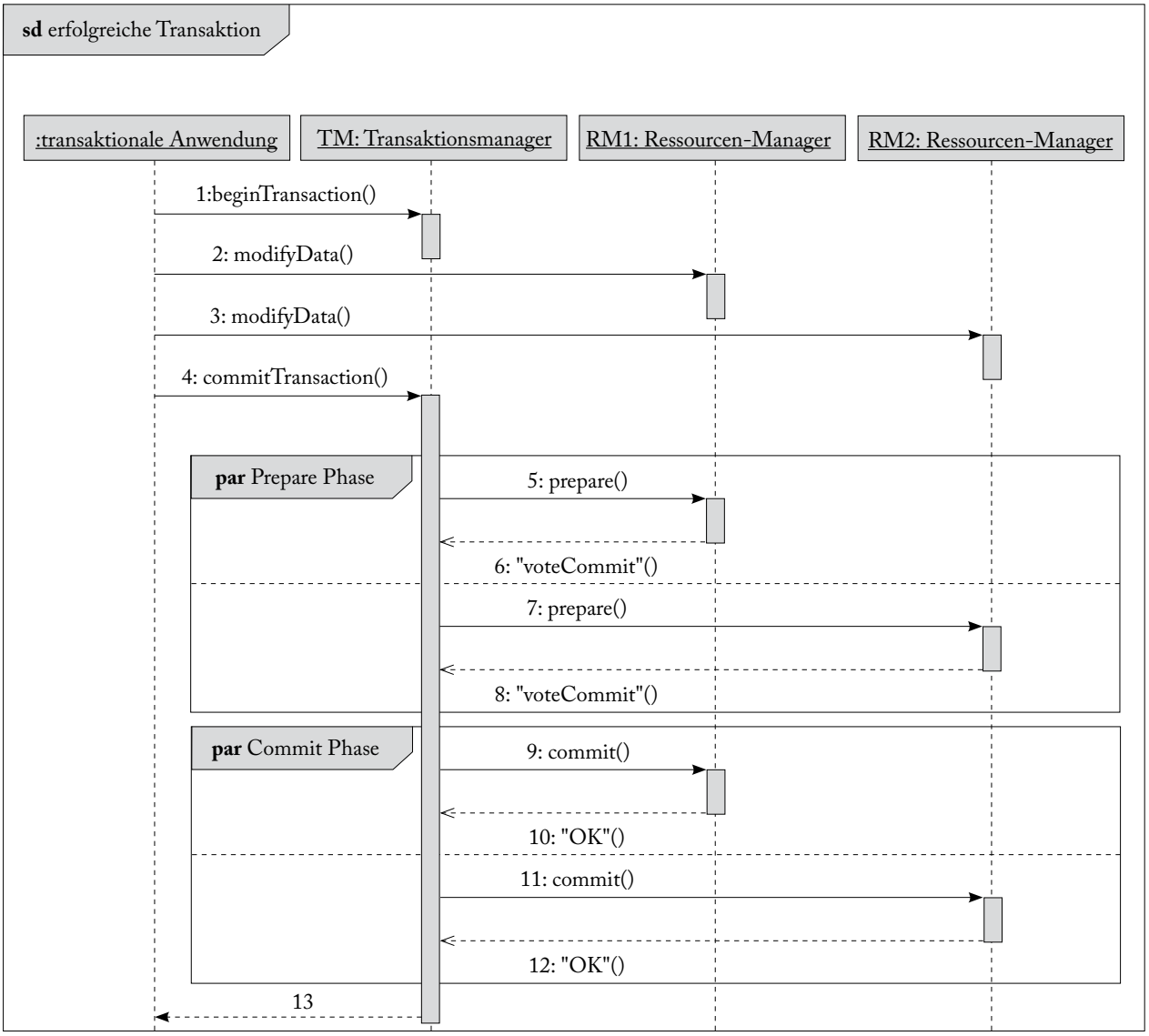



Abb. 2: Verteilte Transaktion mit zwei Ressourcen-Managern

## Und wenn es mal schief geht…

Verlassen wir nun die Wohlfühlzone und begeben uns in die Realität der vernetzen Computer. Wie der Name „verteilte Transaktion“ schon suggeriert, können die Beteiligen über Prozess- und Rechnergrenzen hinweg verteilt sein. Dies bedeutet auch, dass eine neue Fehlerqualität bei diesen Transaktionen auftreten kann. So können einzelne Knoten ausfallen oder das Netzwerk sich von einem „Network“ in ein „Notwork“ verwandeln. Die wichtigsten dieser Fehlersituationen sollen nun betrachtet werden. Eine ausgiebige Betrachtung möglicher Fehlersituationen findet sich in [4].

## Ausfall eines Ressourcen-Managers

Sollte dieser Fall in der *prepare*-Phase (I) eintreten, so wird die Transaktion durch den Transaktionsmanager abgebrochen, indem an alle (ggf. übrigen) Ressourcen-Manager eine *rollback*-Nachricht gesandt wird. (Hierzu ist es aber notwendig, dass der Transaktionsmanager mittels Time-outs feststellen kann, dass ein Ressourcen-Manager nicht fristgerecht antwortet.)

Ist die Transaktion bereits in der *commit*-Phase (II) angelangt, so wird die Situation etwas schwieriger. Sobald der Ressourcen-Manager wieder gestartet wurde, muss er den Zustand der verteilten Transaktion beim Transaktionsmanager erfragen und seine Änderungen entsprechend nachziehen.

## Ausfall des Transaktionsmanagers

Sollte der Transaktionsmanager vor dem Schreiben der *commit*-Nachricht in der *commit*-Phase (II) das Zeitliche segnen, so sendet er bei Wiederanfahren eine *rollback*-Nachricht an alle Beteiligten.

Sollte der Transaktionsmanager zwischen dem Schreiben der *commit/rollback*-Information in seinen stabilen Speicher und dem Schreiben der complete-Information in das Log abstürzen, so wiederholt er das Versenden der *commit*- bzw. *rollback*-Nachricht und die Ressourcen-Manager können entsprechend handeln.

Bei diesem Fehlerfall zeigt sich auch der größte Nachteil des klassischen 2PC-Protokolls: die beteiligten Ressourcen-Manager müssen beim Ausfall des Transaktionsmanager auf dessen erneute Verfügbarkeit warten, um die ACID Eigenschaften der Transaktion zu wahren. Es handelt sich also um ein blockierendes Commit-Protokoll. Dies ist in vielen Situationen nicht akzeptabel, da die durch die Ressourcen-Manager verwalteten Ressourcen wie Datenbanken solange gesperrt sein müssen. Aus diesem Grund wird durch Optimierungen am Protokoll oder durch deren Erweitertung (z. B. 3-Phase-Commit [2]) versucht, diese Blockade zu vermeiden.

## Mutmaßende Teilnehmer

Die XA-Protokollvariante versucht durch Heuristiken das Blockieren der Ressourcen-Manager zu vermeiden. Erhält ein Ressourcen-Manager nicht innerhalb einer bestimmten Zeitspanne zwischen Senden der *voteCommit*- bzw. *vote-Rollback*-Nachricht eine Entscheidung, d. h. eine *commit*- bzw. *rollback*-Nachricht, so kann er eigenmächtig aufgrund seines Zustands eine Entscheidung treffen und seine lokale Transaktion für sich (mit einem *commit* oder *rollback*) beenden. Jedoch ist er gezwungen, diese heuristische Entscheidung in seiner Log-Datei zu speichern, um sie später dem Transaktionsmanager mitteilen zu können.

Wenn der Transaktions-Manager nun wieder erwacht und eine Verbindung zu den Ressourcen-Managern aufbauen kann, so wird er diese heuristische Entscheidung mitgeteilt bekommen und kann sie der transaktionalen Anwendung mitteilen. Die Antwort an die Anwendung kann folgende Szenarien beschreiben:

- *Heuristischer Rollback*: alle Ressourcen-Manager haben sich eigenmächtig für einen Rollback entschieden,
- *Heuristisches Commit:* alle Ressourcen-Manager haben sich eigenmächtig für ein Commit entschieden,
- *Heuristischer Mix*: einige Ressourcen-Manager haben sich für Commit und andere für Rollback entschieden,
- *Heuristischer Hazard*: Der Ausgang der verteilten Transaktion ist unbekannt.

Während die ersten beiden Fälle noch recht harmlos sind, ist in den letzten beiden Fällen das System in einen inkonsistenten Zustand geraten und muss durch administrative Werkzeuge wieder bereinigt werden. Gerade die Vermeidung solcher Zustände und die Unterstützung in einem solchen Fall unterscheidet gute Transaktionssysteme von schwächeren.

## Fazit

Wir haben nun einen Überblick über verteilte Transaktionen erhalten. Aus der Beschreibung der 2PC-Protokolls lässt sich der notwendige Kommunikationsaufwand für eine verteilte Transaktion ersehen (und die damit drohenden Performanzprobleme). Mögliche Optimierungen (wie *vote-ReadOnly* als Antwort auf die prepare-Nachricht, das Umschalten auf eine lokale Transaktion bei nur einem beteiligten Ressourcen-Manager, das Zwischenschalten (Interposition) von Transaktions-Managern [5] und das Recovery wurden bewusst in diesem Artikel ausgelassen.

Leider bietet das 2PC-Protokoll keinen 100 %-igen Schutz vor Inkonsistenten, d. h. das „Alles oder Nichts" gilt nicht zwangsläufig bei verteilten Transaktionen. Insbesondere kann es sogar zum Blockieren der Anwendung oder der beteiligten Ressourcen-Managern (und Ressourcen) führen. Aus diesem Grund sollte bei der Auswahl eines Transaktionsmanagers grundsätzlich dessen Verhalten mit den eingesetzten Ressourcen-Managern überprüft werden, um die Lebendigkeit und die Konsistenz des Systems zu garantieren oder zumindest die Problembereiche im Zusammenspiel zu kennen.

### Referenzen


[1] Gray, J. et al. *Operating Systems, An Advanced Course,* in Lecture Notes in Computer Science, Volume 60, Springer, 1978, (http://research.microsoft.com/~Gray/papers/DBOS.pdf)

[2] Singhal, M.; Shivaratri, N. *Advanced Concepts in Operating Systems,* McGraw-Hill, New York, 1994

[3] The Open Group *Distributed Transaction Processing: The XA Specification,* http://www.opengroup.org/onlinepubs/009680699/toc.pdf

[4] Kudlich, H. *Verteilte Datenbanken. Systemkonzepte und Produkte,* Siemens, Berlin, 1992

[5] Little, M.; Maron, J.; Pavlik, G. *Java Transaction Processing: Design and Implementation,* Prentice Hall, 2004


### Kurzbiographie




Thomas Haug (thomas.haug@mathema.de) ist als Consultant und Trainer für die MATHEMA Software GmbH in Erlangen tätig. Seine Themenschwerpunkte umfassen die Java Standard und Enterprise Edition (Java SE und Java EE) sowie das .NET-Umfeld, insbesondere im Hinblick auf Enterprise-Anwendungen. Neben seiner Projekttätigkeit hält er Technologietrainings für MATHEMA und berät verschiedene Projekte hinsichtlich des optimalen Einsatzes von .NET- oder Java-Technologien.



Copyright © 2008 Bookware 1865-682x/01/08/001 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

# Echt abgefahren!

## Einführung in Groovy

von Falk Sippach

Für die Virtuelle Maschine von Java (JVM) gibt es mehr als zweihundert Programmiersprachen. Im Jahre 2003 hat sich auch Groovy aufgemacht sich als sogenannte dynamische Skriptsprache einen Platz in diesem hart umkämpften Markt zu erobern. Dabei glänzt Groovy nicht nur mit einer prägnanten Syntax und dynamischen Eigenschaften, sondern auch mit der perfekten Integration mit Java. Dieser Artikel gibt eine kurze Einführung in die wichtigsten Features dieser modernen Programmiersprache.

### Wo Groovy herkommt

Javas Erfolg der letzten Jahre ist unbestritten. Er beruht allerdings weniger auf der eigentlichen Sprache und deren Syntax, sondern vielmehr auf der hohen Qualität und der großen Verbreitung seiner Plattform, der Java Virtual Machine (JVM). Die gute Abstraktion der darunter liegenden Maschinen-Architektur ermöglicht die Entwicklung und den stabilen, effizienten Einsatz von Komponenten, Frameworks und Services auf den unterschiedlichsten Betriebssystemen.

Java ist verglichen zu Sprachen wie Ruby, Python, Lisp oder Smalltalk weniger elegant und ausdrucksstark. Groovy versucht diese Lücke zu schließen, in dem es beide Welten zusammenbringt. Mit Groovy wurde übrigens erstmals eine weitere Sprache nach Java für die JVM standardisiert (JSR 241) [1].

Das Groovy-Projekt wurde 2003 von James Strachan und Bob McWriter mit der Intention gegründet, die Eleganz von Ruby auf die Java-Plattform zu bringen. Im Januar 2007 erschien das finale Release 1.0 und mittlerweile steht bereits Version 1.1 in den Startlöchern. Das Open-Source-Projekt ist bei Codehaus gehostet [2] und wird von Guillaume Laforge geleitet.

Es gibt mittlerweile mehrere Bücher, wobei „Groovy in Action“ von Dierk König et al. [3] als das Standardwerk bezeichnet werden kann. Zu besonderen Ehren kam Groovy im Mai 2007, als ihm der JAX Innovation Award verliehen wurde.[1]

### Was Groovy ausmacht

Es gibt zwei Begriffe, mit denen Groovy immer in Verbindung gebracht wird: dynamisch und Skriptsprache.

Unter einem Skript versteht man ein Programm, das in einem Interpreter läuft, und zwar gewöhnlicherweise ohne separaten Kompilierungsschritt. Wenn man es genau nimmt, ist Groovy nach dieser Definition gar keine richtige Skriptsprache, da der Code vor dem Ausführen immer erst in Java-Bytecode kompiliert wird. Groovy fühlt sich aber wie eine Skriptsprache an, da die Kompilierung unbemerkt hinter den Kulissen erfolgt. Man hat aber auch explizit die Möglichkeit, die Groovy Quellen in Bytecode zu übersetzen.

Da sich Java- und Groovy-Bytecode nicht unterscheiden, lässt sich bereits die enge Integration zwischen den beiden Sprachen erahnen. Es können sowohl gegenseitig Objekte aufgerufen (Groovy nutzt und erweitert Javas mächtige Klassenbibliothek), als auch Tools wie Profiler, Debugger etc. ohne Einschränkung für Groovy verwendet werden.

Dynamisch bedeutet, dass Typen von Variablen sowie Verhalten von Objekten erst zur Laufzeit ermittelt werden und bereits kompilierter Code im Stil der Aspektorientierten Programmierung (AOP) um neue Funktionalitäten erweitert werden kann. Für den letzten Punkt ist das Meta Object Protocol (MOP) zuständig. Dadurch grenzt sich Groovy von vielen anderen sogenannten dynamischen Skriptsprachen ab, die nämlich nur das dynamische Typisieren unterstützen.

### Hallo Welt

Doch wie sieht die Syntax nun eigentlich aus? Als erstes Beispiel ist das übliche Hello-World-Beispiel als Groovy-Skript abgedruckt.

```
println 'Hello Groovy-World!'
```

Dem gegenüber steht die Java-Variante dieser simplen Anwendung.

```
public class SayHello {
  public static void main(String[] args) {
    System.out.println("Hello Java-World!");
  }
}
```

[1] Zum Vergleich, im Jahr zuvor ging diese Auszeichnung an das Spring Framework.

Zunächst fällt auf, dass der Quellcode von Groovy statt fünf nur eine Zeile umfaßt. Die Syntax scheint also kürzer zu sein, doch ist sie auch ausdrucksstärker und prägnanter? Da es sich um eine Art Skriptsprache handelt, muss man keine Klassen oder Programmeinstiegspunkte (main-Methode) schreiben, wie man dies von Java gewöhnt ist.

Allerdings ist Groovy genau wie Java eine objektorientierte Sprache. Es geht sogar noch einen Schritt weiter, denn es kennt im Gegensatz zu Java keine primitiven Datentypen sondern nur Objekte. Groovy-Skripte werden, versteckt für den Entwickler, durch den Compiler automatisch in Klassen eingepackt und mit einer mainMethode versehen.

## In der Kürze …

Beim nächsten Hinschauen fallen weitere Unterschiede zwischen der einen Zeile Groovy-Code und der entsprechenden Java-Zeile auf. Die Methode ‚println' ist eine Art Alias für das längere Java-Konstrukt ‚System.out.println' und tut genau das, was man erwarten würde. Des weiteren sollten die fehlenden Klammern und das nicht existierende Semikolon ins Auge springen.

Und damit kommen wir gleich zu einem Grundsatz: Kurze und leserliche Syntax. Daraus ergeben sich einige Änderungen zu Java.

Zum einen importiert Groovy automatisch eine Vielzahl von oft verwendeten Klassen aus den Paketen groovy.lang, groovy.util, java.lang, java.util, java.net, java.io, und java.math. Java tut dies nur für java.lang-Paket.

Desweiteren können Zeichen weggelassen werden, die für den Compiler zur Interpretation des Codes nicht unbedingt nötig sind. Es mag für erfahrene Java-Entwickler anfänglich etwas eigenartig aussehen, wenn Klammern, Semikolons, return-Anweisungen, Exception-Deklarationen und public-Schlüsselwörter fehlen. Aber nach einer Eingewöhnungsphase lernt man den kürzeren Code schätzen.

## Das Detail macht den Unterschied

Trotzdem ist das Hello-World-Beispiel in Java auch ein korrektes Groovy-Programm. Die vielen Gemeinsamkeiten zwischen den beiden Sprachen lassen den Gedanken aufkommen, dass Java-Code kompatibel zu Groovy ist. Dies ist allerdings nur bedingt richtig. Vielmehr ist die Syntax von Groovy lediglich an die von Java angelehnt.

So fehlt zum Beispiel das Konstrukt der inneren Klassen. Nicht zu vergessen ist auch die geänderte Bedeutung gewisser Sprachkonstrukte. So sind Attribute, die in Java als „package private" deklariert sind (kein Access Modifier), in Groovy Properties (Getter und Setter). Diese wenigen Unterschiede verhindern die direkte Portierung von Java-Code. Trotzdem überwiegen die grundsätzlichen Gemeinsamkeiten zwischen den beiden Sprachen und erleichtern Java-Entwicklern den Einstieg in Groovy deutlich.

In den grundlegenden Konzepten folgt Groovy nämlich dem Vorbild und bietet zusätzlich vereinfachte Schreibweisen und nette Zusätze an (syntaktischer Zucker). Kommentare, Pakete, Imports, Klassen, Methoden, Felder und Initialisierungsblöcke sind in ihrer einfachsten Erscheinungsform identisch mit Java.

Für Imports gibt es zusätzlich ein Type-Aliasing in der Form:

```
import java awt.BorderLayout as BL
// […]
def borderLayout = new BL()
```

Statische Typisierung von Attributen, Methodenparametern und Rückgabewerten kann entfallen, dass heißt, es wird erst zur Laufzeit geprüft, ob ein übergebener Parameter gültig ist. Groovy stellt dafür das Schlüsselwort **def** zur Verfügung. Übrigens wird die Referenz unter der Haube sehr wohl typisiert und zwar als java.lang.Object.

## Duck-Typing

Zur Laufzeit verwendet Groovy dann das sogenannte Duck-Typing, um Methoden auf untypisierten Referenzen aufzurufen. Das Prinzip dahinter ist einfach erklärt: Alles was wie eine Ente aussieht, wie eine Ente watschelt und wie eine Ente quakt, muss eine Ente sein.

Damit Groovy eine bestimmte Methode aufrufen kann, muss diese also nur an dem entsprechenden Objekt vorhanden sein, andernfalls würde ein Laufzeitfehler auftreten. Im unten stehenden Beispiel interessiert Groovy zur Übersetzungszeit noch nicht, ob Auto und BobbyCar eine Methode fahren besitzen. Erst zur Laufzeit wird festgestellt, ob beide Objekte fahren können.

```
class Auto {
  def fahren() {
    println 'Auto fährt'
  }
}
class BobbyCar {
def fahren() {
  println 'BobbyCar fährt'
  }
}
def auto = new Auto()
def bobbyCar = new BobbyCar()
[auto, bobbyCar].each {it.fahren}
```

## Parameterübergabemechanismen

Für Methoden gibt es im Gegensatz zu Java weitere, viel flexiblere Möglichkeiten zur Argumentübergabe. Dazu zählen Parameter mit Vorgabewerten, variable Parameter (ähnliches Prinzip wie bei den variablen Argumentenlisten in Java 5), benannte Argumente und das Übergeben von Listen bzw. Arrays, die auf die entsprechenden Methodenparameter aufgesplittet werden (mit dem sogenannten Spread-Operator „*“).

Für Konstruktoren gibt es neben den eben aufgezählten noch zwei zusätzliche Varianten. Existiert ein (parameterloser) Default-Konstruktor, kann man ähnlich den benannten Argumenten bei Methoden eine Map übergeben. Groovy füllt dann automatisch den Inhalt der Map in die entsprechenden Instanz-Properties, wenn diese existieren und entsprechend heißen.

Die zweite Möglichkeit für Konstruktoren besteht in der Zuweisung einer Liste, welche die Konstruktor-Argumente in der richtigen Reihenfolge enthält.

```
class G {
  def a
  def b
  String s
  int i
  G() {
  }
  G(String s, int i, def a) {
    this.a = a
    this.s = s
    this.i = i
  }
  String toString() {
    "G: $a, $b, $s, $i"
  }
  def defaultArgs(int a, def b = new Date(), String c = 'abc') {
    "default args: $a, $b, $c"
  }
  def varArgs(int[] a) {
    "variable args: $a"
  }
  def namedArgs(Map args) {
    "named args: $args"
  }
}
def ge1 = new G()
assert 'G: null, null, null, 0' == ge1.toString()
def ge2 = new G('irgendwas', 4, new Date())
try {
  new G(new Date(), 'irgendwas, 4')
} catch (Exception e) {
  println 'Constructor doesn\'t exist.'
}
def ge3 = new G(a: 4, b: 3, i:5)
G ge4 = ['asdf', 3, new Date()]
assert 'default args: 3, http://grails.org, abc' ==
  ge1.defaultArgs(3, new URL('http://grails.org'))
assert 'variable args: [1, 2, 5, 6]' ==
  ge1.varArgs(1, 2, 5, 6)
assert 'named args: ["alter":35, "name":"Otto"]' ==
  ge1.namedArgs(name:'Otto', alter:35)
def args = [4, 'string1', 'string2']
assert 'default args: 4, string1, string2' ==
  ge1.defaultArgs(*args) // Spread-Operator
```

## Objekte erster Klasse

In einem Punkt grenzt sich Groovy jedoch deutlich von Java ab: Alles ist ein Objekt. Groovy verzichtet – wie bereits erwähnt – gänzlich auf primitive Datentypen und verwendet ausschließlich Referenzen auf Objekte. Statt **int** wird so immer das Objekt Integer verwendet. Im Code darf man allerdings auch weiterhin **int** schreiben.

In Java ist der Umgang mit Objekten oftmals sehr umständlich und wenig intuitiv. Groovy bietet an vielen Stellen direkte Sprachunterstützung, um die Verwendung oft genutzter Datentypen zu vereinfachen. Zu diesen Datentypen erster Klasse (First Class Citizens) gehören unter anderem Zahlen, Strings, reguläre Ausdrücke, Collections und Closures.

Im nachfolgenden Beispiel sieht man, wie einfach Listen erzeugt werden und wie anschließend auf einzelne Elemente zugegriffen werden kann.

```
def list = [1, 2, 3] // ArrayList mit 3 Werten
def emptyList = [] // leere ArrayList

assert 3 == list[1] + list[2]
```

## Überladen von Operatoren

Ein Grundpfeiler für die First Class Citizens ist das Überladen von Operatoren. In Java gab es damals gute Gründe, dieses Feature nicht zu implementieren. Aber seit jeher wird über die Vor- und Nachteile diskutiert. In Groovy wurde das Überladen sehr schön gelöst. Zu jedem Operator gibt es eine entsprechende Methode, die man einfach implementieren muss.

Um also zwei Objekte zu addieren, muss es beim Typ des ersten Summanden eine „plus()“-Methode geben, die als Parameter den Typ des zweiten Summanden aufnehmen kann. Auf diese Art können Objekte sehr einfach mittels Operatoren verbunden werden, wie man es nur von Javas primitiven Datentypen (1 + 2) und Zeichenketten ("foo" + "bar") kennt.

| Operation | Operator | Operator-Methode |
|---|---|---|
| Addition | a + b | a.plus(b) |
| Multiplikation | a * b | a.multiply(b) |
| Inkrementierung | a++ | a.next() |
| Indizierung | a[b] | a.getAt(b) |
| ... | | |

Das nächste Beispiel zeigt die Anwendung der Operatoren. Alle vier Zeilen sind inhaltlich gleich. Zu beachten ist, dass das Literal 1 in Groovy einer Referenz auf ein Integer-Objekt mit dem Wert 1 entspricht.

```
assert 2 == new Integer(1).plus(new Integer(1))
assert 2 == new Integer(1) + new Integer(1)
assert 2 == 1 + new Integer(1)
assert 2 == 1 + 1
```

## Zahlenspiele

Bei der Zuweisung von Zahlen verhält sich Groovy übrigens auch etwas anders, als man es von Java gewöhnt ist. Definiert man eine ganze Zahl, wird daraus ein Integer-Objekt erzeugt, so wie man es erwarten würde. Bei Gleitkommazahlen verwendet Groovy allerdings nicht Double sondern BigDecimal. Dadurch wird gerade bei der Entwicklung von finanzorientierter Software die Verwendung von gebrochen rationalen Werten stark vereinfacht.

Eine weitere Änderung ergibt sich bei der Division. Die ganze Zahl 1 geteilt durch die ganze Zahl 2 ergibt nun 0.5 statt 0, d. h. Groovy sucht automatisch den sinnvolleren Datentyp heraus. Will man bei der Definition von Zahlen spezielle Typen verwenden, stehen genau wie in Java die entsprechenden Präfixe zur Verfügung (1L … Long, 1F … Float, 1G … BigInteger, …).

## Zeichenketten

Groovy kennt verschiedene Möglichkeiten, Zeichenketten literal zu definieren. Java am ähnlichsten sind Strings in einfachen und doppelten Hochkommata.

In der zweiten Variante, den sogenannten GStrings, können zusätzlich Platzhalter verwendet werden. Allerdings werden die Platzhalter erst ausgewertet, wenn der GString als String verwendet wird. Das ermöglicht die Erstellung flexibler Vorlagen (Templates) oder auch die Erzeugung von echten Prepared Statements ohne die lästige Fragezeichen-Syntax (Groovy-SQL).
Groovys Zeichenkettenunterstützung geht aber noch weiter. Mehrzeilige Zeichenketten erhält man, indem die entsprechenden Hochkommata verdreifacht werden.

```
def s = 'ein einfacher String'
def gs = "$person.name ist ${person.getAlter} Jahre alt"
sql.execute "select from Person where name = $name"
def zweiZeilen = """erste Zeile
zweite Zeile"""
```

Nicht zuletzt erleichtert Groovy die Arbeit mit regulären Ausdrücken durch die „Slashy Syntax“. Wird ein String durch Slashes (Schrägstriche) begrenzt, entfällt das Escaping von Sonderzeichen, insbesondere dem bei regulären Ausdrücken sehr oft verwendeten Backslash.

```
"\\d\\d:\\d\\d" == /\d\d:\d\d/
```

## Reguläre Ausdrücke

Bei regulären Ausdrücken geht Groovys Unterstützung aber noch viel weiter. Es bietet die Möglichkeit aus einem String ein java.util.regex.Pattern-Objekt zu erstellen. Darüber hinaus stellt es einen find-Operator (=~) und einen match-Operator (==~) zu Verfügung. Ersterer erzeugt ein Matcher-Objekt (java.util.regex.Matcher) mit allen gefundenen Passagen, während der zweite abprüft, ob die ganze Zeichenkette dem gegebenen Muster entspricht.

```
if ('eine Uhrzeit: 12:23' =~ /\d\d:\d\d/) {
    println "enthält irgendwo mindestens eine Uhrzeit"
}
if ('zwei Worte' ==~ /\w+ \w+/) {
    println "besteht aus genau zwei Wörtern"
}
```

## Collections

Ein weiterer Stern am Groovy-Himmel ist die viel einfachere Verwendung der Collections. Ein Beispiel für die Listen wurde eben schon gezeigt. Für die Schlüssel-Werte-Paar Container (java.util.Map) gibt es ähnliche Deklarationsvarianten.

Um auf die Werte zuzugreifen gibt es wieder den getAt()-Operator ([]). Wenn als Schlüssel Strings verwendet werden, kann der Zugriff wie auf ein Attribut erfolgen.

```
def map = [:] // leere HashMap
map = ["eins":1, "zwei":2]
assert 1 == map["eins"]
map["drei"] = 3
assert 2 == map.zwei
map.vier = 4
```

Zusätzlich wurde mit dem Range ein ganz neuer Collection-Typ eingeführt. Hierbei handelt es sich um Intervalle (Wertebereiche), die durch eine linke und rechte Grenze definiert werden.

Die Verwendungen für Ranges sind vielfältig. Sie bieten einerseits eine intuitivere Art des Iterierens (z. B. mit der each-Methode). Andererseits kann man mit ihrer Hilfe sehr einfach auf Bereiche einer Liste zugreifen.

```
def range = 1..5                    // von 1 bis 5
def range = 5..3                    // Reverse Range: von 5 bis 3
for (i in 1..3) {
   println i
}                                   // Schleife von 1 bis 3
('a'..'e').each {
   print it
}                                   // gibt 'abcde' aus
(3..<10).each {
   print it
}                                   // -> 3456789, halbexklusiv
def substr = 'Kleiner Test'[0..6]   // Ergebnis: 'Kleiner'
list = [1, 2, 3, 4]
assert [1, 2] == list[0..1]
list[-3..-1] = ['a', 'b']           // Ersetzen der letzten 3
                                       Werte (-3..-1)
assert [1, 'a', 'b'] == list
```

## Die Wahrheit in Groovy

Die Steuerungsstrukturen funktionieren in Groovy nicht viel anders als in Java, können aber teilweise flexibler eingesetzt werden. Allerdings muss man vor ihrer Anwendung verstehen, wie in Groovy boolesche Ausdrücke verarbeitet werden. Die „Groovy Truth“ geht nämlich über simples **true** und **false** hinaus.

So evaluieren zum Beispiel Collections, Strings und Matcher (reguläre Ausdrücke) zu **true**, wenn sie nicht leer sind und zu **false** andernfalls. Zahlen werden als wahr angesehen, wenn ihr Wert ungleich 0 ist. Alle anderen Objekt-Referenzen sind wahr, wenn sie nicht auf null zeigen.

```
assert true
assert !false
assert [1, 2, 3] // wahr, da nicht leer
assert false == [] // leere Collection
assert 5 // Zahlen sind "wahr", wenn sie ungleich 0 sind
assert !(0)
assert new Date() // true, da ungleich null
assert !(null) // false, da gleich null
if (kunde) {
   println kunde.toString()
}
def i = 5 - 5
if (i) {
   assert false
}
```

## Bedingte Anweisung

Die bedingte Anweisung (if-else) funktioniert analog zu Java. Weitaus flexibler gibt sich dagegen die switch-Anweisung. Ihre case-Zweige arbeiten nicht nur mit primitiven ganzzahligen Datentypen und Enumerationen, wie man es von Java gewöhnt ist, statt dessen kann jedes Objekt verwendet werden.

Möglich wird dies durch die Methode isCase, die Groovy bei der Klasse Object zur Verfügung stellt (und für manche Typen überlädt, wie bei Class oder Collection).

```
def x = new Date()
switch(x) {
  case "Groovy" : println "in x steht der String Groovy"
    break
  case Date : println "x ist ein Datum"
    break
  case 1..10 : println "x ist Zahl zwischen 1 und 10"
    break
  default : println "x ist etwas anderes"
}
```

## Schleifen

Die Verwendung von Schleifen in Groovy unterscheidet sich nur im Detail zu Java. Die klassische for-Schleife gibt es allerdings erst seit Version 1.1. Das aus Java 5 bekannte foreach-Konstrukt sieht in Groovy ebenfalls etwas anders aus (Schlüsselwort **in** statt :).

Hinzu kommt, dass in Groovy mittels Closures sehr einfach Schleifen simuliert werden können, die zudem noch besser lesbar sind. Im folgenden Beispiel sind vier Variationen aufgeführt, die alle zum gleichen Ergebnis führen.

```
for (int i = 1; i <= 10; i++) {
   print i
}
for (i in 1..10) {
   print i
}
10.times {
   print it
}
(1..10).each {print it}
```

Dass es in Groovy keine primitiven Datentypen gibt, kann man sehr gut in der dritten Zeile sehen. Dort wird an dem Integer-Objekt mit dem Wert 10 die Methode times aufgerufen. Diese Methode ruft dann zehn Mal die als ersten und einzigen Parameter übergebene Closure

auf. Klammern bei Methodenaufrufen können weggelassen werden, die geschweiften Klammern gehören an dieser Stelle zur Definition der Closure[2].

## Closures

In Groovy gehören Closures bereits zu den wertvollsten Features. Ihre Verwendung hat nachhaltigen Einfluss auf den gesamten Programmierstil. Aus Java bekannte Design-Patterns werden dadurch entweder gar nicht benötigt oder lassen sich viel einfacher umsetzen.

Eine Closure ist ein wiederverwendbares Stück Code, das irgendwann definiert und erst zu einem späteren Zeitpunkt ausgeführt wird. Da es sich letzten Endes um ein Objekt handelt, kann es referenziert und zum Beispiel als Argument an eine Methode übergeben werden. Eine Closure hat allerdings im Gegensatz zu normalen Objekten keinen Zustand, sondern nur Verhalten. Es erinnert dadurch stark an die aus C bekannten Funktionszeiger.

Definiert werden Closures als Codeblöcke, die von geschweiften Klammern umschlossen sind. Sie können beim Aufruf Parameter erhalten und natürlich ein Ergebnis zurückliefern.

Aufpassen muss man bei der Sichtbarkeit von Variablen, die außerhalb der Closure definiert sind. So verweist das Schlüsselwort **this** in der Closure auf die umschließende Klasse und nicht auf die Closure selbst.

Wenn man eine Closure definiert hat, kann man sie mit der Methode call aufrufen und ggf. Parameter übergeben. Argumente werden direkt hinter der öffnenden Klammer deklariert und mit einem Abbildungs-Pfeil (->) vom Body abgetrennt. Wurden keine Argumente deklariert, dann gibt es zumindest den Standardparameter *it*.

```
def c = {
   println 'I am a closure ' + it
}
c.call('test')
('a'..'g').eachWithIndex {item, index ->
   println "$index: $item"
}
```

In Groovy wird sehr viel Gebrauch von Closures gemacht. Um Schleifen zu simulieren, kann, wie weiter oben beschrieben, auf einer Collection die each-Methode aufgerufen und ihr den Schleifenkörper in Form der Closure übergeben werden.

Des weiteren kommen Closures beim Event-Handling in der GUI-Programmierung zum Einsatz. Die Groovy-Klasse Closure, die hinter diesen wiederverwendbaren Codeblöcken steht, implementiert außerdem das Interface Runnable, so dass Closures als eigene Threads gestartet werden können.

## Properties und Attribute

Eine weitere nette Erweiterung erwartet Groovy-Entwickler bei der Verwendung von GroovyBeans. Sie stellen das Pendant zu den JavaBeans dar. Das sind Klassen, die nach bestimmten Konventionen programmiert sind. Eines der Hauptmerkmale der JavaBeans sind die privaten Member-Variablen mit zugeordneten Getter- und Setter-Methoden. Je nachdem, wie viele Attribute eine Klasse besitzt, wird es durch den Wildwuchs der vielen, meist generierten Getter und Setter sehr schnell unübersichtlich und lenkt von der eigentlich fachlichen Logik der Bean ab.

Groovy geht einen denkbar einfachen Weg. Attribute ohne Angaben zur Sichtbarkeit werden automatisch als Properties, also als private Member behandelt. Zusätzlich werden die entsprechenden get- und set-Methoden generiert. Wenn es benötigt wird, kann man natürlich eigene Zugriffsmethoden implementieren.

```
class Person {
   long id;
   String firstName;
   String lastName;

   String getEmail() {
      'info@oio.de'
   }

   // Geschäftsmethoden
}
```

Der Zugriff auf die Felder muss nicht zwangsläufig über die Getter und Setter erfolgen. Groovy unterstützt den direkten Zugriff auf die Attribute. Dabei wird aber nicht das Attribut selbst abgefragt (das ist ja private); der Zugriff erfolgt vielmehr automatisch über getEigenschaft() bzw. setEigenschaft(). Das ermöglicht eine sehr kurze und intuitive Schreibweise. Es geht sogar soweit, dass man eigene Setter oder Getter schreiben kann, ohne dass ein entsprechendes Attribut angelegt sein muss.

Wie im nachfolgenden Beispiel an der E-Mail zu sehen ist, kann dann ebenfalls mit p.email darauf zugegriffen werden.

```
def p = new Person()
p.setFirstName('Otto')
p.lastName = 'Mustermann'
println p.getFirstName()
println "$p.firstName $p.lastName"
println p.email
```

[2] Java unterstützt bislang Closures nicht direkt, man kann es dort aber mit anonymen inneren Klassen nachbauen. Im Moment ist außerdem eine große Diskussion im Gange, wie man Closures direkt in die Sprache Java integrieren kann (vermutlich ab Java 7).

## XML in Groovy

Neben dem Groovy Development Kit (GDK), welches die Java Standard-Bibliothek erweitert, liefert Groovy noch viele eigene Bibliotheken mit.

Ohne ins Detail zu gehen, sei an dieser Stelle der hervorragende Support für den Zugriff auf Datenbanken, die einfache Verarbeitung von XML und die Erstellung grafischer Benutzeroberflächen (Swing, SWT) genannt. Mit den Groovlets (Groovy Servlet) und dem einfachen Template-Mechanismus (GString, Groovy Server Pages (GSP)) kann man sehr leicht kleine Web-Anwendungen erstellen. Es gibt außerdem eine Handvoll externe Module, wobei hier stellvertretend die gute und vor allem einfache Unterstützung für Web-Services erwähnt werden soll.

Ein Teil des XML-Supports und die Erzeugung grafischer Benutzeroberflächen geht auf das Builder-Konzept zurück, welches in Groovy an vielen Stellen zur Anwendung kommt. Builder ermöglichen es auf einfache Art und Weise hierarchische Baumstrukturen (wie XML oder die Anordnung von Komponenten in einem GUI) in einer deklarativen Weise zu erzeugen.

```
def builder = new groovy.xml.MarkupBuilder()
builder.html {
  head {
    title 'My Homepage'
  }
  body {
    h1 'Welcome'
    p 'Welcome to my Homepage'
  }
}
```

In diesem Beispiel werden einige Zeilen HTML erzeugt.

```
<html>
 <head>
  <title>My Homepage</title>
 </head>
 <body>
  <h1>Welcome</h1>
  <p>Welcome to my Homepage</p>
 </body>
</html>
```

Wie übersichtlich die Entwicklung von grafischen Benutzeroberflächen aussehen kann, zeigt das folgende Beispiel.

```
def swing = new groovy.swing.SwingBuilder()
def frame = swing.frame(title:'Groovy Swing Demo',
size:[300,300]) {
  borderLayout()
  panel(constraints:java.awt.BorderLayout.CENTER) {
    label(text:"Hello World")
    button(text:'Click Me', actionPerformed: {
      println "clicked"
    })
  }
  panel(constraints:java.awt.BorderLayout.SOUTH) {
    label(text:"How are you?")
  }
}
frame.pack()
frame.show()
```




## Pfadfinder

Um mit Baumstrukturen arbeiten zu können, muss man über die Knoten navigieren. Auch hier bietet Groovy ein „Goody“ an, dass in Anlehnung an XPath GPath heißt. Als Beispiel soll folgender Objektgraph dienen: eine Bank, die mehrere Kunden hat, die wiederum einen Namen und eine E-Mail-Adresse haben. Mit Hilfe von GPath kann man dann sehr einfach alle Kundennamen abfragen:

```
println bank.kunden.name
// -> [Otto, Fritz, Heinrich]
println bank.kunden.email?.toLowerCase()
// -> ['otto@mail.de', null, 'h@abc.de']
```

Wenn man alle E-Mail-Adressen der Kunden umgewandelt in Kleinbuchstaben abfragen möchte, würde man eine NullpointerException erhalten, wenn ein Kunde keine E-Mail-Adresse hat. Aber auch hier bietet Groovy ein nettes Gimmick, nämlich das sichere Dereferenzieren mit dem ?-Operator. Wenn der Teil des Ausdrucks bereits Null ist, wertet ihn Groovy nicht mehr weiter aus.

## Was Groovy dynamisch macht

Wenn man in Java eine Methode an einem bestimmten Objekt eines gewissen Typs aufruft, dann wird dieser Aufruf bei der Kompilierung fest eingespeichert. In Groovy ist dies ganz anders. Erst zur Laufzeit wird herausgefunden, welche Methode aufgerufen werden soll.

Dies gelingt, weil ein Methodenaufruf durch eine Kette von Framework-Klassen geleitet wird. Dadurch kann jederzeit Einfluss darauf genommen werden, ob der Aufruf überhaupt erfolgen soll oder ggf. an eine ganz andere Stelle umgeleitet wird. Methoden können so auch noch zur Laufzeit in den Aufrufmechanismus eingeklinkt werden, obwohl sie zur Compile-Zeit noch gar nicht existiert haben.

Eine Möglichkeit, mit der man dynamisch programmieren kann, stellen die Expandos dar. Es handelt sich dabei um einen Container für Daten (Zustand) und Methoden (Verhalten), den man quasi zur Laufzeit „befüllen" kann.

```
def duck = new Expando()
duck.name = 'Duffy'
duck.fly = {length -> println "$name flies about $length km"}
duck.fly(5)
```

In dem Beispiel sieht man, wie das Objekt *duck* nach der Instanziierung ein Attribut *name* mit dem Wert 'Duffy' zugewiesen bekommt. Methoden ergänzt man durch das Setzen einer Closure.

Durch seine Eigenschaften eignet sich Expandos sehr gut als Dummy-Objekte für Testzwecke (Stubs oder Mocks). Sie haben aber auch einige Einschränkungen. So können sie keine bereits existierenden Klassen erweitern, keine Interfaces implementieren und auch nicht von anderen Klassen erben.

## Kategorien

Wie bereits angedeutet erweitert Groovy mit dem GDK bereits bestehende Java-Klassen um neue Funktionen. Diesen Mechanismus kann man in Groovy in ähnlicher Weise mit Hilfe des Schlüsselwort **use** selbst implementieren.

Um einer bestehenden Klasse eine Methode hinzuzufügen, muss diese Methode zuerst in einer Kategorieklasse als statische Methode definiert werden. Dabei muss der erste Parameter immer vom Typ des Objekt sein, an welchem später der Aufruf erfolgen soll.

```
class StringCategory {
  static String swapCase(String self) {
    def sb = new StringBuffer()
    self.each {
      sb << (Character.isUpperCase(it as char) ?
        Character.toLowerCase(it as char) :
        Character.toUpperCase(it as char))
    }
    sb.toString()
  }
}
use (StringCategory) {
  assert 'AlITTLEtEST' == 'aLittleTest'.swapCase()
}
```

Durch **use** kann man für einen Code-Block das zusätzliche Verhalten an bestimmte Klassen anhängen. Allerdings macht diese Syntax die Verwendung der Kategorien sehr umständlich.

## Meta-Klassen

Seit Groovy 1.1 gibt es mit der ExpandoMetaClass eine einfachere Variante. Der Unterschied zwischen den beiden Varianten ist, dass Kategorien immer nur in einem bestimmten Thread zusätzliche Funktionalität an Objekte anhängen, während dies die ExpandoMetaClass global für alle Objekte einer bestimmten Klasse tut.
Im nachfolgenden Beispiel erfolgt die Implementierung der swapCase-Methode ein zweites Mal, diesmal für alle Objekte der String-Klasse.

```
String.metaClass.swapCase = {->
    def sb = new StringBuffer()
    delegate.each {
        sb << (Character.isUpperCase(it as char) ?
            Character.toLowerCase(it as char) :
            Character.toUpperCase(it as char))
    }
    sb.toString()
}

assert 'tEsT' == 'TeSt'.swapCase()
```

Die Magie der ExpandoMetaClass endet noch nicht an dieser Stelle. Man kann zum Beispiel auch Konstruktoren, statische Methoden usw. definieren. Durch Groovys Methoden-Pointer-Syntax kann eine Klasse sogar Methoden einer anderen Klasse borgen, wie das folgende Beispiel zeigt.

```
class Person {
  String name
}
class Dog {
  def bark() {
    'Bark'
  }
}
def lender = new Dog()
Person.metaClass.imitateDog = lender.&bark
def p = new Person()
assert 'Bark' == p.imitateDog()
```

## Meta-Objekt-Protokoll

Das Geheimnis hinter all diesen netten Mechanismen nennt sich Meta Object Protocol (MOP). Dieses Protokoll stellt Möglichkeiten zur externen Modifikation des Verhaltens von Objekten zur Verfügung.

In Groovy hat jede Klasse eine Metaklasse, durch die man Informationen zu der Klasse erfragen kann. So kann man zum Beispiel herausfinden, ob die Klasse eine bestimmte Methode implementiert bzw. eine Property anbietet.

Weiterhin gibt es sogenannte Hook-Methoden:

- invokeMethod – Abfangen aller Methoden, die an der Klasse aufgerufen werden
- methodMissing – Abfangen aller fehlgeschlagenen Methodenaufrufe (wenn Methode nicht existiert)
- get/setProperty – Abfangen von Zugriff auf Properties
- propertyMissing – Abfangen von fehlgeschlagenen Property-Zugriffen

Mit Hilfe dieser Hook-Methoden kann man in den Aufrufzyklus eingreifen, Interceptoren schreiben (wie in AOP), oder Aufrufe komplett auf dynamische Implementierungen umleiten.

```
def someObject = 'Test'
// print all the methods
someObject.metaClass.methods.each { println it.name }
// print all the properties
someObject.metaClass.properties.each { println it.name }
```

Es geht sogar soweit, dass man aus Java heraus dynamisch Attribute oder Methoden zu Groovy-Objekten (abgeleitet von GroovyObject) hinzufügen und aufrufen kann. Allerdings ist diese Arbeitsweise im Gegensatz zum internen Einsatz in Groovy recht umständlich.

## Domänenspezifische Erweiterungen

Das MOP stellt auch die Grundlage für das Erstellen von Domain Specific Languages (DSL) dar. Eine DSL ist eine Sprache, die eine Syntax zu einer speziellen Thematik (Domäne) anbietet und damit eine für das Domänenproblem höhere Ausdruckskraft erreicht. Sie ist außerdem für die fachlichen Experten leichter erlernbar, als zum Beispiel eine komplexe Programmiersprache.

In Groovy kann man sogenannte interne DSLs erzeugen. Dabei handelt es sich um eingebettete Sprachen, welche Sprachelemente der Muttersprache übernehmen.

Eine in Groovy implementierte DSL profitiert von nachfolgenden Punkten:

- ausdrucksstarke und prägnante Syntax
- benannte Parameter bei Methodenaufrufen
- Meta Object Protocol (dynamisch Methoden hinzufügen)
- Überladen von Operatoren
- Closures
- Builder

Ein Beispiel für eine einfache DSL ist im nachfolgenden Beispiel zu sehen. Das Addieren physikalischer Größen mit unterschiedlichen Einheiten basiert hauptsächlich auf dem dynamischen Hinzufügen von Methoden zu der Klasse Number (getKm() und getM()), die das Umrechnen der Einheiten übernehmen, und der Überladung von Operatoren. Zur Erinnerung, beim Zugriff auf Attribute kann man statt .getKm() auch direkt .km schreiben.

```
println 4.km + 3 * 1000.m + 5.mile // -> 15.05 km
```

## Wozu Groovy gut ist

Grundsätzlich lässt sich Groovy überall dort einsetzen, wo im Moment mit Java programmiert wird. Als großen Vorteil kann Groovy die ausdrucksstarke Syntax und seine dynamischen Fähigkeiten in die Waagschale werfen. Dadurch muss nicht nur weniger Quellcode geschrieben werden, der Code wird auch lesbarer und lässt sich einfacher warten.

Natürlich gehen insbesondere die dynamischen Fähigkeiten zu Lasten der Performanz. Für zeitkritische Anwendungen ist Groovy dementsprechend weniger zu empfehlen.

Java galt allerdings in seinen Anfangszeiten im Vergleich zu C/C++ als sehr inperformant. Mittlerweile ist der Unterschied nur noch geringfügig und auch bei Groovy wird von Release zu Release an der Performanz gearbeitet. Und auch wenn Groovy sicher nie so performant wie eine statische Programmiersprache sein wird, gibt es durch die enge Integration mit Java (gleicher Bytecode) immer die Möglichkeit, performanzkritische Teile der Anwendung in Java oder sogar nativ (via Java Native Interface) in C/C++ zu implementieren.

## Groovy in der Praxis

In Vorträgen der Groovy-Macher wird immer wieder von erfolgreichen Einsätzen von Groovy berichtet. So hat eine große amerikanische Versicherungsgesellschaft mittlerweile etwa 50.000 Codezeilen in ein Modul zur Risikoberechnung investiert. Andere große Anwender von Groovy sind das französische Justizministerium und das Europäische Raumfahrt Konsortium (EADS).

Groovys Anwendungsmöglichkeiten reichen von punktuellen Einsätzen in Java-Programmen, über Scripting zur Laufzeit, Testunterstützung, prototypischen Umsetzungen bis hin zu kompletten Anwendungen und der Integration in Java EE Projekten. Durch die gute Integration zwischen Groovy und Java bietet sich der Einsatz von Groovy speziell für kleine, spezifische Pro-

blemstellungen in Java-Anwendungen an. Seine Stärken spielt Groovy zum Beispiel bei der Verarbeitung von Daten aus.

Das Parsen von XML mit anschließender GPath-Navigation lässt sich viel kürzer, schneller und intuitiver lösen als mit jeder Java-API. Bei Datei-Zugriffen nimmt Groovy dem Entwickler das Schreiben von Boiler Plate Code ab (z. B. um Dateien zu schließen) und unterstützt ihn außerdem durch viele nützliche GDK-Methoden.

Weitere Anwendungsmöglichkeiten sind der Zugriff auf Datenbanken, die Entwicklung von Benutzeroberflächen oder der Einsatz von Web-Services.

## Scripting

Ein weiteres Anwendungsgebiet ist das Scripting. Unterpunkte davon sind das Abarbeiten von wiederkehrenden administrativen Aufgaben, die Terminierung von Tasks oder die Verwaltung von Projekt-Infrastrukturen mittels Build-Skripten (Ant). Für Windows Benutzer gibt es eine Bibliothek names Scriptom [4], die aufbauend auf der Java COM Bridge (Jacob) [5] zum Beispiel das Fernsteuern von Microsoft Office erlaubt.

An dieser Stelle nicht zu vergessen sind die sogenannten Laufzeit-Makros. In OpenOffice kann Groovy als Makrosprache eingesetzt werden. Denkbar wäre auch, dem Benutzer in einer laufenden Java-Anwendung die Möglichkeit für das Ausführen von Skripten zu geben, z. B. die Eingabe von Funktionen in einer Mathematik-Software (f(x) = sin(x)).

Der Groovy-Code könnte aber auch genauso gut aus Datenbanken kommen oder über das Internet nachgeladen werden. So wäre die Codebasis zentralisiert und der Anwender würde immer den aktuellen Stand ausführen, ohne explizit lokale Updates durchführen zu müssen.

Um Groovy Skripte zur Laufzeit auszuführen, gibt es übrigens mehrere Möglichkeiten. Seit Java 6 wird zum einen eine standardisierte Schnittstelle für die Einbettung von Skriptsprachen mitgeliefert (JSR 223 [6]). Groovy selbst bietet mit der GroovyShell, der GroovyScriptingEngine und dem GroovyClassLoader drei weitere Möglichkeiten an.

## Testen mit Groovy

Testen wird in Groovy ebenfalls zum Kinderspiel und qualifiziert Groovy damit auch für die testgetriebene Entwicklung (TDD). Neben den in den Sprachumfang eingebauten **assert**-Anweisungen (im Gegensatz zu Java sind sie nicht abschaltbar), liefert Groovy einen speziellen JUnit-TestCase mit erweiterten **assert**- und **fail**-Methoden.

Und allein hierdurch wird der Testcode verkürzt und im Hinblick auf Testdokumentation im wahrsten Sinne des Worte lesbarer. Durch seine dynamischen Fähigkeiten kann es zusätzlich bei der Erzeugung von Stub- und Mock-Objekten glänzen. Ohne weitere Bibliotheken kann man in Groovy dank des MOP Stellvertreter erzeugen und damit kleine Einheiten entkoppelt von ihren Abhängigkeiten in echter Unit-Manier testen.

## Ausblick

Ein weiteres spannendes Thema ist das Webframework Grails [7], das an das Prinzip von Ruby on Rails [8] anlehnt, welches technisch auf Groovy und vielen bekannten Java APIs (Hibernate, Spring, ...) aufbaut.

Die Mächtigkeit dieser noch relativ jungen Sprache lässt sich, da hier in diesem Rahmen bestenfalls an der Oberfläche der vielen nützlichen Features gekratzt werden konnte, vermutlich nur erahnen. Die Einsatzmöglichkeiten sind vielfältig und die Einstiegshürde liegt insbesondere für Java-Entwickler sehr niedrig.

Aber wer Groovy ausprobiert, wird sich vermutlich schnell verlieben.

### Referenzen


[1] Java Community Process *JSR 241: The Groovy Programming Language*, http://jcp.org/en/jsr/detail?id=241

[2] Codehaus *Groovy – An agile daynamic language for the Java Platform*, http://groovy.codehaus.org

[3] König, D. *Groovy in Action*, Manning, 2007

[4] Codehaus *COM Scripting*, http://groovy.codehaus.org/COM+Scripting

[5] SourceForge *JACOB - Java COM Bridge*, http://sourceforge.net/projects/jacob-project

[6] Java Community Process *JSR 223: Scripting for the Java Platform*, http://jcp.org/en/jsr/detail?id=223

[7] Grails *See the Light – Agile, Industrial Strength, Rapid Web Application Development Made Easy*, http://grails.org

[8] Rails *Ruby On Rails*, http://www.rubyonrails.org


### Weiterführende Literatur

- groovy.blogs (http://groovyblogs.org)
- AboutGroovy (http://aboutgroovy.com)

### Kurzbiographie



Falk Sippach (sippach@oio.de) ist bei der Mannheimer Firma Orientation in Objects als Trainer, Berater und Entwickler tätig. Schwerpunktmäßig beschäftigt er sich mit Backend-Lösungen (Spring, EJB) und Persistenzmechanismen (JPA, Hibernate). Seit neuestem ist er außerdem dem Charm von Groovy und Grails verfallen.

Copyright © 2008 Bookware 1865-682x/08/01/002 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

# Das hängt davon ab...

## Eine Einführung in Dependency Injection mit Guice

von Christoph Dietze

Guice ist ein Dependency Injection Framework in Java und entstammt dem Hause Google. Zu den Hauptmerkmalen von Guice gehören seine Leichtgewichtigkeit, der Verzicht auf Fremdsprachen wie XML und die starke Verwendung von Java-5-Features wie Annotations und Generics. Dieser Artikel gibt eine Einführung, wie Dependency Injection mit Guice realisiert werden kann und welche weitergehenden Features Guice unterstützt.

### Was ist Dependency Injection?

Die meisten Objekte können ihre Aufgaben nicht alleine erfüllen, sondern benötigen dazu andere Objekte. Oft besteht ein merklicher Teil der Initialisierung eines Objekts darin, sich Referenzen auf solche Abhängigkeiten zu beschaffen und diese zu initialisieren. Dependency Injection (DI) ist ein Ansatz, der Objekte von der Aufgabe befreit, sich ihre Abhängigkeiten selbst zu beschaffen. Stattdessen geben Objekte lediglich bekannt, welche Abhängigkeiten sie haben und erwarten, dass sie diese bei der Initialisierung übergeben (injiziert) bekommen. Es ist also eine Anwendung des „Hollywood-Prinzips“: „don‘t call us, we‘ll call you“.

### Dependency Injection mit Google-Guice

Dependency Injection kann ohne zusätzliche Hilfsmittel realisiert werden. Das Fabrik-Entwurfsmuster wird in diesem Zusammenhang häufig verwendet. Es gibt jedoch Frameworks, die den Programmierer unterstützen und die Anwendung von DI vereinfachen soll.

Einen jungen Vertreter der Gattung der DI-Frameworks stellt Google-Guice (gesprochen: juice) [1] dar. Guice ist als unterstützende Bibliothek für Googles Applikation „AdWords“ entstanden. Google hat Guice unter der liberalen Apache 2.0 Lizenz aktuell in der Version 1.0 der Öffentlichkeit zur Verfügung gestellt. Guice ist eine leichtgewichtige Java-Bibliothek, welche speziell die Aufgaben für DI erfüllt.

Guice setzt voll auf die seit JDK 5 verfügbaren Annotations und Generics. Weder zur Konfiguration, noch bei der Benutzung muss man bei Guice auf andere Sprachen wie XML zurückgreifen – der Entwickler braucht seine Java-Insel also nicht zu verlassen.

Ein minimales Beispiel zum Einsatz von Guice:

```
class MyClient {
  @Inject
  private ServiceX serviceX;
  public void work() {
    serviceX.work();
  }
}
```

Die Klasse MyClient benötigt um ihre Aufgaben zu erledigen eine Referenz auf ein ServiceX-Objekt. Dem Prinzip von DI entsprechend besorgt sie sich diese aber nicht selbst, sondern gibt lediglich bekannt, dass sie eine ServiceX-Referenz benötigt (durch die Annotation @Inject). Es liegt nun in der Verantwortlichkeit von Guice, diese Abhängigkeit zu erfüllen und bei der Initialisierung einer Instanz von MyClient eine Referenz auf eine ServiceX-Objekt bereitzustellen. Eine Applikation, die dieses Objekt verwendet, könnte so aussehen:

```
class MyProgram {
  public static void main(...) {
    Injector injector = Guice.createInjector();
    MyClass mc = injector.getInstance(MyClass.class) ;
    mc.work();
  }
```

Das zentrale Objekt, welches bei Guice die tatsächlichen Instanzen der gewünschten Klassen erzeugt und deren Abhängigkeiten auflöst, ist der Injector. In diesem Programm ist keine Konfiguration des Injectors notwendig und wir können uns eine Referenz einfach mit dem Aufruf der statischen Methode Guice.createInjector() beschaffen. Mit der Methode getInstance(Class *clazz*) fordert man den Injector auf, eine Instanz vom Typ *clazz* zu beschaffen. Unter der Haube hat Guice bei diesem Aufruf einiges zu tun: Es muss einen Graph der Objekte und ihrer Abhängigkeiten erzeugen, entscheiden, welche

konkreten Instanzen erzeugt werden müssen und die jeweils erforderlichen Abhängigkeiten injizieren.

Im obigen Beispiel wurde eine Abhängigkeit in eine Member-Variable injiziert. Es gibt jedoch auch die Möglichkeiten in Methoden und in den Konstruktor injizieren zu lassen, z. B.:

```
class SomeService {
  @Inject
  ServiceA fieldInjectedService;
  @Inject
  void setServiceB(ServiceB methodInjectedService) {
    ...
  }
  @Inject
  SomeService(ServiceC constructorInjectedService) {
    ...
  }
}
```

Zu beachten ist jedoch, dass Feld- und Methoden-Injektionen nach dem Konstruktor-Aufruf ausgeführt werden. Das heißt man hat das Problem, dass im Konstruktor diese Abhängigkeiten noch nicht zur Verfügung stehen und eine eventuell notwendige Initialisierung noch nicht gemacht werden kann. Sind solche Initialisierungen notwendig, oder möchte man diesem Problem generell aus dem Wege gehen, so ist es empfehlenswert auf Feld- und Methoden-Injektion zu verzichten und ausschließlich Konstruktor-Injektion zu verwenden [2].

## Konfiguration des Injectors

Nehmen wir an, im obigen Beispiel ist ServiceX ein Interface von dem die konkrete Klasse ServiceX1 abgeleitet ist, dann kann Guice in MyProgram nicht entscheiden, was es für MyClients Abhängigkeit *serviceX* injizieren soll und wirft folgende Exception:

```
com.google.inject.ConfigurationException:
Error at MyClient.serviceX(MyClient.java:4)
Binding to ServiceX not found.
No bindings to that type were found.
```

Um solche Bindings (wie hier das fehlende Binding von ServiceX zu ServiceX1) zu definieren, verwendet man ein sog. Module, das dann zur Konfiguration des Injectors verwendet wird. Module ist ein Interface, mithilfe dessen flexibel und zur Laufzeit austauschbar die Konfiguration eines Injectors definiert werden kann. Beim Erstellen eines Injectors können eine oder mehrere Module-Instanzen angegeben werden, also z. B.:

```
Injector injector = Guice.createInjector(module1, module2);
```

Um beispielsweise stets ServiceX1-Instanzen für ServiceX-Abhängigkeiten zu verwenden, würde folgendes Module dienen:

```
public class MyModule implements Module {
  public void configure(Binder binder) {
    binder.bind(ServiceX.class).to(ServiceX1.class);
  }
}
```

Praktikabler, als von Module direkt abzuleiten, ist es, die Hilfsklasse AbstractModule zu verwenden. Diese stellt eine Reihe nützlicher **protected** Methoden bereit um die Konfiguration zu erleichtern. Oben genannter Code würde dann wie folgt aussehen:

```
public class MyModule extends AbstractModule {
  public void configure() {
    bind(ServiceX.class).to(ServiceX1.class);
  }
}
```

Eine Alternative, welche die Verbindung bereits zur Compile-Zeit festlegt, besteht darin das Interface ServiceX mit der Annotation @ImplementedBy(ServiceX1.class) zu versehen.

## Definieren von Bindings

Zentrales Interface, um zu definieren, welche konkrete Klasse wo injiziert wird, ist der Binder. Jedes Module hat Zugriff auf einen Binder, welcher in der Methode configure übergeben wird, siehe obiges Beispiel. Der Binder verwendet eine „embedded domain-specific language“ (EDSL) um sog. Bindings zu definieren. Besser als die einzelnen Methoden der relevanten Klassen zu betrachten, ist es, die Funktionalität anhand von Beispielen zu zeigen.

Wie im letzten Abschnitt schon verwendet, kann mit dem Befehl

```
bind(ServiceX.class).to(ServiceX1.class);
```

mitgeteilt werden, dass überall wo ServiceX-Abhängigkeiten bestehen, Objekte vom Typ ServiceX1 injeziert werden.

Noch feiner kann man mit Hilfe von Annotationen differenzieren: Wir hätten z.B. der Klasse MyClient2 auch eine eigene Annotation @MyAnnotation zuweisen können und das Binding dann wie folgt definieren können:

```
bind(ServiceX.class)
  .annotatedWith(MyAnnotation.class)
  .to(ServiceX2.class);
```

Eine weitere Funktionalität des Binders ist, an bestimmte Instanzen zu binden. Möchten wir beispielswei-

se, dass immer dieselbe Instanz von ServiceX injiziert wird, so können wir dies wie folgt erreichen:

```
bind(ServiceX.class).toInstance(new ServiceX2());
```

Dieser Mechanismus lässt sich auch dazu verwenden, um Konstanten zu injizieren. Hätten wir beispielsweise einen String mit einer eigenen Annotation @ConfigPath versehen, so könnten wir mit folgendem Befehl angeben, was in diesen injiziert werden soll:

```
bind(String.class)
   .annotatedWith(ConfigPath.class)
   .toInstance("test/config/");
```

Für das Binden an Konstanten gibt es auch eine kürzere Schreibweise:

```
bindConstant()
   .annotatedWith(ConfigPath.class)
   .to("test/config/");
```

Um keine eigene Annotation definieren zu müssen, kann man mithilfe der @Named Annotation eine bestimmte Abhängigkeit mit einem String „taggen“:

```
@Inject @Named("ListeningPort")
int port;
```

Im Binder kann man dann wie folgt Bezug nehmen:

```
bindConstant()
   .annotatedWith(Names.named("ListeningPort"))
   .to(1234);
```

## Provider

Oft ist es nicht möglich oder erwünscht, Referenzen auf bestimmte Instanzen direkt zu injizieren. In solchen Fällen kann statt des Objekts selbst ein Provider für dieses Objekt übergeben werden. Provider ist ein generisches Interface für einen Typ T, mit der einzigen Methode get(), welche eine Referenz auf ein T liefert. In folgendem Beispiel wurde das einführende Beispiel so geändert, dass die ServiceX-Referenz nicht mehr direkt injiziert wird, sondern ein Provider:

```
class MyClientUsingProvider {
   @Inject
   private Provider<ServiceX> serviceXProvider;
   public void work() {
      ServiceX serviceX = serviceXProvider.get();
      serviceX.work();
   }
}
```

Durch die Verwendung von Providern lassen sich beispielsweise mehrere Instanzen zur Laufzeit erzeugen. Im obigen Beispiel wird bei jedem Aufruf von work() eine neue ServiceX-Instanz erzeugt. Ein weiterer Vorteil von Providern ist, dass die benötigten Instanzen erst zu einem späteren Zeitpunkt erzeugt werden können – oder auch gar nicht.
Ein wichtiger Verwendungszweck für selbstdefinierte Provider ist, Objekte, in denen man keine Guice-Annotations wie @Inject verwenden kann oder möchte (z. B. fremde Bibliotheken), trotzdem mit Guice erzeugen zu lassen. Als Beispiel wollen wir in MyClient nun nicht mehr ServiceX1 verwenden, sondern ServiceX2:

```
class ServiceX2 implements ServiceX {
   private int port;
   public ServiceX2(int port) {
      this.port = port;
   }
   public void work() { /* ... */ }
}
```

Definiert man in Module nun, dass ServiceX2 statt ServiceX1 injiziert werden soll und startet MyProgram, so wird folgende Exception geworfen:

```
com.google.inject.CreationException:
Guice configuration errors:
Error at ServiceX2.<init>(ServiceX2.java:4):
Could not find a suitable constructor in ServiceX2.
Classes must have either one (and only one) constructor
annotated with @Inject or a zero-argument constructor.
```

Direkt kann Guice ein ServiceX2-Objekt also nicht instanziieren, da dazu entweder ein Default-Konstruktor oder ein nicht-trivialer Konstruktor mit Annotation @Inject vorhanden sein muss. Ohne die Definition von ServiceX2 zu verändern, kann ein eigener Provider Abhilfe schaffen:

```
class ServiceX2Provider implements Provider<ServiceX>
{
   public ServiceX get() {
      return new ServiceX2(1234);
   }
}
```

Nun muss man den Injector konfigurieren, dass er ServiceX-Abhängigkeiten mittels des ServiceX2-Providers auflösen soll:

```
bind(ServiceX.class)
   .toProvider(ServiceX2Provider.class);
```

So hat man das Gewollte erreicht: Guice kann nun ServiceX2-Instanzen erzeugen, obwohl die Klasse selbst nicht mit Guices Inject-Mechanismus kompatibel ist.

## Scopes

Standardmäßig erzeugt Guice für jede benötigte Abhängigkeit eine neue Instanz des jeweiligen Objekts. Mithilfe von sog. Scopes kann man definieren, in welchen Bereichen bereits erzeugte Instanzen wiederverwendet werden sollen. Prominenteste Anwendung ist es, ein Singleton-Objekt zu definieren. Hierbei wird ein Objekt an den Scope Singleton gebunden. Dies kann entweder an dem Objekt selbst geschehen:

```
@Singleton
class MySingleton { /* ... */ }
```

Oder man spezifiert es in der Definition des Modules:

```
bind(MySingleton.class).in(Scopes.SINGLETON);
```

Die Guice-API bietet dem Entwickler eine einfache und flexible Schnittstelle, um eigene Scopes zu definieren. Welche Scopes sinnvoll sind, hängt von der jeweiligen Anwendung ab, weshalb Singleton der einzige vordefinierte Scope ist. Bei Servlets bieten sich Scopes stark an, um mit ihnen die Gültigkeitsbereiche von Instanzen auf Sessions und Requests zu beschränken, so dass diese Scopes bereits in einem optionalen JAR-Archiv definiert sind, welches Teil des Guice-Projekts ist.

## Bindings von generischen Typen

Möchte man ein Binding für einen generischen Typ definieren, so stösst man auf eine Besonderheit, da Java keine Klassenobjekte von generischen Typen erzeugt – Ausdrücke wie List<String>.class führen daher zu Fehlermeldungen des Compilers. Um Bindings für generische Typen dennoch zu ermöglichen, stellt Guice die Hilfsklasse TypeLiteral<T> bereit. Meistens wird man hiervon anonyme Klassen ableiten und instantiieren, die mit dem gewünschten Typ parametrisiert sind, also z. B.

```
TypeLiteral<List<String>> listOfStringsType
  = new TypeLiteral<List<String>>() {};
```

Durch das parametrisierte Ableiten hat Java auch zur Laufzeit die Typinformationen des gewünschten Typs zur Verfügung. Beim Definieren von Bindings lässt sich ein solches TypeLiteral ähnlich verwenden, wie übliche Klassenobjekte:

```
bind(listOfStringsType)
  .toInstance(new ArrayList<String>());
```

oder

```
bind(listOfStringsType)
  .to(new TypeLiteral<ArrayList<String>>() {});
```

Ein TypeLiteral stellt somit einen integrierten und recht eleganten Workaround dar, um Generics mit Guice injizieren zu lassen.

## Unit-Testing

Um Unit-Tests für eine Applikation zu schreiben, die Guice verwendet, gibt es zwei Ansätze:

1) Auf Guice und den Injector komplett zu verzichten und die Konstruktoren der benötigten Klassen selbst aufrufen.

2) Guice nach wie vor verwenden, den Injector vorher nur entsprechend zu konfigurieren, so dass dieser die entsprechenden Mock-Up Klassen verwendet.

Abhängig vom jeweiligen Anwendungsfall kann eine Variante angebrachter als die andere sein oder aber auch beide verwendet werden.

## Fazit

Guice ist ein interessantes Framework um Dependency Injection in einer Applikation zu verwenden. Herausragende Eigenschaften sind insbesondere, dass es sich um eine leichtgewichtige Bibliothek handelt, welche sowohl für den Einsatz in Server-Software, als auch in Client-Applikationen einsetzbar ist. Google hat mit Guice ein Projekt ins Leben gerufen, das man im Auge oder zumindest im Hinterkopf behalten sollte.

### Referenzen


[1] Google *Guice 1.0 User's Guide,* http://google-guice.googlecode.com/files/Guice%201.0%20User%27s%20Guide.pdf

[2] Fowler, Martin *Inversion of Control Containers and the Dependency Injection pattern,* http://martinfowler.com/articles/injection.html


### Weiterführende Literatur

- Google *Slides of the „Java on Guice" presentation,* http://google-guice.googlecode.com/files/JavaOnGuice20070426.pdf
- Google *Spring Comparison – How does Guice compare to Spring et al?* http://code.google.com/p/google-guice/wiki/SpringComparison

### Kurzbiographie



Christoph Dietze ist als Software-Entwickler und Trainer für die MATHEMA Software GmbH tätig. Zu seinen Themenschwerpunkten gehört die Software-Entwicklung unter Java, speziell komponentenbasierte Java-Anwendungen.

Copyright © 2008 Bookware 1865-682x/01/08/003 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

## Beispielanwendung für Guice

Als einfaches, aber vollständiges Beispiel für eine Anwendung, die mit Guice arbeitet, soll ein Programm dienen, welches bestimmte Bezeichner auf Netzwerkadressen abbildet. Zentrale Klasse der Anwendung ist der AddressMapper, der sich seine Abhängigkeiten injizieren lässt. Eine Anfrage, einen Namen in eine Adresse umzusetzen, soll drei Stufen durchlaufen:

1. Der *userMapper* mappt – abhängig vom aktiven Benutzer – den angefragten Bezeichner auf einen logischen Namen;
2. Der *nameMapper* mappt den logischen Namen auf den Namen im Netzwerk;
3. Der *serverMapper* mappt den Netzwerknamen auf die zughörige IP-Adresse.

Diese drei Abhängigkeiten sind allesamt vom Typ Map<String, String> und mit @Named annotiert, um die Konfiguration der Bindings zu ermöglichen. Der Mapper wird von der Klasse MyProgram aus verwendet, in der auch Guice's Konfiguration vorgenommen wird. Hier werden unterschiedliche Implementierungen für die drei zu injizierenden Mapper definiert: Der *userMapper* soll sich einer XML-Datei bedienen, der *nameMapper* verwendet JNDI und der *serverMapper* bezieht seine Informationen aus einer Datenbank.

MyProgram lässt in seiner main-Methode von Guice eine Instanz von AddressMapper erzeugen und stellt anschließend die Anfrage, den Namen „printer“ in eine Netzwerkadresse umzusetzen.

**AddressMapper.java**

```
import java.util.Map;
import com.google.inject.Inject;
import com.google.inject.name.Named;
public class AddressMapper {
   @Inject
   @Named("userMapper")
   private Map<String, String> userMapper;
   @Inject
   @Named("nameMapper")
   private Map<String, String> nameMapper;
   @Inject
   @Named("serverMapper")
   private Map<String, String> serverMapper;
   public String getAddress(String resourceName) {
      String logicalName = userMapper.get(resourceName);
      System.out.println("userMapper: " + resourceName + " -> " + logicalName);
      String networkName = nameMapper.get(logicalName);
      System.out.println("nameMapper: " + logicalName + " -> " + networkName);
      String networkAddress = serverMapper.get(networkName);
      System.out.println("serverMapper: " + networkName + " -> " + networkAddress);
      return networkAddress;
   }
}
```

**MyProgram.java**

```
import java.util.Map;
import com.google.inject.AbstractModule;
import com.google.inject.Guice;
import com.google.inject.Injector;
import com.google.inject.TypeLiteral;
import com.google.inject.name.Names;

public class MyProgram {
  private static class MyModule extends AbstractModule {
    @Override
    protected void configure() {
      Map<String, String> userMapper = new XmlFileMap("usermap.xml");
      Map<String, String> nameMapper = new JndiMap();
      Map<String, String> serverMapper = new DatabaseMap("serverMapDb");

      // Java kann keine Class-Objekte von parametrisierten Klassen
      // anlegen. Abhilfe beim Binding erreicht man in Guice, indem man
      // entsprechende Ableitungen von TypeLiteral instantiiert. Siehe
      // auch Abschnitt "Bindings von generischen Typen"
      TypeLiteral<Map<String, String>> mapType = new TypeLiteral<Map<String, String>>() {};

      bind(mapType).annotatedWith(Names.named("userMapper")).toInstance(userMapper);
      bind(mapType).annotatedWith(Names.named("nameMapper")).toInstance(nameMapper);
      bind(mapType).annotatedWith(Names.named("serverMapper")).toInstance(serverMapper);
    }
  }
  public static void main(String[] args) {
    Injector injector = Guice.createInjector(new MyModule());
    ServerNameMapper example = injector.getInstance(ServerNameMapper.class);
    String printerServer = example.getAddress("printer");
    System.out.println("My printer's address: " + printerServer);
  }
}
```

```
> java MyProgram
userMapper: printer -> laserPrinter1
nameMapper: laserPrinter1 -> hp496
serverMapper: hp496 -> 192.168.6.28
My printer's address: 192.168.6.28
>
```

# Super, Packages!

## Das Java-Modul-System

von Jörn Hahn

In der siebten Version von Java wird ein Modul-System eingeführt, das mit einem zusätzlichen „Superpackage“-Sprachkonstrukt, sowie mit einem neuen Modul-Dateiformat und verschiedenen Tools zum Teil lange erwartete Neuerungen bringt. Vor allem die Möglichkeit, unterschiedliche Versionen der Module zu definieren und bei Bedarf gleichzeitig zu laden, soll Erleichterung im Alltag von größeren Projekten bringen.

Vielleicht kennt ja der eine oder andere die Probleme mit den Java-Archiven (JARs): Mehrere Bibliotheken werden in verschiedenen Versionen verwendet und am Ende passieren Dinge, die man so nicht will. Ein Beispiel: Man stellt aus Versehen nicht nur eine Version eines Java-Archivs zur Verfügung, sondern mehrere. Tückisch ist dabei, dass man keinen Fehler erhält, sondern weiterarbeiten kann, auch wenn man unwissentlich eine ganz andere Bibliothek verwendet als man eigentlich wollte.

Ein weiteres typisches Problem ergibt sich, wenn eine Anwendung mehrere Versionen einer Bibliothek verwenden soll. Ohne sich gleich tiefer mit dem Problem auseinander zu setzen und z. B. eigene Klassenlader zu schreiben, gibt es heute keine passende Antwort. Es hat sich ein Begriff für diese Art von Problemen eingebürgert: JAR-Hell[1]. Die Fehlersuche in einem manchmal so undurchschaubarem Gebiet wird von vielen Programmieren als „Hölle“ erlebt. Wer sich mit der Thematik schon in der Vergangenheit auseinander gesetzt hat verwendet vielleicht schon heute eines der Tools und Frameworks, welche die entsprechenden Thematiken behandeln. Das Build-Management-Tool Maven [1] und das OSGi-Framework [2] sind hier als Beispiel für die vielen Optionen genannt, die man heute schon hat, um Abhängigkeiten, sowie das Laden und Entladen von Bibliotheken zu regeln.

[1] Ursprünglich sprach man unter Windows von der DLL-Hell, aber auch dem Java-Programmierer sind diese Schwierigkeiten nicht fremd.

### Superpackages als Einheit über den Packages

Mit Java 7 wird es allerdings eine weitere Möglichkeit geben, um Klassen und Packages zu verwalten und zu verteilen. Die Programmiersprache Java wird um ein weiteres sprachliches Konstrukt erweitert: Das Superpackage [3]. Dieses beschreibt eine Einheit, die oberhalb von Packages steht und es erlaubt, Klassen explizit

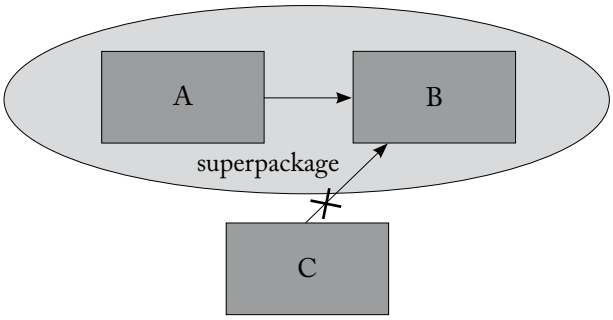



Abb. 1: C kann Methoden von B nicht verwenden, A aber schon, denn A und B sind Members im gleichen Superpackage

als Members zu kennzeichnen oder für den Export und Import zu markieren. Innerhalb eines Superpackages kann sowohl auf alle Members als auch auf alle importierten Klassen zugegriffen werden. Von außen zugänglich ist allerdings nur, was exportiert wird. Auf diese Weise kann beim Information-Hiding auch zwischen verschiedenen Packages unterschieden werden. Schon beim Kompilieren wird daraus eine Metadaten-Datei generiert, die das Konstrukt beschreibt und beim Verpacken weiterverwendet werden kann.

### Das neue JAM-Format als „Super-JAR“

Das JAR-Format, das schon seit Mitte der 90er in Gebrauch ist, wird nun durch die so genannten Java-Module ergänzt, die an der Endung „.jam“ zu erkennen sein werden [4]. In diesen können Klassen, Interfaces, externe Ressourcen, native Bibliotheken und bestehende JAR-Dateien zusammengefasst werden. Durch das hinzufügen der bereits erwähnten Metadaten, wird das ganze selbstbeschreibend.

Wie schon bei JARs gibt es die Möglichkeit, JAM-Dateien zu komprimieren und zu signieren, um die Verteilung über das Internet schneller und sicherer zu machen.

Wird in den Metadaten oder direkt beim Aufruf eine Klasse mit main-Methode angegeben, so kann das Modul auch direkt ausgeführt werden:

```
java -module de.mathama.beispiel:1.3.0.
```

## Ein Modul in verschiedenen Versionen

Eine Neuerung zum JAR-Format ist dabei die optionale Hinzunahme einer Versionsnummer. Diese kann explizit angegeben werden, wie im obigen Beispiel, oder es kann mit einem Ausdruck wie 1.3+ die höchste verfügbare Version ab 1.3 geladen werden.

Angenommen, ein Modul A importiert die Module B und C. Diese importieren wiederum das Modul D in der Version 1+ bzw. 1.4. Das Modul-System lädt nun, sofern vorhanden, zwei Versionen.

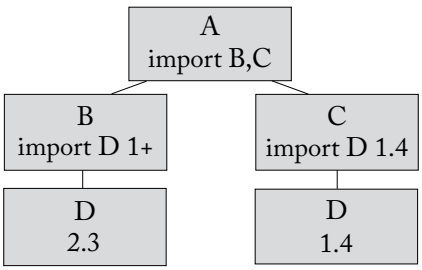



Abb. 2: Mehrere Versionen können gleichzeitig verwendet werden

Wer noch genauer angeben möchte, welches Modul wann importiert werden soll, bekommt dazu die Möglichkeit, indem er eine Import-Policy verwendet. Diese besteht aus einer Klasse, die von ImportPolicy abgeleitet ist und deren Ort in den Metadaten angegeben wird. In dieser Klasse kann nun auf Umgebungsvariablen zugegriffen und nach vorhandenen Modulen gesucht werden. Es stehen alle Möglichkeiten der Programmiersprache Java zur Verfügung, um anzugeben, welches Modul auf welchem System in welcher Version geladen werden soll.

## Ein Repository für Module

Hat man seine Applikation in Superpackages zusammengefasst und in Java-Modulen verpackt, stellt sich die Frage, wie man diese am besten verwaltet. Auch hierfür stellt das Java-Modul-System Möglichkeiten zur Verfügung, die so genannten Repositories. Mit diesen ist es möglich, eine oder mehrere Versionen von Modulen abzulegen, wobei die abgelegten Moduldefinitionen nur bei Bedarf geladen und instantiiert werden. Neben globalen Repositories, in welchen Module systemweit installiert, oder überall für einzelne Benutzer zur Verfügung gestellt werden können, gibt es auch die Möglichkeit für Application-Repositories, die nur zur Laufzeit einzelner Anwendungen verfügbar sind. Darüber hinaus sind URL-Repositories geplant, um Moduldefinitionen bei Bedarf herunterzuladen und lokal zur Verfügung zu stellen.
Ein Punkt, der immer wieder betont wird, ist die Erweiterbarkeit des gesamten Systems. Die Repositories werden zu Beginn das Dateisystem verwenden, um die Module zu speichern, später könnten andere Hersteller allerdings auch weitere Repository-Implementierungen anbieten, die zum Beispiel eine Datenbank zum Ablegen der Module nutzen.

## Noch ein Modulsystem?

Die Frage, ob die zusätzlichen Möglichkeiten unsere Arbeit wirklich einfacher machen oder ob sie doch mehr Verwirrung als Ordnung stiften, bleibt bestehen. Parallel zum Modul-System der Service-Plattformen von OSGi und NetBeans [5], und in Konkurrenz zum Abhängigkeitsmanagement von Maven, wird es ein neuer Standard nicht leicht haben, von der Mehrheit der Programmierer akzeptiert zu werden. Die zum Teil heftigen Diskussionen, die es seit dem Erscheinen des Early-Draft zum Java-Modul-System [6] gab, lassen dabei nichts Gutes ahnen.

Es bleibt die Hoffnung, dass, wie von den Beteiligten angestrebt, aus dem Streit ein Dialog wird und es vielleicht gelingt, das Beste aus allen bestehenden Welten zu vereinigen. Ein Blick auf die Mitglieder der Expertengruppe zeigt, dass ihr neben den Vertretern vieler Firmen auch Fachleute der konkurrierenden Systeme angehören und ihr Know-how einbringen. Die Vorträge und Artikel des Architekten Stanley Ho [7], der immer wieder die Erweiterbarkeit und Integrierbarkeit des neuen Systems hervorhebt, lassen hoffen, dass die Java-Welt ein bisschen einfacher wird, wenn das Java-Modul-System mit Java 7, im nächsten oder spätestens übernächsten Jahr kommt.

### Referenzen

[1] Apache Software Foundation *Maven*, http://maven.apache.org

[2] OSGi Alliance *OSGi – The Dynamic Module System or Java*, http://www.osgi.org/

[3] Java Community Process *JSR 294: Improved Modularity Support in the Java Programming Language*, http://jcp.org/en/jsr/detail?id=294

[4] Java Community Process *JSR 277: Java Module System,* http://jcp.org/en/jsr/detail?id=277

[5] Sun Microsystems *NetBeans*, http://www.netbeans.org

[6] Java Community Process *JSR 277: Java Module System – Early Draft,* http://jcp.org/aboutJava/communityprocess/edr/jsr277

[7] Ho, Stanley *More JSR 277 Early Draft Q & A,* http://weblogs.java.net/blog/stanleyh/archive/2006/11/more_jsr277_ear.html

### Kurzbiographie



Jörn Hahn (joern.hahn@mathema.de) ist studierter Mathematiker und als Softwareentwickler und Trainer für die MATHEMA Software GmbH tätig. Zu seinen Themenschwerpunkten gehören Software-Entwicklung unter Java und Neuerungen der Programmiersprache Java, wie das Modul-System.

Copyright © 2008 Bookware 1865-682x/01/08/004 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

Des Programmierers kleine Vergnügen

# Über die Halbsumme[1] zweier ganzer Zahlen

von Michael Wiedeking

Eigentlich funktioniert unsere Software immer wunschgemäß. Aber gelegentlich schleichen sich trotz aller Sorgfalt Fehler ein, die niemand für möglich gehalten hat.

Müsste man innerhalb eines Arrays nach einem konkreten Element suchen, so würde man intuitiv einfach alle Elemente der Reihe nach durchsuchen und bei Erfolg die entsprechende Position zurück liefern.

```
static int search(int[] a, int key){
   for (int i = 0, n = a.length; i < n; i++) {
      if (a[i] == key) {
         return i;
      }
   }
   return -1;
}
```

Diese lineare Suche ist leider nicht besonders effizient, denn bei *n* Elementen muss man nämlich im schlechtesten Fall – wenn das gesuchte Element nicht enthalten ist – *n* mal vergleichen.

Deutlich einfacher geht es allerdings, wenn das Array bereits sortiert ist. Dazu sucht man sich aus dem Array das mittlere Element und prüft, ob das gesuchte größer oder kleiner als das gewählte Element ist. Dementsprechend reduziert man das Problem der Suche nun nur noch auf die obere oder untere Hälfte des Arrays. Diese Prozedur der Halbierung setzt man dann so lange fort, bis das Element gefunden oder das reduzierte Array leer ist.

Werden die Elemente des Arrays durch die Grenzen *low* und *high* beschrieben, so findet man das mittlere Element dieses Bereichs dadurch, dass man dieses mit (*low* + *high*) / 2 mittelt. Die Halbsumme wird bei vielen Algorithmen – insbesondere auch zum Sortieren – benutzt.

[1] Den Begriff Halbsumme scheinen eher die Astrologen zu verwenden. Der englische Begriff semi-sum gefällt mir da schon viel besser, wohingegen mir Semi-Summe zu gestelzt erschien. Summenhälfte würde es wohl auch treffen und Mittelwert hätte zu banal geklungen. So habe ich mich dann doch für die Halbsumme entschieden.

```
static int search(int[] a, int key) {
   int low = 0;
   int high = a.length - 1;
   while (low <= high) {
      int mid = (low + high) / 2; // Array "halbieren"
      int midVal = a[mid];
      if (midVal < key) {
         low = mid + 1;
      } else if (midVal > key) {
         high = mid - 1;
      } else {
         return mid; // key gefunden
      }
   }
   return -(low + 1);  // key nicht gefunden
}
```

Diese binäre Suche ist deutlich effizienter, denn man muss bei *n* Elementen nur noch $\log_2 n$ mal vergleichen (bei 1024 Elementen nur 10 mal, bei 1048576 nur 20 mal) bis man das gesuchte Element gefunden hat oder weiß, dass es im Array nicht enthalten ist. Der Algorithmus ist also so effizient, dass man sich gut überlegen muss, ob es sich nicht lohnt, dass Array zu sortieren bevor man nach einem Element sucht. Also, noch mal zum Beeindrucken: Ist das Element nicht enthalten, muss man bei zwei Milliarden Elementen linear zwei Milliarden mal vergleichen, binär nur 31 mal!

Leider funktioniert die oben angegebene Methode nur fast immer. Fast immer ist übrigens die Ursache für den überraschenden Tod jeder sonst noch so gut funktionierenden Applikation. Und der Grund dafür ist die Implementierung der Halbsumme als (*a* + *b*) / 2. Bei sehr großen Zahlen gibt es nämlich bei der Berechnung der Halbsumme einen Überlauf, was zu einem überraschenden Ergebnissen (z. B. einem Vorzeichenwechsel) führen kann.

Josh Bloch, der die binäre Suche für Java[2] implementiert hat, spricht in diesem Zusammenhang im Google Research Blog [1] von einem Schock, weil der von ihm verwendete Algorithmus aus dem berühmten Buch Programming Pearls von Jon Bentley [2] stammt, der dort die Korrektheit seines Algorithmus beweist und der leider trotzdem diesen Fehler enthält. Dieser Fehler blieb über zwanzig Jahre lang unentdeckt und die fehlerhafte Version von Bloch knappe neun Jahre. Offensichtlich ist es heutzutage (2004) [3] deutlich wahrscheinlicher, dass jemand ein Array mit mehr als $2^{30}$ Elementen sortieren will, als noch vor einigen Jahren.

Der Überlauf lässt sich natürlich dadurch umgehen, dass man für die Berechnung der Halbsumme kurzfristig auf ein long umsteigt. Damit hätte man aber immer noch das selbe Problem, falls man die Halbsumme zweier long-Werte bestimmen muss. Aus dem gleichen Grund kommt auch ein Umstieg auf double nicht in Frage.

[2] java.util.Arrays.binarySearch(...)

Bei einer allgemeinen Lösung ist es noch komplizierter, denn sollte die Halbsumme zweier beliebiger Zahlen bestimmt werden, so könnten keine grundsätzlichen Aussagen über die beteiligten Werte gemacht werden. Hier könnte *a* kleiner *b* sein oder umgekehrt *b* kleiner *a*, einer oder beide Werte könnten negativ sein.

Das ist deshalb so problematisch, weil die ganzzahlige Division unter Java und C# immer Richtung 0 rundet (d. h. 3 / 2 == 1, aber -3 / 2 == -1). Das führt dazu, dass bei unterschiedlicher Indizierung verschiedene Elemente gefunden werden (die Halbsumme von 1 und 4 liefert erwartungsgemäß 2 (das zweite von vier Elementen), die Halbsumme von -4 und -1 leider -2 (das dritte von vier Elementen).

Liefert die Summe *a* + *b* keinen Überlauf, so liefert zumindest für *a* + *b* >= 0, die ganzzahlige Division das korrekte Ergebnis, für *a* + *b* < 0 bedarf es einer anderen, leicht modifizierten Lösung, die mit Hilfe der Identität[3] (*a* + *b*) / 2 == (*a* + *b*) - (*a* + *b*) / 2 durch Einsatz der ganzzahligen Division das „falsche" Runden korrigiert:

```
static int semiSum1(int a, int b) {
int sum = a + b;
   if (sum >= 0) {
      return sum / 2;
   } else {
      return sum - sum / 2;
   }
}
```

Um zu vermeiden, dass es bei der Summe zu einem Überlauf kommt, kann man (unter der Annahme *a* <= *b*) anstatt die Summe zu halbieren, einfach nur die Hälfte der Differenz (*b* – *a*) zu a addieren:

```
static int semiSum2(int a, int b) {
   if (a <= b) {
      return a + (b - a) / 2;
   } else {
      return b + (a - b) / 2;
   }
}
```

Die erste Lösung muss nun immer dann Anwendung finden, wenn die Summe nicht überlaufen kann. Das ist genau dann sicher der Fall, wenn a und b verschiedene Vorzeichen haben.

```
static int semiSum(int a, int b) {
   if ((a >= 0 && b >= 0) || (a < 0 && b < 0)) {
      return semiSum2(a, b);
   } else {
      return semiSum1(a, b);
   }
}
```

Das allgemeine Berechnen der Halbsumme ist also nicht wirklich trivial. In [4] zeigt und beweist Salvatore Ruggieri sehr schön mit Hilfe von Hoare-Logik die Korrektheit dieser Berechnung.

[3] (*a* + *b*) / 2 == (2*a* – *a* + 2*b* – *b*) / 2 == ((2*a* + 2*b*) - (*a* + *b*)) / 2 == (*a* + *b*) - (*a* + *b*) / 2

Für den obigen Spezialfall der ausschließlich positiven Indizes in Java und C# reduziert sich der Aufwand wenigstens auf *low* + (*high* - *low*) / 2. Es geht aber zum Glück etwas einfacher. Das Schöne ist nämlich, dass die ganzzahligen Operation losgelöst davon funktionieren, ob die Werte vorzeichenbehaftet sind oder nicht. Wird die Summe zweier positiver ganzer Zahlen addiert, so kann sich der Übertrag höchstens auf das Vorzeichen auswirken. Nachdem die nachfolgende Halbierung ein positives Ergebnis liefern soll, ist eine Division durch zwei leider nicht möglich, weil hier das „falsche" Vorzeichen berücksichtigt würde.

Abhilfe schafft hier „unsigned" der Right-Shift-Operator mit *sum* >>> 1, der eigentlich wie *sum* / 2 funktioniert, nur das hier die Semantik des Vorzeichens unberücksichtigt bleibt, d. h. durch das Nachschieben von Nullen verschwindet das Vorzeichen und die gesuchte, korrekte Summe bleibt übrig. Die Halbsumme kann so viel einfacher mit

```
static int semiSum(int x, int y) {
   return (x + y) >>> 1;
}
```

berechnet werden. Das löst allerdings das Problem für negative Summen nicht.

Aber auch hier gibt es eine „triviale" Lösung. Für ganze Zahlen *x* und *y* gilt nämlich erfreulicher Weise (*x* + *y*) == ((*x* & *y*) + (*x* | *y*)) == ((*x* ^ *y*) + 2 * (*x* & *y*)). Wir können also (*x* & *y*) + ((*x* ^ *y*) / 2) verwenden, um die Halbsumme – ohne die Gefahr eines Überlaufs – zu berechnen. Die Division durch 2 kann dann auch noch durch ein Shift ersetzt werden:

```
static int semiSum(int x, int y) {
   return (x & y) + ((x ^ y) >> 1);
}
```

Und diese Variante ist auf meinem Rechner etwa doppelt so schnell wie die intuitiv hergeleitete Variante.

Referenzen


[1] Bloch, Joshua *Extra, Extra - Read All About It: Nearly All Binary Searches and Mergesorts are Broken*
http://googleresearch.blogspot.com/2006/06/extra-extra-read-all-about-it-nearly.html

[2] Bentley, Jon *Programming Pearls,* Second Edition, Addison-Wesley, 2000
siehe auch: http://netlib.bell-labs.com/cm/cs/pearls/

[3] Sun Developer Network *binarySearch() fails for size larger than 1<<30*
http://bugs.sun.com/bugdatabase/view_bug.do?bug_id=5045582

[4] Ruggieri, Salvatore *On computing the semi-sum of two integers*
Information Processing Letters archive, Volume 87, Issue 2 (July 2003), pp. 67 – 71
siehe auch: http://www.di.unipi.it/~ruggieri/Papers/semisum.pdf


Kurzbiographie



Michael Wiedeking ist Gründer und Geschäftsführer der MATHEMA Software GmbH, die sich von Anfang an mit Objekttechnologien und dem professionellen Einsatz von Java einen Namen gemacht hat. Er ist Java-Programmierer der ersten Stunde, „sammelt" Programmiersprachen und beschäftigt sich mit deren Design und Implementierung.

# Vom Unterschied, der den Unterschied macht

## Eine neues Programmier-Paradigma

von Nicole Delong

Vom ersten simplen Rechenknecht bis heute entwickelten sich unsere Computer auf rasante Art und Weise zu schnellen, leistungsfähigen, hochkompatiblen Maschinen. Kompliment, doch eines wurden sie leider nie: schlauer.

Noch immer sitzt der Mensch am Schreibtisch, schreibt kilometerlange Programme, hängt sie an andere kilometerlangen Programme, kürzt, flickt, kombiniert, schreibt. So ist und bleibt der Computer nach wie vor eine leblose Sammlung handgeschriebener Zeilen. Seine Leistung hängt von seinem Programmierer ab, genau wie seine Fehler. Doch das ist ihm leider, damals wie heute, vollkommen egal.

Ganz im Gegensatz zu uns. Denn allmählich verlieren wir den Überblick über unser eigenes Werk. Die Programme werden verflixt komplex und abertausende von Informatikern werden benötigt, um den steigenden Anforderungen gerecht zu werden, um noch ein bisschen was draufzusetzten, ein wenig dranzuhängen, umzustellen oder Fehler zu eliminieren.

Kann das ewig so weitergehen? Sollten wir nicht einmal kurz innehalten und versuchen, das Ganze noch einmal von vorne aufzurollen? Denn eines scheint klar, so sehr wir uns auch bemühen ein auch noch so neues und tolles Programm an ein älteres zu hängen, unser Wunsch der Computer könnte eines Tages aufstehen, uns einen schönen Tag wünschen und uns in den Urlaub schicken (es sei denn wir haben es so programmiert) wird sich so jedenfalls nie erfüllen.

Bleibt die Frage: Was kann der Mensch, was der Computer bis heute nicht kann? Denken, klar, doch wie tut er das, und was macht ihn letztendlich lebendig, reflektierend, kombinierend, nachdenklich, logisch, freundlich, intelligent? Steht hinter all dem vielleicht ein so einfacher Mechanismus, dass er sich in eine Maschine übersetzten lässt?

Betrachten wir doch einmal den Menschen, so wie eben den Computer, vom ersten Moment seiner Existenz an. Ein Säugling kommt auf die Welt und das einzige was er kann ist schreien: „Äääääääh“.

Und dann? Nach einigen Wochen schon quakt er plötzlich „Oh“, und „Iiie“, und so geht es weiter bis er schließlich „Mama“ sagt, „Papa“, „Wauwau“, „Mieze“. Und schließlich vollbringt er seine ersten logischen Sätze wie Wauwau fängt Mieze. Tagein, tagaus erweitert der Mensch sein Repertoire an Lauten, formt Wörter daraus, und kombiniert diese schließlich zu logischen Sätzen.

Doch wie tut er das?

Vor zweieinhalb tausend Jahren hat Aristoteles einen Mechanismus formuliert: Diairesis, was soviel heißt wie aus Zweien eine Wahl treffen.

Übertragen auf das Lernverhalten eines neugeborenen Menschen bedeutet das Folgendes: Ein Säugling kann (akustisch gesehen) nichts, als sein eigenes unangenehmes Empfinden ausdrücken, indem er Äh schreit. Die Mutter reagiert daraufhin mit einem besorgten Oh. Der Säugling hört sich selbst und dann seine Mutter und entdeckt einen ersten Unterschied, nämlich den zwischen den Lauten Äh und Oh. Somit kann er auch seine erste Wahl treffen.

Unwiderrufliche Vorraussetzung für seine Entdeckung ist allerdings, dass er zu dem Oh seiner Mutter einen persönlichen Bezug hat. Das Oh ist für ihn nur deshalb relevant, weil seine Mutter damit auf sein Äh reagiert. Der Säugling braucht also eine Beziehung zwischen sich selbst und seinem Gegenüber.

So und nicht anders lernt der Mensch. Er entdeckt einen Unterschied zwischen sich selbst und etwas außerhalb von ihm und setzt es in Beziehung zueinander. Dadurch ist es ihm möglich etwas Neues in seinen bisherigen Erfahrungsschatz mit aufzunehmen, sprich, er hat etwas dazugelernt.

Rasant dockt in seinem Köpfchen nun eine Erfahrung an die andere. Diesen Mechanismus drückte bereits Platon in seinem berühmte Menon-Paradoxon aus: „Wenn du etwas weißt, kannst Du es nicht lernen. Wenn du es nicht weißt, kannst du es auch nicht lernen, weil du nicht weißt, was du lernen sollst.“ Etwas Neues kann also nur erlernt werden, wenn es zum bereits vorhandenen Wissen in irgendeine Beziehung treten kann. Auch unser Volksmund sagt: Von nichts kommt nichts.

Schließlich kräht der Säugling seinen ersten logischen Satz „Wauwau fängt Mieze“. Gefolgt von „Mieze frisst Maus“. Und dann „Hund frisst Maus“. Scharf kombiniert, nicht?

Im menschlichen Gehirn entsteht ein fortschreitend komplexer werdendes Netzwerk aus miteinander verflochtenen Erfahrungen. Und es sind die Beziehungen der Erfahrungen untereinander, die dafür sorgen, dass ein Mensch selbstständig denkt und reflektiert. Anders als beim herkömmlichen Computer, bei dem eine Programmzeile beziehungslos an die nächste gehängt wird, entsteht im menschlichen Gehirn ein emergentes System.[1]

Wie schön wäre es doch für unsere lieben Lehrer, könnten sie ihren Schülern einfach nur ein von ihnen perfekt zusammengestelltes Wissensprogramm einflößen. Doch das geht leider nicht. Jeder Schüler muss den Lernstoff individuell mit seinen eigenen Erfahrungen in Beziehung setzten können. Erst dann kann er eine Aufgabe selbstständig lösen.

So kennen wir nun also zwei Mechanismen, die für ein emergentes und selbstständig denkendes Systems notwendig sind:

1. das Unterscheiden zwischen dem was man kennt und dem was man noch nicht kennt, und
2. das Integrieren des Neuen mit dem Bewusstsein über dessen Unterschied zum bereits Bekannten (Miteinander in Beziehung setzen).

Lässt sich so etwas auch in eine Maschine übersetzten?

Fangen wir doch noch ein drittes Mal von vorne an. Nehmen wir eine einfache leere Blechkiste (ein Betriebssystem und eine Verbindung zum Internet kann nicht schaden) und füllen sie mit einem klitzekleinen Programm das nur eines vermag, nämlich, zwei unterschiedliche Daten miteinander zu vergleichen (sie in Beziehung zueinander zu setzen), um den optimalen Unterschied zwischen ihnen zu ermitteln, und diesen dann zu speichern. Zeigen wir nun dieser Blechkiste die Farbe Weiß (mit Hilfe von optischen Sensoren die diese Farbe in eindeutige Daten übersetzten). Anschließend zeigen wir ihr die Farbe Schwarz. Und jetzt?

Nun, eines zumindest kann die Kiste mit diesen zwei unterschiedlichen Daten nun tun. Sie kann sie mit Hilfe ihres klitzekleinen Programms miteinander vergleichen, den optimalen Unterschied ermitteln und ihn speichern. Damit erhält sie nach Gregory Bateson ihre erste, selbstständig erarbeitete Information: „Information ist ein Unterschied, der einen Unterschied macht“ [1].

Die Blechkiste kann nun also zwischen Weiß und Schwarz unterscheiden, ohne, dass ihr je ein Mensch etwas von diesen Farben verraten hat, sprich ein klitzegroßes Programm dafür geschrieben hat.

Lassen wir die Maschine nun mit Hilfe ihrer Farbsensoren weitere Farben ablesen. Mit noch immer ein und demselben klitzekleinen Programm kann sie auch diese neuen Farben mit- und untereinander vergleichen (in Beziehung setzten) und den jeweiligen optimalen Unterschied in Form von PatternBits abspeichern. Farbe für Farbe erweitert sie so ihre Farbkenntnisse und zwar vollkommen selbstständig. Die Maschine kennt diese Farben nun. Und zwar allein aus ihrem Vermögen heraus, sie voneinander zu unterscheiden.

Gehen wir nun noch einen Schritt weiter und zeigen wir der Blechkiste einen hellgrünen Fleck auf einem dunkelgrünen Hintergrund. Und jetzt? Nun wer hat behauptet, das klitzekleine Programm in der Kiste könne nur Farben voneinander unterscheiden? Nein, es ermittelt den optimalen Unterschied zwischen Daten, ganz egal welcher Art. Und das bedeutet, dass sich die Blechkiste keineswegs nur auf Farben beschränken muss. Liegt der optimale Unterschied nicht in der Farbe sondern in der Helligkeit, speichert sie eben diesen ab. Liegt er in der Form, ermittelt und speichert sie einen Formunterschied. Liegt er im Klang, speichert sie den. So erarbeitet sich diese einfache Blechkiste vollkommen selbstständig eine Information nach der anderen und wird eventuell bald weiser sein als wir selbst.

Von nun an braucht man einen Computer nur noch mit Fakten füttern. Programmieren? Überflüssig. Dadurch, dass ein Computer Unterscheidungskraft besitzt, und Unterschiede aller Art miteinander in Beziehung setzten kann, wird er von nun an vollkommen selbstständig sein.

Und eines nicht allzu fernen Tages wird er auch mit uns in Beziehung treten können. Und dann wird er aufstehen und uns in den wohlverdienten Urlaub schicken.

Referenzen

[1] Bateson, Gregory *Ökologie des Geistes. Anthropologische, psychologische, biologische und epistemologische Perspektiven,* Suhrkamp, Frankfurt, 1981

Weiterführende Literatur

- Wikipedia *Gregory Bateson*, http://de.wikipedia.org/wiki/Gregory_Bateson

Kurzbiographie



Nicole Delong ist studierte Grafikerin und Tochter eines Erfinders (dem Autor des nachfolgenden Artikels). Ihr Wissen über Computer ist auf berufsbedingten Umgang beschränkt, was sie an die von ihr beschriebene Utopie hoffen und glauben lässt. Denn bislang haben sie Computer leider nur urlaubsreif gemacht.

[1] Emergenz: Ein Lebewesen ist mehr als die Summe seiner Teile, wobei dieses mehr erschaffen wird durch die Beziehungen der einzelnen Teile untereinander, sowie die Beziehung des Lebewesens zu seiner Umwelt.

# Selbststudium für Computer

## Skizze einer Theorie zum autonomen, maschinellen Wissenserwerb

von Wolf-Rüdiger Delong

Seit es Maschinen gibt träumt der Mensch davon eine zu bauen, die ihm ähnlich ist und ihm lästige Arbeit abnehmen kann. Seit es Computer gibt scheint die Verwirklichung in greifbare Nähe gerückt zu sein. Auch wenn sich die Prognosen aus den Anfängen der Künstlichen Intelligenz nicht bewahrheitet haben, hat man dennoch große Fortschritte gemacht. Aber trotzdem haben alle Ansätze die Schwäche, dass der Mensch dem Computer den Rahmen vorgibt. Dieser Artikel skizziert ein Modell, in dem sich der Computer seinen Rahmen selbst erarbeiten kann.

Nehmen wir einmal an, dass alles was in der Welt existiert, nennen wir es die Referenzen[1], in Relation, in irgendeiner Beziehung zueinander steht. Keine Referenz existiert als Solitär, kann als Solitär existieren. Jede Referenz hat mindestens eine Relation, eine Beziehung zu einer anderen Referenz. Damit lässt sich im Umkehrschluss argumentieren: Wenn keine Referenz für sich allein existieren kann, ist es die Menge seiner Relationen zu anderen Referenzen, die eine Referenz überhaupt erst definieren. Die Relationen können als die verschiedenen Blickwinkel auf eine Referenz angesehen werden.

Damit ist die kleinst mögliche Wissenseinheit über die Welt (WE) definiert als Beziehung zwischen zwei Referenzen:

$$WE_{ijk} = Relation_k(Referenz_i, Referenz_j) \quad (1)$$

[1] Anschaulicher wäre hier der Begriff Objekt statt Referenz. Unter Referenz sollen aber auch immaterielle Begriffe wie Geist, Idee usw. verstanden werden.

Umgekehrt ist also die Definition (Def) einer beliebigen $Referenz_i$ gerade die Menge aller Relationen, die mit der $Referenz_i$ in einer Beziehung stehen:

$$Definition(Referenz_i) = \{WE_{ijk} | Relation_k(Referenz_i, Referenz_j)\} \quad (2)$$

Ein starkes Indiz dafür, dass der Mensch die Welt auf diese Weise versteht, ist die menschliche Sprache. In allen natürlichen Sprachen der Welt ist der Kern eines Satzes immer ein Konstrukt der Form [1]:

$$Referenz_1 - Relation_0 - Referenz_2$$
$$Hund - jagt - Katze$$

Daraus ergibt sich: Auch ein Mensch ist eine Referenz. Eine sehr komplexe zwar, aber nur eine Referenz. Er ist ebenso beschreibbar durch die Beziehungen zu anderen Referenzen. Daraus ergibt sich die Frage, was heißt beschreibbar? Wer beschreibt was, wie und warum?

Ein Mensch kann diese Tripel aus einer Beziehung und zwei Referenzen niemals objektiv erfahren. Er kann das Tripel nur mit Hilfe seiner Sensoren, den Sinnesorganen vermessen. Er kann es nur an den Eigenschaften, den Attributen der Relationen und Referenzen erkennen. Er kann nur feststellen, wie es sich den Sinnesorganen gegenüber repräsentiert. Nur solche Messergebnisse über Relationen und Referenzen kann der Mensch erfahren und speichern. Nicht anders, als jedes andere Lebewesen auch. Deshalb kann feststellen nur als Prozess aufgefasst werden. Als aktiver Prozess des Beobachtens.

Und deshalb können solche Speicher mit ihren Speicherinhalten ebenfalls als Referenzen betrachtet werden, die in Beziehungen zu anderen Referenzen in der Welt oder anderen Speichern stehen.

### Praktische Konsequenzen

Wir hatten die kleinst mögliche Wissenseinheit $W_{ijk}$ als $Relation_k(Referenz_i, Referenz_j)$ definiert. Wie muss man sich unter all diesen Gesichtspunkten Relationen und Referenzen praktisch vorstellen? Gregory Bateson hat definiert [2]:

„Information ist der Unterschied, der den Unterschied macht".

Bateson fasst Information als Prozess auf. Damit deckt sich seine kleinst mögliche Informationseinheit vollständig mit unserer Definition der kleinst möglichen Wissenseinheit, wenn wir unsere Relation als einen oder mehrere Prozesse definieren. Eine Relation ist damit die Menge der Prozesse, die zwei Referenzen miteinander verknüpfen. Diese Menge muss mindestens einen Prozess enthalten, nämlich den, die beiden in Beziehung stehenden Referenzen voneinander zu unterscheiden.

Jedes einzelne Neuron eines Gehirns ist prinzipiell in der Lage, eine solche Unterscheidung zu treffen, indem es feuert oder nicht. Das menschliche Gehirn verfügt zwar über mehr als 100 Milliarden Neuronen, was sich nach viel anhört. Gemessen an der Zahl möglicher Wissenseinheiten in der Welt ist diese Zahl aber sehr klein. Der Mensch muss also erkennen, was für ihn vorrangig und was nachrangig ist. Und wie kann er das?

Platon hat schon vor über zweitausend Jahren in seinem Politikos [3] einen Hinweis gegeben [4]:

Während die Methode der Dihairesis[2] das Ziel hat, Dinge voneinander zu unterscheiden, haben Paradigmata die Aufgabe, Dinge zu einer Einheit zusammenzurücken, indem sie Ähnlichkeiten aufzeigen.

Methode ist nur als Prozess zu verstehen, genau wie bei Bateson. Lassen sich auch Platons Paradigmata in unser System einordnen? Wie sind sie zu interpretieren?

Mit seinen Sensoren kann ein Lebewesen lediglich Eigenschaften von Relationen und Referenzen erfassen und messen. Eindimensionale Messergebnisse können an Skalen aufgetragen werden. Beispiel hell – dunkel, warm – kalt usw. Die unterschiedlichen Enden von Skalen repräsentieren stets ein sprachliches Oppositionspaar oder die Enden sind zyklisch geschlossen (Wellen).

Mehrdimensionale Messgrößen lassen sich an Feldern auftragen. Beispiel Farben wie rot, grün, blau, gelb usw. im dreidimensionalen Farbraum rot-grün-blau. Auf diese Skalen oder Felder lässt sich nun wiederum unsere Grundannahme anwenden. Zwei Referenzen lassen sich genau dann unterscheiden, wenn ihre Merkmale an unterschiedlichen Stellen auf einer Skala liegen. Felder lassen sich auf Skalen zurückführen, indem zwischen je zwei Feldelementen eine Skala aufgespannt wird (die Skala rot/grün oder die Skala rot/blau usw.).

Platons Paradigmata können daher als Skalen oder Felder aufgefasst werden und Bewertungen können dann dadurch vorgenommen werden, dass die Lage einer Eigenschaft einer Relation oder einer Referenz auf einer Skala bewertet wird. Je näher die Eigenschaften zweier Relationen oder Referenzen auf einer Skala liegen, umso ähnlicher sind sich die Relationen oder Referenzen. Selbstverständlich sind Skalen ihrerseits Referenzen, die zu anderen Referenzen in Beziehung stehen oder in Beziehung gesetzt werden können, um neue Skalen zu erzeugen.

Dihairese ist der generelle Prozess des Unterscheidens, mit dem elementare Wissens- oder Informationseinheiten realisiert werden können. Dihairese erzeugt Skalen oder Felder, an Hand derer weitere elementare Informationseinheiten realisiert werden können.

[2] Diairesis (griechisch) „Unterscheidung“ aus di – „zwei“ und hairesis – „Wahl“ [4].

Dihairese besitzt zwei weitere Eigenschaften, welche die Strukturierung eines komplexen Wissensbestandes erlauben.

1. Wenn etwas unterschieden werden soll, so muss das zu Unterscheidende etwas Gemeinsames haben und innerhalb des Gemeinsamen etwas Trennendes.

Dies erlaubt die wichtigen menschlichen Fähigkeiten der Deduktion und der Induktion, denn die zu unterscheidenden Referenzen sind stets an ihren Oberbegriff gekoppelt. Das ist die Grundlage zur Bildung von Hierarchien.

Beispiel: Induktion < > Deduktion
Lautgebung < > (Hund) bellt, (Katze) miaut

2. Skalen können Widersprüche aufdecken.

Stößt das System beispielsweise auf zwei Aussagen wie:
Mann 1 ist arm (Einkommen 10.000 €/Jahr)
Mann 2 ist reich (Einkommen 1.000 €/Jahr)

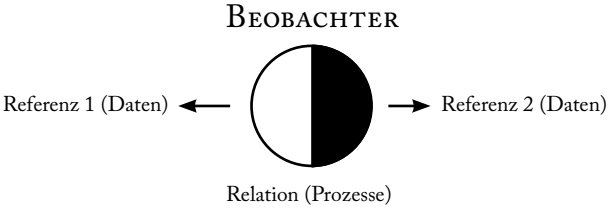
dann ist das ein Widerspruch. Dieser lässt sich über weitere Referenzen im Kontext aufklären: Mann 1 lebt in Deutschland, Mann 2 in der Sahelzone.

Beim autonomen Wissenserwerb gibt es ein grundsätzliches Problem, das ebenfalls schon Platon aufgeworfen hat, das Menon-Paradoxon:

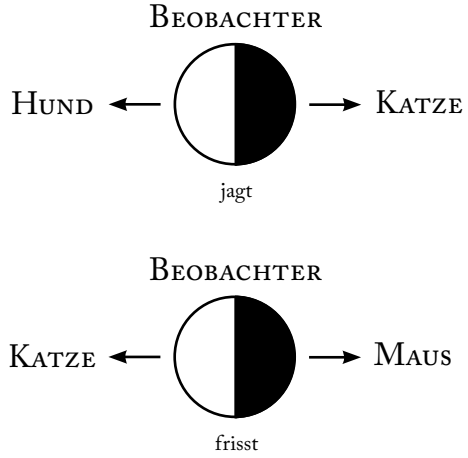
Wenn du etwas weißt, kannst du es nicht lernen, wenn du es nicht weißt, kannst du es auch nicht lernen, weil du nicht weißt, was du lernen sollst.

Durch die obige Definition Def (Referenz$_i$) = {WE$_{ijk}$}, lässt sich das Menon-Paradoxon sehr elegant auflösen und damit die Strukturbildung des Wissens angeben.

Die Grundannahme, Relation$_k$(Referenz$_i$, Referenz$_j$), lässt sich in einem graphischen Symbol darstellen. In Anlehnung an die zwei Zustände eines üblichen Computer-Bits soll eine solche Einheit PatternBit heißen.




Damit lässt sich ein Sachverhalt in der Welt wie z. B. „Hund jagt Katze“ in genau einem PatternBit speichern. Dann kann neues Wissen über Koinzidenz der Referenzen angedockt werden.




Die neue Referenz Maus kann deshalb angedockt werden, weil die Referenz Katze schon bekannt ist. Außerdem wird Katze damit genauer spezifiziert. Es wird ein weiterer Blickwinkel auf die Referenz Katze geworfen.

## Prozesse

Das System wurde bisher durch zwei Gleichungen beschrieben:

1. beschreibt die Definition einer elementaren Informationseinheit
2. erlaubt die Strukturbildung zu komplexen Informationsnetzen über die Koinzidenz von Referenzen

Demnach fehlt jetzt eine dritte Gleichung, welche die Relationen, d.h. die Prozesse beschreibt. Die Hirnforschung sagt [6]:

> *Das weist darauf hin, dass in der Hirnrinde pluripotente Verarbeitungsalgorithmen realisiert sind, die für eine Vielzahl von Funktionen verwendet werden können und deren Iteration allein zur Emergenz qualitativ neuer Leistungen führen kann.*

Zwei dieser Prozesse sind klar und im gesamten System einheitlich. Sie betreffen die Unterscheidung von zwei Referenzen. Zwei Prozesse deshalb, weil

3 a. das Lernen einer Unterscheidung und
3 b. die Anwendung des Gelernten unterschieden werden muss.

Diese Prozesse werden im System durch die Klassifikation mittels Trennebenen dargestellt. Auf unterster Ebene, d. h. auf der Verarbeitungsstufe von den Sensoren bis zu formal prozessierbaren Skalen oder Feldern sind sicher eine ganze Reihe spezieller Prozesse zu programmieren, wenn man bedenkt wie komplex z. B. das Hörorgan aufgebaut ist. Diese unterliegen natürlich ebenfalls den Gleichungen (1) und (2), die eigentliche Komplexität steckt hier aber in den Prozessen.

Auf den höheren Ebenen des Sprachverstehens sind aber möglicherweise keine speziellen Prozesse mehr erforderlich.

## Referenzen

[1] Schulze, Wolfgang *Sprache als kommunizierte Wahrnehmung*, in Druck, 2007

[2] Bateson, Gregory *Ökologie des Geistes. Anthropologische, psychologische, biologische und epistemologische Perspektiven,* Suhrkamp, Frankfurt/M., 1981

[3] Platon *Politicos* http://www.textlog.de/platon-politikos.html

[4] Colloud-Streit, Marlis *Warum gibt es keinen Mythos im Sophistes* http://www.nd.edu/~plato/plato4issue/Marlis.pdf

[5] Wikipedia *Dihairese* http://de.wikipedia.org/wiki/Dihairese

[6] Singer, Wolf *Der Beobachter im Gehirn*, Suhrkamp, Frankfurt/M., 2002

## Kurzbiographie



Wolf-Rüdiger Delong ist studierter Nachrichtentechniker. Er war in einem großen deutschen Konzern für die Entwicklung sehr schneller Bildrechner in der Medizintechnik zuständig. Heute gilt seine Leidenschaft der Forschung an der möglichen Grundlage für Intelligente Systeme.

Copyright © 2008 Bookware 1865-682x/08/01/007 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

# Destruktive Robustheit

## Wie gut gemeinte Fehlervermeidung das Leben erschwert

von Jan Lessner

Robustheit bezeichnet die Fähigkeit eines Systems, mit Ausnahmesituationen klar zu kommen. In übertriebener Form ist diese Eigenschaft aber hinderlich, weil sie Fehlerfälle verschleiert und die Analyse erschwert. Dieser Artikel zeigt an ein paar einfachen Beispielen wie schnell das passieren kann.

### Mit besten Absichten

Fehler sind das Letzte, was man in einer Software haben will, und so ist die Fehlervermeidung konsequenter Weise eine der höchsten Tugenden bei der Programmierung. Nun ist aber bekanntermaßen vollkommene Fehlerfreiheit in nennenswert großen Systemen kaum zu erreichen, und daher muss man sich auch der unangenehmen Nebenbeschäftigung, dem Behandeln von Fehlern, widmen.

Auch dabei besteht bei vielen Entwicklern ein Hang zur quasi „nachträglichen" Fehlervermeidung durch Kompensation, mit der guten Absicht, die Software selbst in unvorhergesehenen Situationen am Leben zu erhalten. Wenn das gelingt, spricht man von einem „robusten" System, was grundsätzlich betrachtet eine positive, nichtfunktionale Eigenschaft ist.

Geht die Kompensation aber zu weit und neigt dazu, aufgetretene Fehler unter den Teppich zu kehren, gefährdet sie andere gleichbedeutende Eigenschaften, wie z. B. die Datenintegrität. Dazu ein kleines, plakatives Beispiel, das den Grundgedanken verdeutlicht.

```
try {
  arbeite();
} catch (Exception x) {
  ;
}
```

Tritt in der Funktion *arbeite* ein Fehler auf, wird diese beim Aufrufer genauso akzeptiert, als wenn die Funktion fehlerfrei gearbeitet hätte. Das Programm läuft unverändert weiter, ungeachtet der Tatsache, dass die zu leistende Arbeit gar nicht getan wurde. Es gibt durchaus Fälle, wo solch extreme Robustheit tatsächlich erwünscht ist, aber dann muss es im Code wenigstens durch einen Kommentar erkennbar sein.

```
try {
  sendeBestellBestätigungsEmail();
} catch (Exception x) {
  // Weiter im Programm, auch wenn
  // der E-Mail-Versand nicht geklappt
  // hat. Der Kunde bekommt ohnehin
  // schon am nächsten Tag eine
  // Rechnung zugeschickt
}
```

Die erkennbare Bewusstheit der Kompensation ist ein wichtiger Punkt, wenn es um die Frage geht, ob Robustheit im richtigen Sinne eingesetzt wurde. Nicht immer ist der Fall so klar erkennbar wie oben. Einer nur leicht abweichenden Variante – hier am Beispiel einer Datenbankanfrage – begegnet man in der Praxis schon öfter.

```
try {
  return queryBestellungen();
} catch (SQLException x) {
  return new Bestellung[0];
}
```

Mit den besten Absichten mag sich hier der Programmierer überlegt haben, den Fall irgendwelcher Kommunikationsprobleme mit der Datenbank auf die leere Ergebnismenge abzubilden und damit dem Aufrufer zu ersparen, den Fehlerfall separat zu behandeln. Wenn der Aufrufer *nichts* bekommt, wird er auch *nichts* verarbeiten und daher passiert auch *nichts* Falsches.

Soweit plausibel gedacht, die Frage ist nur, wie jemals irgendjemand einen Fehlerfall als solchen erkennen kann, um ggf. Behebungsmaßnamen zu ergreifen. Der Fall einer heruntergefahrenen Datenbank stellt sich genauso dar wie ein Programmierfehler in der Funktion *queryBestellungen* und dies beides wiederum genauso wie der Fall, dass tatsächlich keine Bestellungen in der Datenbank vorliegen.

Die gut gemeinte Robustheit stellt hier keinen Vorteil dar, sondern sorgt im Gegenteil für eine Erschwernis bei der Suche nach Fehlern. Diese Art der Robustheit wirkt sich destruktiv aus.

## Null-Sicherheit

Auch das zweite Beispiel oben ist noch relativ leicht als fragwürdig zu durchschauen. Undurchsichtig wird es dort, wo es nicht um Fehler in Form von Ausnahmen geht, sondern um anderweitig unvorhergesehene Situationen. Allen voran ist dies der Umgang mit Null-Werten. Ein weit verbreitetes Muster für die Vermeidung von NullPointerExceptions ist der verdrehte Vergleich von Werten mit Konstanten [1].

```
if ("bezahlt".equals(bestellStatus)) {
  bestellungAusliefern();
}
```

Diese Art des Vergleichs führt auch dann zum Ergebnis, wenn *bestellStatus* **null** ist. In vielen Fällen ist so ein verdrehter Vergleich aber ein ganz typischer Fall von destruktiver Robustheit, denn der Fall, dass der zu prüfende Wert unvorhergesehener Weise Null ist, wird dabei unter den Teppich gekehrt.

Besser ist es hier, den Vergleich richtig herum vorzunehmen (was schon aus ästhetischen Gründen zu empfehlen ist) und den Null-Fall zuvor über eine Assertion abzusichern.

```
assert (bestellStatus != null);
if (bestellStatus.equals("bezahlt")) {
  bestellungAusliefern();
}
```

Natürlich ist es nicht schön, dass nun das Programm mit einem AssertionError abgebrochen wird, wenn sich bei der Ermittlung des Bestellstatus ein Fehler eingeschlichen hat und der Wert unbeabsichtigt Null ergibt. Aber es bringt das Wartungspersonal bei einer Fehleranalyse schon wesentlich näher an die Quelle des Problems.

Der verdrehte Vergleich hätte zur Folge, dass Bestellungen, bei denen der Fehler auftritt, einfach nicht ausgeliefert werden und ansonsten keinerlei Fehlermeldung erfolgt. Bei der Analyse würde man vermutlich an einer völlig falschen Stelle ansetzen, da das Systemverhalten irreführend ist.

Die Verwendung von **assert** ist übrigens nicht zwingend notwendig, um auf der sicheren Seite zu sein. Schließlich muss man nicht in jeder Zeile mit dem Schlimmsten rechnen und den Programmcode durch übermäßigen Gebrauch von Assertions „verschmutzen". Lässt man sie weg, erhält man im Fehlerfall eine NullPointerException. Das ist zwar nicht so schön, weil es den Eindruck erweckt, der Fehler liege in dem Code, der die Exception produziert – ein AssertionError hingegen macht deutlich, dass der Fehler beim Aufrufer des Codes liegt. Trotzdem ist die NullPointerException immer noch besser, als den Fehler zu verschweigen.

Und was ist, wenn Null-Werte beim Vergleich ausdrücklich erlaubt sind? Hier gilt das gleiche, wie schon im Beispiel des leeren **catch**-Blocks: Es sollte erkennbar sein, dass die Kompensation bewusst vorgenommen wurde. Im Falle des Null-sicheren Vergleichs ließe sich das ebenfalls mit einem Kommentar bewerkstelligen.

Noch schöner ist es aber, wenn der Code für sich selbst spricht, daher wäre eine weniger kompakte Formulierung deutlich aussagekräftiger.

```
if ((emailReady != null) && emailReady.equals("ja")) {
  sendeEmail();
}
```

Hier wird auch für einen Außenstehenden sofort deutlich, dass Null-Werte bewusst in Kauf genommen werden.

## Design-By-Contract

Im folgenden Beispiel ist die bewusste Null-Sicherheit im Code deutlich zu sehen, allerdings stellt sich hier die Frage, ob dies zu einer sinnvollen Robustheit führt:

```
void aktualisiere(Bestellung b) {
  if (b ! = null) {
    // Führe SQL-Update durch
  }
}
```

Die Funktion soll Daten einer Bestellung in einer Datenbank aktualisieren und kann Dank der Null-Prüfung auch ohne Bestellung aufgerufen werden. Sehr robust zwar, aber wofür? Hier wird potentiellen Aufrufern ein sinnloser Freiheitsgrad gegeben, der vor allem die Gefahr unbemerkt fehlerhafter Nutzung der Funktion birgt.

Besser wäre auch hier wieder die Verwendung einer Assertion, um Null-Werte als Übergabeparameter zurückzuweisen:

```
void aktualisiere(Bestellung b) {
  assert (b ! = null);
  // Führe SQL-Update durch
}
```

Hier kommen die Prinzipien des sogenannten Design-By-Contract[1] zum tragen [2]. Eine Grundidee darin sieht die Absicherung des korrekten Aufrufs von Funk-

[1] "Design by Contract" ist eingetragenes Warenzeichen von Interactive Software Engineering.

tionen durch Vorbedingungen und die Absicherung des korrekten Ablaufs von Funktionen durch Nachbedingungen vor.
Diese Prinzipien stammen ursprünglich aus der Programmiersprache Eiffel [3], sind aber von dieser im Wesentlichen unabhängig. In Java, C# und in anderen Programmiersprachen lassen sie sich rudimentär mit Hilfe von Assertions umsetzen.

Darüber hinaus gibt es Toolkits zur weiterreichenden Unterstützung für Java (OVal [4], Contract 4J [5]), C# (Spec# [6]), Perl (Class Contract [7]), C (DBC 4C [8]) und D (Contract Programming [9]).

Losgelöst von einer konkreten Umsetzung sind die Konzepte eine empfehlenswerte Grundlage, wenn es um die Frage geht, ob eine Software in sinnvollem Maße robust gestaltet wurde. Stehen die programmierten Fehlerkompensationen im Widerspruch zu einem Design-By-Contract, könnte dies ein Hinweis auf destruktive Robustheit sein, die mehr schadet als nutzt.

## Referenzen


[1] Wikibooks *Java Programming – Preventing NullPointerExeption* http://en.wikibooks.org/wiki/Java_Programming/Preventing_NullPointerException

[2] Meyer, Bertrand *Object-Oriented Software Construction*, Prentice-Hall, 1988

[3] Meyer, Bertrand *Eiffel: The Language,* Prentice-Hall, 1992

[4] SourceForge *OVal – The Object Validation Framework for Java 5 or later*, http://oval.sourceforge.net

[5] SourceForge *Contract4J: Design by Contract for Java*, http://www.contract4j.org

[6] Microsoft Research *Spec#*, http://research.microsoft.com/specsharp

[7] CPAN *Class::Contract – Design-by-Contract OO in Perl*, http://search.cpan.org/~ggoebel

[8] RubyForge *DBC for C*, http://dbc.rubyforge.org

[9] Digital Mars *Contract Programming*, http://www.digitalmars.com/d/dbc.html


## Weiterführende Literatur

- Venners, Bill; Meyer, Bertrand *Design by Contract – Conversation with Bertrand Meyer,* December 2003 http://www.artima.com/intv/contracts.html
- Wikipedia *Design by Contract*, http://de.wikipedia.org/wiki/Design_By_Contract

## Kurzbiographie



Jan Lessner (jan.lessner@mathema.de) ist als Software-Architekt, IT-Consultant und Trainer für die MATHEMA Software GmbH tätig. Er hat langjährige Erfahrung im Design komplexer, verteilter Systeme, in den letzten Jahren mit dem Schwerpunkt Java EE. Neben seiner Projekt-Tätigkeit hält er Technologietrainings in Deutschland und der Schweiz.

Copyright © 2008 Bookware 1865-682x/01/08/008 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.



Herbstcampus

Wissenstransfer par excellence

Der Idee des **Herbstcampus** liegt die Motivation zugrunde, eine überregionale Technologiekonferenz für Software-Entwickler und Projektleiter in Nürnberg zu schaffen, die sowohl durch hochwertige Wissensvermittlung als auch durch gezielten, nutzbringenden Interessensaustausch überzeugt.

Die Schwerpunkte des **Herbstcampus** liegen auf dem .NET-Framework und der Java-Technologie.

Der **Herbstcampus** bietet ein umfassendes und hochwertiges Vortragsprogramm mit namhaften Referenten, das den Teilnehmern wichtiges Know-how vermittelt und über sämtliche aktuellen Entwicklungen informiert.

in der Woche vom
15.–19. September 2008
in Nürnberg

http://www.herbstcampus.de

Deutsch für Informatiker

# Das Kreuz mit der elektronischen Post

von Kerstin Brockmann

Mit der deutschen Sprache ist es ja oft wie mit dem Essen: Manchmal gibt es „Schmankerl“, die man sich sprichwörtlich auf der Zunge zergehen lassen kann. Hübsch formulierte Artikel in Magazinen oder Zeitungen, ein gut geschriebenes Buch, ein Satz, geschmückt mit einem der seltenen „Schatzkästchen“-Wörter unseres Wortschatzes – so macht Deutsch richtig Spaß. Doch vieles, was man so sagt, liest oder hört, hat einen komischen Beigeschmack oder ist schlicht und einfach nicht in Ordnung.

Gerade in der IT wimmelt es von völlig unverständlichen Abkürzungen, E-Mails werden in furchtbarem Stil und in unvollständigen Sätzen versendet, und auch die Grammatik, die Rechtschreibung sowie der gute Ton kommen in vielen Fällen zu kurz. Doch muss man sich damit einfach so abfinden? Nein, muss man nicht!
Als Liebhaberin der deutschen Sprache bin ich der Meinung, dass unsere schöne Sprache nicht weiter verkommen darf. Ich möchte den Lesern deshalb einige Eigenheiten und Besonderheiten, an die man im täglichen Gebrauch kaum noch denkt, nahebringen und so einen Beitrag dazu leisten, dass Informatiker „wie du und ich“ sich wieder mehr Mühe geben bei der Kommunikation in Wort und Schrift.

Dieser Artikel widmet sich der Kommunikation via E-Mail, der wohl beliebtesten Form des Austauschs unter Informatikern und allen, die mit ihnen zu tun haben.

Der Siegeszug der E-Mail begann gegen Ende der 1980er Jahre, und auf der Beliebtheitsskala der Internet-Benutzer rangiert sie ungeschlagen auf Platz eins (noch vor dem World-Wide-Web). Wußten Sie eigentlich, dass für den Begriff „E-Mail“ im Deutschen drei unterschiedliche Artikel möglich sind?

- Das E-Mail – vom Englischen „e-mail“
- Der E-Mail – nach der deutschen Übersetzung „der elektronische Brief“
- Die E-Mail – nach der deutschen Entlehnung „die elektronische Post“

Ich bevorzuge definitiv die feminine Form, wobei angeblich hier in Süddeutschland auch das Neutrum weit verbreitet sein soll. Auch bei den Schreibweisen gehen die Geschmäcker auseinander. Manche schreiben Email (leicht zu verwechseln mit „Emaille“, dem Schmelzüberzug z.B. auf metallenem Geschirr), andere eMail (in Anlehnung an Begriffe wie „eCommerce“ oder „eGovernance“). Laut Duden sind die Schreibweisen Email und E-Mail erlaubt, wobei aber die Schreibweise mit dem Bindestrich die bevorzugte ist. Nicht nur die Schreibweise und der Artikel werden bei E-Mails sehr unterschiedlich gebraucht – auch die inhaltliche, grammatikalische und formale Gestaltung variiert sehr stark [1].

Neulich erhielt ich zum Beispiel den Urlaubsantrag eines Kollegen mit dem Titel:

*Urlaub am 2007-05-30,*

der Inhalt blieb (bis auf die automatisch generierte Adress-Fußnote) leer. Hm!

Ähnlich kurz und knapp war die Antwort eines unserer freien Mitarbeiter auf eine ausführliche Projektinformationsmail, die ich mit der Bitte um seine Handynummer und eine kurze Terminbestätigung abgeschlossen hatte. Die Antwort war:

*Die Handynummer ist 555 4321. Einen schönen Feierabend!*

Na, immerhin war in dieser E-Mail eine Grußformel enthalten.

Doch ich möchte mich nicht beklagen, denn solche Mails sind ja noch verständlich. Kürzlich erzielt ich eine Nachricht von einem Kunden, die sich ungefähr so las:

*Hallo Fr. Brinkmann,*

*Hr. Ding kann am Montag 8.7. anfangen. Die vertragliche Details klären noch in der nächsten Woche.*

*MfG,*

*D. Ong*

Aaaaha! Dazu fällt mir nur ein: ich heiße nicht Brinkmann, sondern Brockmann! Und ich frage mich, ob man nicht mindestens einmal kurz das Geschriebene

durchlesen kann, bevor man den „Senden"-Knopf drückt?

Lassen Sie mich etwas weiter ausholen: Früher, vor langer, langer Zeit, gab es das Medium E-Mail noch gar nicht. Damals schrieb man sich Briefe, (ja, tatsächlich auf Papier!) die normalerweise einer festen Form genügten. Von Adresse über Betreffzeile, Anrede, Inhalt und Grußformel musste in einem Geschäftsbrief alles enthalten sein – sonst hätte das schon sehr komisch ausgesehen. Hatte man es eilig oder gab es nicht viel zu sagen, schickte man ein Telegramm, das dann weniger formlos und so kurz wie möglich formuliert werden durfte.

Ein ganz wichtiger Aspekt bei der Briefeschreiberei war damals, dass sich die Meisten gar nicht selber darum kümmern mussten, die Form des Mediums Brief einzuhalten. Damals kümmerte man sich lediglich grob um den Inhalt, indem man z. B. der Sekretärin die wichtigsten Punkte diktierte oder in Stichpunkten notierte. Für die Ausformulierung und äußere Gestaltung des jeweiligen Briefs war somit jemand zuständig, der gelernt hatte, wie man so etwas macht. Aus diesem Grund erhielt man also meist Briefe, die gut aussahen, alle wichtigen formalen Kriterien erfüllten und zudem nett formuliert waren.

Später gab es dann die elektronische Post, die heute einen Großteil der geschäftlichen Kommunikation abdeckt. Sie ist einerseits sehr schnell und bequem, andererseits auch oft sehr verwirrend, unklar, mehrdeutig und missverständlich. Aus der Zeit des Briefeschreibens ist also nicht mehr viel geblieben. Das liegt einerseits daran, dass man einfach nicht immer Zeit und Lust hat, alles der Form gerecht zu gestalten. Andererseits ist der Grund wohl auch, dass inzwischen jeder, wirklich jeder, E-Mails um die Welt schickt, wobei aber beileibe nicht jeder gelernt hat, wie man einen Brief schreibt und was – wenn auch nur der Form halber – hinein gehört.

Klar, auch ich schreibe nicht jedes Mal einen Roman (oder eine Kolumne), wenn ich eine E-Mail versende. Aber ich vertrete die Ansicht, dass der Inhalt einer E-Mail immer dem Zweck angemessen sein sollte.

Eine weitere Episode hierzu: Während meines Studiums hatte ich verschiedene Vorlesungen bei einem Professor, der seine Studenten ermutigte, ihm Fragen zum Lernstoff, Anregungen oder sonstige Anliegen gern per E-Mail zu senden. Eines Tages aber kam dieser Professor mit einer ungewöhnlich miesen Laune in den Vorlesungssaal und hielt allen Anwesenden einen viertelstündigen Vortrag darüber, dass man, nur weil man E-Mails schreibt, die Höflichkeit des täglichen Umgangs nicht vergessen dürfe. Hintergrund dieser Aktion war, dass er eine E-Mail von einem Studenten erhalten hatte, in der die Anrede nicht stimmte, der Text unvollständig war und unverständliche Abkürzungen verwendet wurden:

*Hallo Hr. Bömmel,*[1]

*bitte geben sie mir bescheid wegen dem referatstermin.*

*Cu,*

*Chris D.*

Also, ich konnte den Unmut meines Professors durchaus nachvollziehen!

Vielleicht können die folgenden „Regeln" dazu beitragen, dass so etwas nicht passiert und die Form, Sprache und Grammatik beim täglichen E-Mailen ein bisschen erträglicher wird.

In diesem Sinne wünsche ich Ihnen weiterhin frohes E-Mailen!

**Regeln für E-Mails**

1. Nachdenken, was man tut
   - Wem schreibe ich?
   - Warum schreibe ich?
   - Was möchte ich vermitteln?
2. Grammatik und Rechtschreibung beachten
3. Groß- und Kleinschreibung beachten
4. Auf Abkürzungen verzichten, wo es geht
5. Anrede und Gruß nicht vergessen
6. Das Gelesene vor dem Versand noch einmal überprüfen

Referenzen

[1] Wikipedia *E-Mail*, http://de.wikipedia.org/wiki/E-Mail

Kurzbiographie



Kerstin Brockmann (kerstin.brockmann@adidas.com) ist studierte Informatikerin und eine ausgesprochene Liebhaberin der deutschen Sprache. Neben ihrer Tätigkeit im Bereich Sales Development bei der adidas AG, bei der sie Vertriebsprojekte mit IT-Bezug betreut, hält sie bei verschiedenen Konferenzen Vorträge und befasst sich mit Themen, die die deutsche Sprache betreffen.

[1] Nach: Professor Bömmel aus dem Spielfilm „Die Feuerzangenbowle" (1944)

Copyright © 2008 Bookware 1865-682x/01/08/009 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

Informatik und Gesellschaft

# Hex and the City

von Oliver Szymanski

Zum Start möchte ich uns die Antwort auf alle Fragen, die in Zusammenhang mit dieser Kolumne auftreten, geben. 42. Diese Antwort ist natürlich nicht von mir. Aber ich denke (und hoffe), ich muss nicht erklären, woher sie stammt.[1] Nachdem wir uns jetzt auf den Umgang mit Fragen geeinigt haben, können wir beginnen. Immerhin ist der Fortschritt der Zeit zu konstant um Pausen einzulegen. Was gerade wir in unserer täglichen Arbeit stets beobachten. Kaum beenden wir ein Projekt – erfolgreich versteht sich[2] – um in Produktion zu gehen mit der API ABC in Version 1.2.3 und dem Framework XYZ in Version 4.5.6, müssen wir feststellen (meist in einem kaffeebegleitenden Gespräch oder in einem Artikel), dass es ABC jetzt in Version 1.3.7 und XYZ in Version 5.0.b8 gibt. Außerdem heißt XYZ jetzt Xyzoon. Oder so ähnlich. Und alles andere zu benutzen gilt als nostalgisch.

Nun gut, in manchen Bereichen bezeichne ich mich gern als klassisch, also nein, dass abgeschlossene Projekt wird nicht mehr geändert. Oder doch? Wie sollen wir mit Neuerungen umgehen? Entwicklung3 ist die große Herausforderung, der wir uns jeden Tag stellen müssen. Wir könnten vorpreschen und immer das Neueste benutzen oder immer vorsichtig sein und eine Version erst dann einsetzen, wenn andere sie jahrelang in Benutzung haben. Die Erfahrung zeigt, dass Extreme selten die besten Lösungen bieten. Welche Aspekte sind wichtig, wenn wir über die Einführung von Neuerungen nachdenken?

## Mitarbeitermotivation

Hoffentlich kein Fremdwort. Wenn man Mitarbeitern stets verbietet mit neuen Technologien zu arbeiten, bleibt nicht nur die Technologie auf einem Stand stehen, sondern auch unser *wertvollstes Gut*. Stürzen wir sie hingehen ständig in neue Welten, kann sich ebenso Frustration breit machen. Ziel sollte eine ausgewogene und koordinierte Einführung neuer Technologien in jedem Projekt sein. Daher zu Beginn eines jeden Projektes die verfügbaren und in Frage kommenden Technologien kurz evaluieren und die Risiken bewerten. Mindestens eine Neuerung sollte in ein Projekt einfließen, *Arbeit am Fließband* macht immerhin niemand gern.[4] Aber ein bestehendes Projekt? Wir sind auch froh, wenn wir einmal etwas abschließen können. Also ein Projekt nicht künstlich offen halten, dass motiviert sicher nicht.

## Beseitigung von Fehlern/Stabilität

Ich würde Technologien in einem laufenden Projekt nur anpassen, wenn die neuen Versionen Fehler beheben, auf die wir stoßen oder für das weitere Projekt benötigte Features liefern, die uns das Leben erleichtern. Allerdings muss hier unbedingt das Risiko abgewogen werden, zum Beispiel was Kreuzanforderungen an andere Technologien betrifft. Muss man vielleicht deutlich mehr auf neueren Stand bringen, wenn man „nur“ eine API austauscht? Fast immer ist die Antwort darauf „Ja“ (oder 42, wie wir ja gelernt haben), daher beim Aktualisieren *niemals leichtfertig* handeln. Und die Entwicklungsdauer ist hier ebenso entscheidend. Ein Projekt, das bald in Produktion geht, tut sich schwerer mit Änderungen, als ein Projekt, dass noch ein Jahr laufen wird. Ganz entscheidend sind hier die Tests: Wir müssen anhand von *ausreichenden Tests* unbedingt verifizieren, dass sich an der Funktionalität und Lauffähigkeit unseres Projektes nichts verändert hat. Lediglich wer eine umfangreiche Testsuite besitzt, kann meines Erachtens Versionsupdates überhaupt in Betracht ziehen. Und mit umfangreich ist wirklich umfangreich gemeint. Das schließt also Blackbox- wie Whitebox-Unit-Tests, Integrationstests und viel wichtiger – als Basis für alle diese Tests – ausgeklügelte Testszenarien ein.

## Neue Projekte

Was ist mit neuen Projekten? Alles rein, was es Neues gibt? Bereits unter dem Punkt Mitarbeitermotivation erwähnte ich die Risikobewertung. Selbst bei einem neuen Projekt können wir uns nicht immer leisten, auf Bewährtes zu verzichten. Neue Technologien bedeuten neue Fehlerquellen, nicht intensiv eingearbeitete Teammitglieder, kaum Berichte über *Erfahrungen* in der weiten Welt des Internets. Wir sollten gut prüfen, welche Features wir brauchen, ob die enthaltenen Bugfixes vonnöten sein werden, und wie viele Risiken wir auf uns nehmen wollen. Bloß nicht jeden Hype mitmachen. Und wenn, dann Technologien nur in der Umgebung einsetzen, für die sie gedacht sind. Neues ist gut und wichtig, aber wir dürfen nicht die *Kontrolle* über

[1] Falls doch sei dem Leser nahe gelegt, diese Kolumne in Zukunft erst recht aufmerksam zu verfolgen.*

[*] Und sie nicht zu ernst zu nehmen.**

[**] Sowie vielleicht einmal den „Hitchhiker's Guide to the Galaxy“ [1, 2] von Douglas Adams zu lesen. Und wer bereitwillig diese Fußnoten liest, dem seien die Scheibenwelt [3] Romane von Terry Pratchett [4] ans Herz gelegt. Ehre wem Ehre gebührt.

[2] Man merkt, mit welcher Erwartungshaltung der Autor den Leser betrachtet.

[3] Hier Fortschritt, nicht das stumpfsinnige Schreiben von (teils) logischen Befehlen in kryptischen Sprachen.

[4] Oder mag hier jemand die Hand heben?

ein Projekt verlieren. Daher die Bereiche identifizieren, in denen wir uns Neuerungen erlauben können und eventuell Berater ins Team holen, die uns mit Rat und Tat zur Seite stehen und ihr Wissen an das Team weitergeben.

## Freigabe von Software

In einigen Unternehmen gibt es Bollwerke, welche die Einführung neuer Software erschweren. Hier muss man Freigaben erhalten, um etwas produktiv nutzen zu können. Gerade unter solchen Bedingungen müssen wir darauf achten, dass es den Sinn des Schutzes einer *konsistenten Software-Landschaft* nicht verliert und zu einer reinen *Blockade* wird, die uns zu langsam fortschreiten lässt. Wer hier Aktualisierungen durchführt, sollte die Freigaben rechtzeitig prüfen bzw. beantragen. Der Sinn eines solchen Bollwerkes soll an dieser Stelle natürlich nicht in Frage gestellt werden. Es ist eine weitere Instanz zur Kontrolle von Neuerungen, die projektübergreifend handelt. Gemeinsames Planen zu Beginn des Projektes, bzw. der Release-Phasen kann spätere Probleme minimieren und ein gemeinsames Verständnis für die Zukunft in der Software-Landschaft und in der Entwicklung fördern.

## Neue Wege statt neuer Software

Manchmal sind es keine Software-Aktualisierungen oder neue Tools, die Mitarbeiter vorschlagen und gern einsetzen würden. Oft geht es lediglich um neue Arten mit Problemen bei der täglichen Programmierung umzugehen. Wir stoßen auf Fragen wie „Warum benutzen wir hier keine Factory-Klassen?“ oder Bemerkungen der Art „Es wäre viel schöner, wenn wir da Aspektorientierte Programmierung einsetzen“ und ebenfalls „Das kann man auch gut in einer Zeile schreiben.“ Teilweise legitime Vorschläge, wobei mir gerade bei letzterem einige Szenarios einfallen, bei dem man dem Entwickler getrost sagen kann, dass es bezüglich Wartung, Lesbarkeit und Verständnis eine nicht ganz ausgereifte[5] Idee ist.

Aber zu dem generellem Umgang mit solchen Vorschlägen: Ich denke, sie sollten jeweils durchaus auf den Prüfstand gestellt und im Einzelfall sollte darüber entschieden werden. Und in keiner Form sollte jemals eine Antwort wie „Nein, dass haben wir noch nie so gemacht“, oder „Das machen wir aber immer so“ geäußert werden. Leider muss ich selbst manchmal solche Antworten weiterreichen, wenn ich lediglich Zwischenstation in der Entscheidungkette bin. Mir liegt dann immer direkt eine Entschuldigung an den Vorschlagseinreicher auf den Lippen, denn solche Willkür ohne echte Begründung lässt mich auch bloß den Kopf schütteln. Anders, wenn der Vorschlag selbst nicht ausreichend begründet ist: „Aber in einem Projekt hat das damals prima geklappt“. Hier können wir gern sagen: „Das Risiko ist uns zu groß. Nur weil es einmal geklappt hat, muss es bei uns nicht funktionieren“. Es ist wie in der großen kulinarischen Welt: Es gibt einen Grund, warum lokale Spezialitäten die regionalen Grenzen nicht überschritten haben.

[5] Um nicht zu sagen „dumme“!

## Die Neue Welt

Und ein letzter Punkt, den man meines Erachtens auch nicht aus den Augen verlieren darf. Der Zahn der Zeit nagt an allem, wir dürfen den *Anschluss* nicht verpassen. Wer sich jahrelang auf das Bestehende verlässt und den Fortschritt um sich ignoriert, den wird irgendwann dieses Versäumnis einholen. Dann besitzt man nur noch Mitarbeiter, die sich mit dem Neuen nicht auskennen, und wenn der Tag kommt, an dem man tatsächlich umrüsten muss, sieht man sich mit großen Problemen konfrontiert. Wissensbeschaffung ist eines davon. Und jahrelanges Wissen und Erfahrung kann sich selbst der beste Mitarbeiter nicht kurzfristig aneignen. Dazu kommt, welche Mitarbeiter unter solchen Voraussetzungen überhaupt die letzten Jahre treu geblieben sind – vielleicht nicht unbedingt die, welche sich in Neues gern einarbeiten.

Nicht zu viel und nicht zu wenig. Nicht jedes Gewürz beim Kochen ausprobieren, sondern *Ausgewogenheit* schaffen. Nichts ist so konstant wie die Lageänderung[6]. Daher müssen wir flexibel mit ihr umgehen.

[6] Vermutlich ein Zitat, wie ich dumpf im Hinterkopf habe – vielleicht sogar von mir. Leider kann ich die Quelle nicht auftreiben.

### Referenzen


[1] Adam, Douglas *The Hitchhikers Guide to the Galaxy*, Pan Books; 1979

[2] Adam, Douglas *Per Anhalter durch die Galaxis*, Heyne, 1981

[3] Wikipedia *Scheibenwelt*, http://de.wikipedia.org/wiki/Scheibenwelt

[4] Wikipedia *Terry Pratchett*, http://de.wikipedia.org/wiki/Terry_Pratchett


### Weiterführende Literatur

- Wikipedia *Per Anhalter durch die Galaxis,* http://de.wikipedia.org/wiki/Per_Anhalter_durch_die_Galaxis

### Kurzbiographie



Oliver Szymanski (oliver.szymanski@mathema.de) ist Consultant und Trainer bei der MATHEMA Software GmbH in Erlangen. Seine Themenschwerpunkte umfassen die Java Enterprise Edition und das .NET-Umfeld, sowie auch weiterreichende Gebiete wie z.B. Requirements Engineering. Er befasst sich ständig mit Neuheiten im Java- und .NET-Umfeld und hält Technologietrainings in Deutschland und der Schweiz.

Copyright © 2008 Bookware 1865-682x/01/08/010 Von diesem KaffeeKlatsch-Artikel dürfen nur dann gedruckte oder digitale Kopien im Ganzen oder in Teilen gemacht werden, wenn deren Nutzung ausschließlich privaten oder schulischen Zwecken dient. Des Weiteren dürfen jene nur dann für nicht-kommerzielle Zwecke kopiert, verteilt oder vertrieben werden, wenn diese Notiz und die vollständigen Artikelangaben der ersten Seite (Ausgabe, Autor, Titel, Untertitel) erhalten bleiben. Jede andere Art der Vervielfältigung – insbesondere die Publikation auf Servern und die Verteilung über Listen – erfordert eine spezielle Genehmigung und ist möglicherweise mit Gebühren verbunden.

# Kaffeesatz

von Michael Wiedeking

Eigentlich wollten wir an dieser Stelle einen Artikel einer renommierten Frauenzeitschrift nachdrucken, in dem eine Frau auf sehr amüsante Weise erzählt, wie sie ihren jetzigen Partner „kennen lernt“. Aufgrund von minimalen Angaben (sein Name, sein mutmaßlicher Wohnort und der Name seiner Ex-Freundin) findet sie nämlich über Google so ziemlich alles über ihren „Zukünftigen“ heraus, was man halt so über jemand im Internet erfahren kann: Lebenslauf, Sportverein, Familie, Doktorarbeit, etc. Nebenbei – durch geeignetes Bildmaterial belegt – auch etwas über die Ex-Freundin. Ganz nach dem Motto: Wozu vom Traummann träumen, wenn man doch alles über ihn nachlesen kann.

Wir hätten den Artikel gerne nachgedruckt, weil er – losgelöst von der Vollständigkeit der daraus resultierenden Implikationen – sehr schön veranschaulicht, wie den geneigten Lesern des Frauenmagazins – ohne Rückschlüsse auf deren Verständnis für Informatik schließen zu wollen – die Möglichkeiten und Gefahren des Internets nahe gebracht werden.

Allerdings dürfen wir diesen Artikel nicht nachdrucken. Der genaue Grund ist uns nicht bekannt. Aber soviel ist sicher: Eine Frauenzeitschrift ist tendenziell keine Fachzeitschrift.

Zunächst hatte eine Nachfrage des Verlags bei der Autorin zwar ergeben, dass wir den Artikel veröffentlichen dürfen. Allerdings ohne Foto und Kurzbiografie: Der Name der Autorin wäre ein Pseudonym „und das solle auch so bleiben“. Gegen eine nicht unangemessene Lizenzgebühr dürften wir den Artikel wunschgemäß in dieser Ausgabe drucken und diese schließlich auch als PDF über das Internet zum Download bereitstellen. Letzteres allerdings nur für ein Jahr.

Das hat uns ein bisschen irritiert. Eine Nachfrage klärte, ja, gegen Gebühr auch für ein weiteres Jahr. Für immer? Nein, für immer ginge nicht. Unser Bemühen, dieses Magazin als eines zu gestalten, das Artikel referenzierbar macht, wurde nicht gewürdigt. Ganz im Gegenteil: das sei nicht erwünscht.

Bekanntermaßen ist nichts so alt wie die Zeitung von gestern, wahrscheinlich gilt das auch für die Frauenzeitschrift von letzter Woche. Aber dass man nicht referenziert werden will? Zugegeben, wenn ich mir Fotos von meinem zwanzig Jahre jüngeren Ich ansehe, wundere ich mich schon sehr über meinen Modegeschmack (und bei dem von vor zwanzig Jahren, kann ich das nicht einmal auf den wohlwollenden, wenngleich nicht weniger irritierenden Modegeschmack meiner Mutter schieben). Aber es ist mir nicht so peinlich, dass ich sämtliche Bilder (und deren Negative!)[1] verbrenne oder unauffindbar verstecke.

Unsere Irritation kam daher, dass der Verlag bei einer Veröffentlichung im Internet offensichtlich nicht berücksichtigt, dass dieses – sicher zur Überraschung vieler Leichtsinniger und zum Ärger vieler Geläuteter – nichts vergisst. Aus dem Web entfernte Dokumente sind noch Wochen später über Google auffindbar, Einträge in Mailinglisten praktisch unlöschbar und wirklich interessante Dinge findet man beliebig oft kopiert immer auf irgendwelchen fremden Web-Sites. Darüber hinaus gibt es spezielle Organisationen, die es sich zur Aufgabe gemacht haben, die (kurzlebigen) Informationen des World Wide Webs zu archivieren [2]. Die Bereitstellung eines Dokuments für etwa einen Monat kommt damit schon praktisch einer Verewigung gleich.

Die Variante, den Artikel nur in die gedruckte Version aufzunehmen, schien uns das Problem souverän zu lösen. Aber bevor wir uns wieder melden konnten – die Frage, ob der Artikel auch in elektronischer Form zu haben wäre oder, ob wir ihn gegebenenfalls abschreiben müssten, war noch unbeantwortet geblieben – kam die Aussage, dass sich der Verlag eben gegen einen Nachdruck entschieden hätte.

Was nun den Verlag zu dieser Entscheidung bewogen hat, wird wohl auf ewig ein Geheimnis bleiben. Dass es ein Geheimnis ist, wird nach der Archivierung im WWW allerdings gleichermaßen immerwährend keins mehr sein. So bleibt für mich nur zu hoffen, dass ich – um es nicht ewig zu bereuen – nicht doch all zu viele Indizien angegeben habe, so dass man über Google die Frauenzeitschrift, den Artikel und die Autorin ermitteln kann.

Referenzen

[1] Wikipedia *Negativfilm*, http://de.wikipedia.org/wiki/Fotonegativ

[2] Internet Archive, http://www.archive.org/

Weiterführende Literatur

- Wikipedia *Internet Archive*, http://de.wikipedia.org/wiki/Internet_Archive
- Wikipedia *Internet Archive*, http://en.wikipedia.org/wiki/Internet_Archive
- Wikipedia *Wayback Machine*, http://en.wikipedia.org/wiki/Wayback_Machine
- Projekt Gutenberg, http://www.gutenberg.org
- Wikipedia *Projekt Gutenberg*, http://de.wikipedia.org/wiki/Project_Gutenberg

[1] Für die jüngeren Leser: Es gab eine Zeit, da gab es noch keine Digitalkameras und man musste ungefähr eine Woche darauf warten, das Ergebnis seiner fotografischen Fähigkeiten überhaupt sehen zu können. Diese wurden auf einen Negativfilm [1] gebannt, der Fotos beliebig reproduzierbar macht. Aus Film und Fernsehen konnte man lernen – ohne die fatale Erfahrung selbst machen zu müssen –, dass man bei der Vernichtung von unliebsamen Beweismaterial nicht nur sämtlicher Fotos, sondern auf jeden Fall auch deren Negative gedenkt.

# User Groups

Fehlt eine User Group? Sind Kontaktdaten falsch? Dann geben Sie uns doch bitte Bescheid.

Bookware
Henkestraße 91, 91052 Erlangen
Telefon: 0 91 31 / 89 03-0
Telefax: 0 91 31 / 89 03-55
E-Mail: redaktion@bookware.de

# Java User Groups

## Deutschland

**JUG Hamburg**
Java User Group Hamburg
http://jughh.dev.java.net/

**JUG Deutschland e.V.**
Java User Group Deutschland e.V.
c/o asc-Dienstleistungs GmbH
Ehrengard-Schramm-Weg 11, 37085 Göttingen
http://www.java.de (office@java.de)

**Rhein JUG**
Java User Group Düsseldorf
Bagelstr. 130, 40479 Düsseldorf
http://www.rheinjug.de
Kontakt: Herr Michael Jastram oder Herr Heiko Sippel (micky@alum.mit.edu)

**jugk**
Java User Group Köln
http://www.jugcologne.org (jugkoeln@yahoogroups.de)
Kontakt: Herr Ulf Zibis (ulf.zibis@gmx.de)

**JUGS e.V.**
Java User Group Stuttgart e.V.
Schönaicherstraße 3, 70597 Stuttgart
http://www.jugs.de
Kontakt: Herr Dr. Micheal Paus (mp@jugs.org)

**jugm**
Java User Group München
Jupiterweg 8, 85586 Poing
http://www.jugm.de
Kontakt: Herr Andreas Haug (ah@jugm.de)

**JUG ER-N**
Java User Group Erlangen-Nürnberg
c/o MATHEMA Software GmbH
Henkestraße 91, 91052 Erlangen
http://www.jug-ern.de
Kontakt: Frau Alexandra Specht (alexandra.specht@jug-ern.de)

## Schweiz

**JUGS**
Java User Group Switzerland
Postfach 2322, 8033 Zürich
http://www.jugs.ch (info@jugs.ch)
Kontakt: Frau Ursula Burri

# .Net User Groups

## Deutschland

**DNUG-Köln**
DotNetUserGroup Köln
Goldammerweg 325, 50829 Köln
http://www.dnug-koeln.de
Kontakt: Herr Albert Weinert (info@der-albert.com)

**.net Usergroup Frankfurt**
c/o Thomas Sohnrey, Agile IService
Mendelssohnstrasse 80, 60325 Frankfurt
http://www.dotnet-ug-frankfurt.de
Kontakt: Herr Thomas 'Teddy' Sohnrey (thomas.sohnrey@gmx.de)

**.Net User Group Leipzig**
Brockhausstraße 26, 04229 Leipzig
http://www.dotnet-leipzig.de
Kontakt: Herr Alexander Groß (alexander.gross@dotnet.leipzig.de)

**.Net User Group Bonn**
Bonner .NET User Group "Bonn-to-Code.Net"
Langwartweg 101, 53129 Bonn
http://www.bonn-to-code.net (mail@bonn-to-code.net)
Kontakt: Herr Roland Weigelt

**Dodned**
.NET User Group Franken
c/o Axinom GmbH
Kurgartenstr. 37, 90762 Fürth
http://www.dodned.de
Kontakt: Herr Damir Tomicic (damir.tomicic@dodned.de) oder Herr Johannes Jauch (johannes.jauch@dodned.de)

**.NET User Group Bielefeld**
c/o Acocon GmbH
Eckendorfer Str. 2-4, 33609 Bielefeld
http://blog.outofcoffeeexception.de
Kontakt: Herr Mathias Raacke (raacke@web.de)

**.NET User Group Paderborn**
c/o Net at Work Netzwerksysteme GmbH,
Am Hoppenhof 32, 33104 Paderborn
http://www.dotnet-paderborn.de
(raacke@dotnet-paderborn.de)
Kontakt: Herr Mathias Raacke

**.net Developer-Group Ulm**
c/o artiso solutions GmbH
Oberer Wiesenweg 25, 89134 Blaustein
http://www.dotnet-ulm.de
Kontakt: Herr Thomas Schissler (tschissler@artiso.com)

**.Net Developers Group Stuttgart**
TietoEnator Stuttgart, Conference Center, 4.Etage,
Mittlerer Pfad 2, 70499 Stuttgart
http://www.devgroup-stuttgart.de
(GroupLeader@devgroup-stuttgart.de)
Kontakt: Frau Catrin Busley

**INdotNET**
Ingolstädter .NET Developers Group
http://indot.net
Kontakt: Herr Markus Winhard (mw@bingo-ev.de)

**.NET DGH**
.NET Developers Group Hannover
Landwehrstraße 85, 30519 Hannover
http://www.dotnet-hannover.de
Kontakt: Herr Friedhelm Drecktrah
(friedhelm@drecktrah.de)

## Österreich

**.NET Usergroup Rheintal**
c/o Computer Studio Kogoj
Josefgasse 11, 6800 Feldkirch
http://usergroups.at/blogs/dotnetusergrouprheintal/default.aspx
Kontakt: Herr Thomas Kogoj (thomas@kogoj.com)

**.NET User Group Austria**
c/o Global Knowledge Network GmbH,
Gutheil Schoder Gasse 7a, 1101 Wien
http://usergroups.at/blogs/dotnetusergroupaustria/default.aspx
Kontakt: Herr Christian Nagel (ug@christiannagel.com)

Java User Group
JUG
Erlangen-Nürnberg

Ab dem 14. Februar 2008 trifft sich die Java User Group Erlangen-Nürnberg jeden zweiten Donnerstag im Monat.

Das Treffen findet im IZMP in Erlangen, Henkestraße 91, in den Schulungsräumen der MATHEMA statt.

Das Thema ist:
Der Super-Rechner auf dem Schreibtisch
Nebenläufigkeit unter Java
*Michael Wiedeking*

Der Vortrag findet von 18.00-19.30 Uhr statt, mit anschließendem Beisammensein im Café *Schwarz & Stark*.

Anmeldungen bitte unter:
anmeldung@jug-ern.de

# Herbstcampus

## Wissenstransfer par excellence

Der Idee des **Herbstcampus** liegt die Motivation zugrunde, eine überregionale Technologiekonferenz für Software-Entwickler und Projektleiter in Nürnberg zu schaffen, die sowohl durch hochwertige Wissensvermittlung als auch durch gezielten, nutzbringenden Interessensaustausch überzeugt.

Die Schwerpunkte des **Herbstcampus** liegen auf dem .NET-Framework und der Java-Technologie.

Der **Herbstcampus** bietet ein umfassendes und hochwertiges Vortragsprogramm mit namhaften Referenten, das den Teilnehmern wichtiges Know-how vermittelt und über sämtliche aktuellen Entwicklungen informiert.

in der Woche vom
15. – 19. September 2008
in Nürnberg

http://www.herbstcampus.de

Eberhard Wolff

## Spring 2

Framework für die Java-Entwicklung

iX Edition

2., aktualisierte und erweiterte Auflage

*2007, 392 Seiten, Broschur*
*€ 39,00 (D) · ISBN 978-3-89864-465-5*

*Das Buch bietet eine umfassende Einführung in die Spring-Grundlagen wie Dependency Injection und aspektorientierte Programmierung. Darüber hinaus wird die Unterstützung für Transaktionen, Persistenz, verteilte Anwendungen und das Spring Web Framework erläutert. Schließlich erhält der Leser einen Überblick über fortgeschrittene Spring-Eigenschaften sowie Technologien aus dem Spring-Umfeld. Eine Beispielanwendung illustriert die Themen.*
*Die 2. Auflage wurde vollständig überarbeitet und auf die Version 2.0 von Spring aktualisiert. Sie enthält neue Kapitel zur Java Persistence API, zu Spring Web Services und zu Spring OSGi.*







Robert F. Beeger, Arno Haase, Stefan Roock, Sebastian Sanitz

## Hibernate

Persistenz in Java-Systemen mit Hibernate und der Java Persistence API

2., überarb. und erw. Auflage

*2007, 404 Seiten, Broschur*
*€ 39,00 (D) · ISBN 978-3-89864-447-1*

*Dieses Buch beschreibt die aktuelle Version 3.2 des objektrelationalen Mapping-Frameworks Hibernate in Form eines Tutorials. Nach einer grundlegenden Einführung ins O/R-Mapping wird Hibernate im Detail vorgestellt. Darauf wird die Umsetzung verschiedener Software- und Systemarchitekturen mit Hibernate behandelt (Rich-Client, Web, Client/Server mit J2EE/Java EE). Abschließend wird Hibernate in Beziehung gesetzt zu angrenzenden Java-Technologien wie JDO, EJB 3.0, JBoss u.a. In der 2. Auflage wurde das Buch komplett auf die Hibernate-Version 3.2 aktualisiert. Neu hinzugekommen sind die Themen Java Persistence API und Spring.*

Oliver Ihns, Dierk Harbeck, Stefan M. Heldt, Holger Koschek

## EJB 3 professionell

Grundlagen- und Expertenwissen zu Enterprise JavaBeans 3 für Einsteiger, Umsteiger und Fortgeschrittene

iX Edition

Unter Mitarbeit von Jo Ehm, Carsten Sahling und Roman Schlömmer. Mit einem Geleitwort von Floyd Marinescu

*2007, 524 Seiten, Festeinband*
*€ 49,00 (D) · ISBN 978-3-89864-431-0*

*Dieses Buch vermittelt Softwareentwicklern und -architekten sowie Projektleitern das notwendige Praxiswissen für die Entwicklung verteilter komponentenorientierter Systeme mit Enterprise JavaBeans 3.0. Anhand praxisnaher Beispiele werden die Architektur sowie die zugrunde liegenden Konzepte (POJO, Meta-Annotationen, Dependency Injection, Configuration by Exception) beschrieben und illustriert. Auch die Unterschiede zu EJB 2.x, projektrelevante Themen wie Security und Testen sowie Hinweise für die Migration existierender EJB-Applikationen sind Thema dieses umfassenden Grundlagen- und Nachschlagewerkes.*







Andreas Spillner, Tilo Linz

## Basiswissen Softwaretest

Aus- und Weiterbildung zum Certified Tester - Foundation Level nach ISTQB-Standard

3., überarb. und akt. Auflage

*2005, 296 Seiten, Festeinband*
*€ 39,00 (D) · ISBN 978-3-89864-358-0*

*Dieses Buch umfasst den benötigten Stoff zum Ablegen der Prüfung »Certified Tester« (Foundation Level) nach dem Standard des International Software Testing Qualifications Board (ISTQB) und ist auch für das Selbststudium geeignet. Es behandelt die wichtigsten Methoden zum Testen von Software und zum Prüfen der Programmdokumentation. Die 3. Auflage ist konform zum aktuellen Lehrplan des ISTQB (Stand Juli 2005) und berücksichtigt auch dessen neuen Stoffanteile (u.a. exploratives Testen, Risk Based Testing).*
***Website: www.dpunkt.de/certified-tester/***

Heiko W. Rupp

## EJB 3 für Umsteiger

Neuerungen und Änderungen gegenüber dem EJB-2.x-Standard

*2007, 216 Seiten, Broschur*
*€ 29,00 (D) · ISBN 978-3-89864-429-7*

*Java-EE-Entwickler, die von Enterprise JavaBeans 2.x auf die Version 3 umsteigen wollen, erfahren in diesem Buch kompakt, was sich mit der neuen Fassung des EJB-Standards geändert hat und wie damit umzugehen ist. Ein Schwerpunkt des Buches widmet sich der Geschäftslogik der Enterprise JavaBeans, die sich nun mit weniger technischem Ballast entwickeln lässt. Ebenfalls breiten Raum nimmt die neue Java Persistence API ein, die die alten Entity Beans ersetzt und die sowohl im Java-EE- als auch im Java-SE-Umfeld zum Einsatz kommen kann. Behandelt werden außerdem die Abfragesprache JP-QL, Dependency Injection, Webservices mit EJB 3, Referenzen sowie Annotationen und Deskriptoren.*







Gunther Popp

## Konfigurationsmanagement mit Subversion, Ant und Maven

Grundlagen für Softwarearchitekten und Entwickler

2., aktualisierte Auflage

*2008, 350 Seiten, Broschur*
*€ 39,00 (D) · ISBN 978-3-89864-487-7*

*Anhand von zahlreichen Beispielen zeigt der Autor, wie man einen Konfigurationsmanagement-Prozess auf der Basis von Open-Source-Werkzeugen im laufenden Projekt schnell und sauber aufsetzen kann. Subversion übernimmt dabei die Verwaltung der Konfigurationselemente, Ant und Maven unterstützen die Projektautomatisierung sowie die Qualitätssicherung mit Tests und Metriken. Nach der Lektüre ist man in der Lage, mit Subversion, Ant und Maven eine solide Infrastruktur für ein Projektteam bereitzustellen. Die Neuauflage wurde auf Subversion 1.4.4, Ant 1.7 und Maven 2.0.7 aktualisiert.*




## Standardwerk

Guido Krüger und Thomas Stark zeigen Ihnen, welche Änderungen Sie erwarten und wie Sie mit der neuen Java-Plattform leistungsfähige Anwendungen erstellen. Anfänger erhalten einen einfachen Einstieg in die Sprache und die wichtigsten Klassenbibliotheken, Fortgeschrittene und Profis ein umfassendes Nachschlagewerk für die tägliche Arbeit. Sie lernen Java in seiner Gesamtheit kennen, von den Grundlagen bis hin zu komplexen Themen wie Grafik- und Netzwerkprogrammierung oder auch Kryptografie. Durch die klare Sprache und zahlreiche, gut verständliche Beispielprogramme eignet sich das Buch hervorragend zur Verwendung im Unterricht und zum Selbststudium.

Von Guido Krüger, Thomas Stark

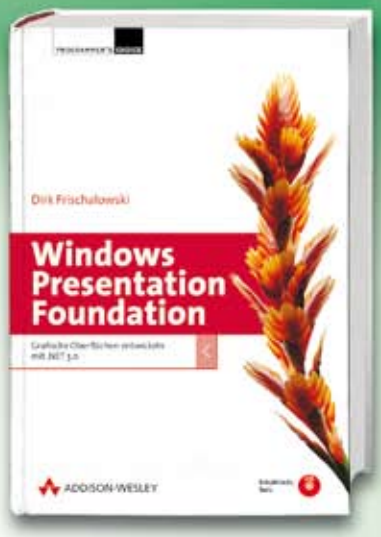
ISBN 978-3-8273-2373-6
1280 Seiten, 1 DVD
€ 49,95 [D]




Das ultimative »Kochbuch« für Java 6! Im hochwertigen Premium-Codebook finden Skript-Köche zahlreiche Rezepte zu den unterschiedlichsten Themen der Java 6-Programmierung. Da alle Codes einfach von der Interaktiv-CD überspielt werden können, ist es bereit zum sofortigen Einsatz in Ihren Anwendungen. Eine Anpassung der Beispiele an eigene Bedürfnisse ist dabei schnell und unkompliziert möglich.

Von Dirk Louis, Peter Müller

ISBN 978-3-8273-2465-8
€ 99,95 [D]




Das Buch liefert einen gut strukturierten Überblick über sämtliche Bereiche der WPF-Entwicklung. Schwerpunktmäßig werden die Neuerungen mithilfe von XAML vorgestellt, aber auch die Codierung mit C# kommt nicht zu kurz. Zahlreiche Beispiele beschreiben die vielfältigen Möglichkeiten der WPF. Besonders hervorzuheben sind separate Kapitel zu Expression Blend, der UI Automation, den Dokumentservices, zur 3D-Grafik und der Erstellung von Bildeffekten.

Von Dirk Frischalowski

ISBN 9788-3-8273-2522-8
550 Seiten, 1 CD
€ 49,95 [D]




Mit diesem Video-Training erhalten Sie einen Überblick zu Sprache, Entwicklungsumgebung und OOP. Darauf aufbauend lernen Sie, Applikationen mit ansprechendem User-Interface, Datenbank-Anbindung und Netzwerk-Funktionen zu entwickeln.

Von Andreas Romanczyk

ISBN 978-3-8273-6060-1
1 CD, 24seitiges Bonusmagazin
€ 39,95 [D]*

www.addison-wesley.de

[The Sign of Excellence]
ADDISON-WESLEY

**NEU** Der Addison-Wesley-Blog und unser Video-Podcast »Die Photoshopper« auf **www.addison-wesley.de** – reinhören und gleich kostenlos abonnieren

*unverbindliche Preisempfehlung

Designed by Martin Horngacher München

PROGRAMMER'S CHOICE
interaktiv

NEU

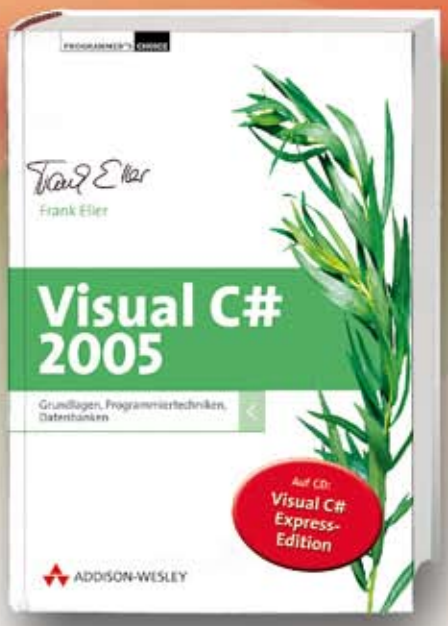



**Programmieren erleben!**

Dieser interaktive Lernkurs bietet Ihnen die Möglichkeit in kurzer Zeit die Programmiersprache JAVA 6 zu beherrschen. Ein Intensivkurs, mit dem Sie Ihren Lernweg individuell gestalten und die Lernmenge selbst einteilen können. Sie können Aufgabenstellungen aktiv lösen und bekommen Feedback. Anhand integrierter Tests überprüfen Sie Ihren Lernerfolg und trainieren mit Praxis-Übungen das Gelernte. Dieses einmalige Lernpaket umfasst eine CD mit 14 Stunden JAVA 6-Lernprogramm inklusive Java-Entwicklungsumgebung Eclipse und als ideale Ergänzung ein umfangreiches Begleitbuch zum Vertiefen und Nachschlagen.

Von Ulrike Böttcher

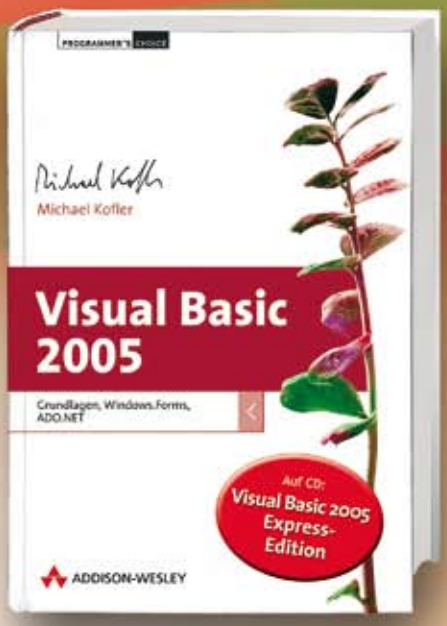
ISBN 978-3-8273-2439-9
€ 89,95 [D]*




Fortgeschrittene und Profis erhalten hier umfassendes Know-how zur Windows-Programmierung mit Visual C# in der Version 2. Besondere Schwerpunkte: .NET-Klassenbibliothek und Windows Forms, Datenbankzugriff mit ADO.NET.

Von Frank Eller

ISBN 978-3-8273-2288-3
1104 Seiten, 2 CDs
€ 49,95 [D]




Michael Kofler bietet eine umfassende Spracheinführung, Konzepte zur objektorientierten Programmierung und Lösungen für häufig benötigte Aufgaben. Der Anwendungsteil konzentriert sich auf Windows-Programmierung und Datenbank-Anwendungen. Ein neues Standardwerk.

Von Michael Kofler

ISBN 978-3-8273-2338-5
1344 Seiten, 2 CDs
€ 59,95 [D]

www.addison-wesley.de

[The Sign of Excellence]
ADDISON-WESLEY

NEU Der Addison-Wesley-Blog und unser Video-Podcast »Die Photoshopper« auf **www.addison-wesley.de** – reinhören und gleich kostenlos abonnieren

*unverbindliche Preisempfehlung

Designed by Martin Horngacher München

## Toptitel LDAP für Java-Entwickler

NEU

Stefan Zörner

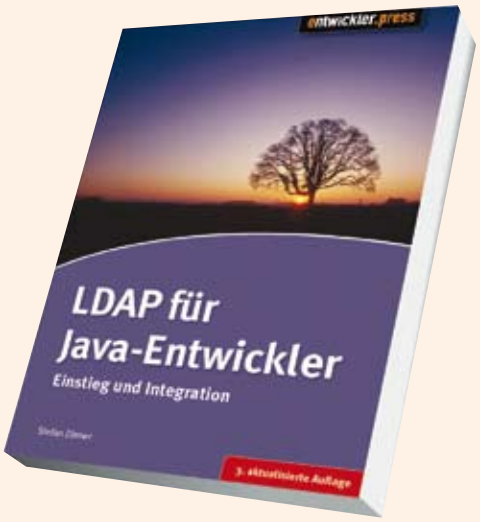
### LDAP für Java-Entwickler

Einstieg und Integration

3. aktualisierte Auflage

ca. 250 Seiten, Softcover
ca. 29,90 € (D), 30,80 € (A)
ISBN 978-3-939084-07-5
erscheint November 2007

Das Lightweight Directory Access Protocol (LDAP) ist der vorherrschende Zugriffsmechanismus auf Verzeichnisdienste. Stefan Zörner zeigt, wie Sie im Projektalltag Verzeichnisdienste in Ihre Lösungen integrieren. Neue Themen in der dritten Auflage sind Eclipse Plug-ins, Spring LDAP und Single Sign-on.







NEU

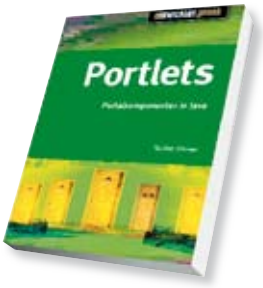
Alexander Merz

### JasperReports

Profesionelles Reporting mit Java

ca. 400 Seiten, Softcover
ca. 39,90 € (D), 41,10 € (A)
ISBN 978-3-939084-42-6
erscheint Februar 2008

JasperReports gilt als populärstes Open Source Reporting Tool, um große Datenmengen übersichtlich darzustellen. Alexander Merz erläutert die Grundlagen und zeigt, wie man die wichtigsten Probleme löst. Eine durchgehende Beispielanwendung wird mit jedem Kapitel ausgebaut.




Stefan Zörner

### Portlets

Portalkomponenten in Java

242 Seiten, Softcover
24,90 € (D), 25,60 € (A)
ISBN 978-3-935042-73-4
2006

Mit Portlets als Bausteinen lassen sich Web-Oberflächen ganz genau auf die Bedürfnisse des jeweiligen Nutzers zuschneiden. Stefan Zörner zeigt Entwicklern und Architekten mit vielen praktischen Beispielen, wie Java-Portlets vom Entwurf bis zur Inbetriebnahme in einem standardkonformen Portal realisiert werden.




NEU

Andy Bosch

### Portlets und JavaServer Faces

ca. 240 Seiten, Softcover
ca. 34,90 € (D), 35,90 € (A)
ISBN 978-3-939084-89-1
erscheint August 2008

Die Verbindung von JavaServer Faces und Portlets steht vor einer vielversprechenden Zukunft. Mit dem JSR 301 entsteht ein einheitlicher Standard. Allerdings ist die Kombination der beiden Technologien nicht ganz einfach. Andy Bosch zeigt, wie die verschiedenen Integrationslösungen funktionieren. Ein Buch für Entwickler, die JSF in einer Portlet-Umgebung zum Laufen bringen wollen.




Thilo Frotscher, Marc Teufel, Dapeng Wang

### Java Web Service mit Apache Axis 2

2. überarbeitete Auflage

582 Seiten, Softcover
44,90 € (D), 46,20 € (A)
ISBN 978-3-935042-81-9
2007

Mit Apache Axis2 können moderne Web-Service-Anwendungen programmiert werden. Dieses Buch zeigt, was das Java Web Service Framework der dritten Generation alles kann. Neulinge, Umsteiger und Profis erhalten solide Informationen über das Framework und seine Komponenten, die internen Abläufe sowie den Einsatz in geschäftskritischen Anwendungen.



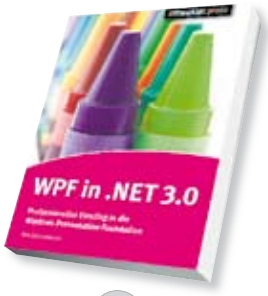

Sissi Closs

### Single Source Publishing

Topicorientierte Strukturierung und DITA

270 Seiten, Softcover
34,90 € (D), 35,90 € (A)
ISBN 978-3-935042-98-7
2006

Die Wiederverwertung vorhandener Inhalte ist angesichts des zunehmenden Kostendrucks für alle Unternehmen ein Thema. Dank Single Source Publishing müssen Inhalte nur an einer Stelle gepflegt werden, können aber für unterschiedliche Zwecke und Ausgabeformen eingesetzt werden. Sissi Closs stellt Umsetzungsmöglichkeiten vor, die sich auf einer breiten Basis durchsetzen können.




NEU Mit CD

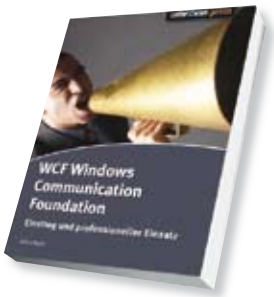
Jörg Wegener

**WPF in .NET 3.0**

Professioneller Einstieg in die Windows Presentation Foundation

ca. 450 Seiten, Softcover, CD
ca. 39,90 € (D), 41,10 € (A)
ISBN 978-3-939084-01-3
erscheint Dezember 2007

Die Windows Presentation Foundation ist ein attraktives Framework für grafische Oberflächen. Jörg Wegener zeigt, wie Sie mit der Markup-Sprache XAML komplexe Animationen realisieren, Oberflächenelemente an Daten binden oder mit C# die Oberfläche mit der Geschäftslogik verbinden können.




NEU

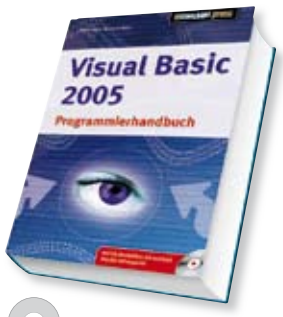
Wilko Waitz

**WCF – Windows Communication Foundation**

Einstieg und professioneller Einsatz

ca. 320 Seiten, Softcover
ca. 39,90 € (D), 41,10 € (A)
ISBN 978-3-939084-86-0
erscheint Dezember 2007

Das WCF (Windows Communication Foundation)-Framework gehört zu den zentralen Bestandteilen von .NET 3.0 und ermöglicht komplett neue Wege in der Windows-Programmierung. Dieses Buch zählt zu den ersten deutschsprachigen Einführungswerken.




Mit CD

Günter Born, Benjamin Born

**Visual Basic Programmierhandbuch**

792 Seiten, Hardcover, CD
44,90 € (D), 46,20 € (A)
ISBN 978-3-939084-04-4
2006

Ausgehend von einem Überblick über die Grundzüge von .NET führen Sie Günter und Benjamin Born in Visual Basic 2005 sowie in die Erstellung von .NET-Anwendungen ein. Sie lernen unter anderem den Umgang mit Benutzeroberflächen und Formularen kennen. Wenn Sie eine Frage zu Visual Basic haben, dieses Buch beantwortet sie.

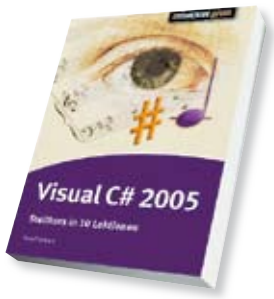



Mit CD

Günter Born, Benjamin Born

**Visual C# 2005 Programmierhandbuch**

822 Seiten, Hardcover, CD
49,90 € (D), 51,30 € (A)
ISBN 978-3-939084-40-2
2007




Rudolf Huttary

**Visual C# 2005**

Steikurs in 10 Lektionen

256 Seiten, Softcover
29,90 € (D), 30,80 € (A)
ISBN 978-3-939084-16-7
2007




NEU

Jürgen Kotz

**C# 3.0 – Die Neuerungen**

106 Seiten, Softcover
9,90 € (D), 10,20 € (A)
ISBN 978-3-939084-58-7
erscheint Oktober 2007

## Toptitel WPF und XAML Mit CD

NEU

Rainer Stropek, Karin Huber

### WPF und XAML Programmierhandbuch

ca. 550 Seiten,Hardcover, CD
ca. 49,90 € (D), 51,30 € (A)
ISBN 978-3-939084-60-0
erscheint Dezember 2007

Mit der .NET-3.0-Plattform will Microsoft der Konkurrenz die Stirn bieten. Der Einstieg in die Windows Presentation Foundation ist aber alles andere als trivial. Die Autoren erklären von den grundlegenden Strukturen über Templates bis zur Architektur moderner Softwarelösungen all das, wodurch die Anwendungsentwicklung mit .NET Entwicklern wieder Spaß macht.



# Galileo Computing: Java & Eclipse




Gottfried Wolmeringer

## Java 6 lernen mit Eclipse

**Für Programmieranfänger geeignet; auf Basis des Callisto-Paketes**

Dieses Buch bietet einen Schritt-für-Schritt-Einstieg und folgt konsequent dem Konzept des Learning-by-Doing. Sie erfahren alles Wichtige von den Grundlagen der Sprache bis zur Objektorientierung und zum Zusammenspiel mit Datenbanken.

*Ein Lernbuch im besten Sinne!*
JavaSpektrum

595 S., 2. Aufl. 2007, mit DVD, 29,90 €, 49,90 CHF
ISBN 978-3-89842-872-9

**www.galileocomputing.de/1219**




Bernhard Steppan

## Einstieg in Java 6

Bernhard Steppan hat eine ausführliche Einführung in Java geschrieben, die vor allem für Leser ohne Programmierkenntnisse geeignet ist. Grundbegriffe, Sprachelemente und der objektorientierte Ansatz werden kompetent erklärt, bevor es zum praxisorientierten Teil des Buches geht. Hier werden Java-Anwendungen geschrieben und kommentiert.

588 S., 3. Aufl. 2007, mit CD, 29,90 €, 49,90 CHF
ISBN 978-3-89842-841-5

**www.galileocomputing.de/1285**




Christian Ullenboom

## Java ist auch eine Insel

**Programmieren mit der Java Standard Edition Version 6**

- **1400 Seiten plus DVD mit Openbook, Tools, Software, Beispielen**

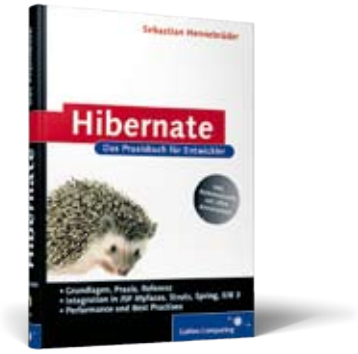
*... hinreichend detailliert, extrem aktuell und didaktisch äußerst gut aufbereitet ...*
schockwellenreiter.de

*Ein sehr praxisnahes Buch ...* c't

*Ein geniales, verständlich geschriebenes Buch, das Lust auf die Programmierung mit Java macht.* media-mania.de

1444 S., 6. Aufl. 2007, mit DVD, 49,90 €, 81,90 CHF
ISBN 978-3-89842-838-5

**www.galileocomputing.de/1318**




NEU Sebastian Hennebrüder

## Hibernate

**Das Praxisbuch für Entwickler**

- **Integration in JSF, MyFaces, Struts, Spring, EJB 3**
- **Inkl. Performance und Best Practices**

Neben einem fundierten Einstieg wird v. a. die professionelle Anwendung gezeigt. Besonderer Fokus liegt dabei auf den problematischen Punkten in Hibernate wie z. B. Session, Lazy Initialization und Optimistic Locking. Lösungsmöglichkeiten werden mit Vor- und Nachteilen erläutert.

370 S., 2007, mit Referenzkarte, 34,90 €
59,90 CHF, ISBN 978-3-89842-635-0

**www.galileocomputing.de/978**




Stephan Niedermeier, Michael Scholz

## Java und XML

**Grundlagen, Einsatz, Entwicklung**

- **Alles zu DOM, SAX, JAXP, StAX, JAXB und Webservices**

Java und XML bilden die ideale Lösung für webbasierte Enterprise-Anwendungen. Das Werk richtet sich an Programmierer, die auf dieser Grundlage komplexe Anwendungen erstellen möchten. Angefangen bei den entsprechenden Werkzeugen bis hin zur Bindung von Dokumenten an Klassen, RPC und XML-Frameworks bleibt keine Frage offen.

*Ein rundum empfehlenswertes Buch!*
Entwickler Magazin

759 S., 2006, mit CD, 39,90 €, 67,90 CHF
ISBN 978-3-89842-646-6

**www.galileocomputing.de/1012**




Bernhard Lahres, Gregor Rayman

## Praxisbuch Objektorientierung

- **Grundlagen, Prinzipien, Entwurf und Umsetzung**
- **Beispiele in C++, Java, JavaScript, Ruby und C#**
- **Aktuelle Programmierkonzepte, Inklusive Aspektorientierung**

*Ganz klar ein Muss in der Hausbibliothek, wenn die Ausbildung von Softwareentwicklern auf dem Programm steht. Gesamtnote: sehr gut.*
dotnetpro

609 S., 2006, 49,90 €, 81,90 CHF
ISBN 978-3-89842-624-4

**www.galileocomputing.de/966**

Portofreie Lieferung in [D] und [A] >> **www.galileocomputing.de**

# Galileo Computing: Java & Eclipse




NEU Thomas Künneth

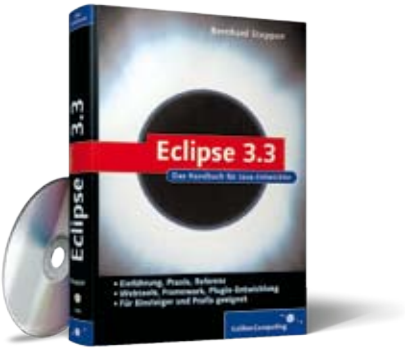
## Einstieg in Eclipse 3.3

**Einführung, Programmierung, Plug-in-Nutzung**

Dieses Buch bietet einen Einstieg in Eclipse für Leser mit Programmierkenntnissen in Java. Es zeigt auf anschauliche Weise, welche Möglichkeiten Eclipse in der Anwendungsentwicklung bietet. Mit wertvollen Praxistipps und Beispielen können Sie das Gelernte gleich in die Tat umsetzen.

ca. 500 S., mit DVD, 34,90 €, 59,90 CHF
ISBN 978-3-89842-792-0, September 2007

**www.galileocomputing.de/1152**




NEU Bernhard Steppan

## Eclipse 3.3

**Einführung, Praxis, Referenz**

Hier finden Sie alles zur Java-Entwicklung mit Eclipse: die Java Development Tools Platform, die Möglichkeiten der Webtools Platform (Webprojekte, J2EE), die Frameworks (SWT,JFace), die Rich Client Platform sowie die Plug-in-Entwicklung.

ca. 800 S., mit CD, 44,90 €, 74,90 CHF
ISBN 978-3-89842-798-2, 1. Quartal 2008

**www.galileocomputing.de/1251**




NEU Heiko Böck

## NetBeans 6

**Rich-Client-Entwicklung mit Java**

- **Einführung in die NetBeans 6 Platform**
- **Entwurf, Entwicklung, Verteilung**
- **Eclipse-Applikationen migrieren**

Dieses Werk vermittelt Ihnen alle notwendigen Techniken, um professionelle Rich-Client-Anwendungen mit der NetBeans 6 Platform entwickeln zu können. Auf praxisnahe Weise erlernen Sie dabei den Umgang mit den zahlreichen Konzepten und APIs von NetBeans 6.

ca. 400 S., 39,90 €, 67,90 CHF
ISBN 978-3-8362-1066-9, November 2007

**www.galileocomputing.de/1542**




Video-Training

Ulrich Cuber

## Eclipse 3 für Java-Entwickler

- **Von der Installation zur professionellen Anwendung**
- **Refactoring und Debugging**
- **JUnit, ANT, CVS, UML**

Anhand vieler Beispiele lernen Sie, wie Sie wichtige Prozesse der Software-Entwicklung mit Eclipse praktisch umsetzen. Entdecken Sie Eclipse Schritt für Schritt – von der Installation und Funktionserweiterung über die Projektverwaltung und den Einsatz des Editors bis hin zur Integration von JUnit, ANT und CVS.

DVD, Win, Mac, Linux, 122 Lektionen, 11 Stunden Spielzeit, 39,90 €, 64,90 CHF
ISBN 978-3-89842-868-2

**www.galileocomputing.de/1368**




Stephan Niedermeier

## Cocoon 2 und Tomcat

**XML-Publishing mit dem Open-Source-Framework**

- **Umfassende Einführung in Cocoon, Tomcat, XML, XSL und XSL-FO**
- **Produktiver Einsatz des XML-Publishing-Systems**
- **Nachschlagewerk für Anwender und Entwickler**

836 S., 2. Aufl. 2007, mit CD, 59,90 €, 96,90 CHF
ISBN 978-3-89842-656-5

**www.galileocomputing.de/1022**




Michael Pfeiffer

## Java Micro Edition

**Mobile Anwendungen mit der MIDP 2.0 entwickeln**

- **Grundlagen, Praxis, Referenz**
- **Anwendungsentwicklung für Handys und PDAs**
- **Inkl. zahlreicher Beispielcodes aus der Praxis**

311 S., 2007, mit CD, 44,90 €, 74,90 CHF, ISBN 978-3-89842-505-6

**www.galileocomputing.de/756**




Thomas Künneth

## Java für Windows

- **Nutzung aller Windows-Ressourcen**
- **Konzepte und Lösungen für eine bessere Integration**
- **Das optimale Entwicklungssystem**

314 S., 2006, mit CD, 29,90 €, 49,90 CHF, ISBN 978-3-89842-727-2

**www.galileocomputing.de/1124**

Galileo Computing
Wissen, wie's geht.

Springer

springer.de




## Management von IT-Projekten

**2. AUFLAGE**

### Von der Planung zur Realisierung

**H. W. Wieczorrek**, Hannover; **P. Mertens**, Rehburg-Loccum

Projektmanagement (PM) wird immer noch zu selten und zu inkonsequent eingesetzt. Viele problematische Projekte sprechen eine eindeutige Sprache. Kenntnisse des PM und seine Anwendungsmöglichkeiten sind für IT-Manager und IT-Entwickler unentbehrlich. Die Autoren stellen die wichtigsten Verfahren und Methoden vor. Sie verfügen über umfangreiche praktische Kenntnisse in der Führung von IT-Projekten. Daraus resultieren konkrete Handlungsempfehlungen für den erfolgreichen Einsatz des PM-Instrumentariums. Die 2. Auflage enthält u.a. eine Neubewertung der Erfolgsfaktoren im PM und ein Modell für Projektpolitik (inkl. Projektportfolio-Strategie).

2., überarb. u. erw. Aufl. 2007. XVI, 394 S. 82 Abb. Geb.
ISBN 978-3-540-48470-7 ► **€ (D) 49,95 | € (A) 51,35 | *sFr 81,50**




## Grundlagen und Modelle des Information Lifecycle Management

**G. Thome**, Inden; **W. Sollbach**, Lienen

Im Zusammenhang mit IT-Governance, Sarbanes-Oxley Act und Basel II ist Information Lifecycle Management (ILM) ein Thema, das alle CIO's beherrschen müssen. Die Autoren fassen Information als Produkt auf, das einem Lebenszyklus unterliegt. Sie gehen auf die Gesetzeslage ein, arbeiten Klassifizierungskonzepte heraus und erläutern Sicherheitskonzepte und deren Anforderungen an Betriebsführung und Unternehmen sowie Disaster Recovery als Bestandteil der Security-Strategie. Ausführlich stellen sie Qualitätsmanagement und Risikomanagement dar. Eine Betrachtung der globalen Aspekte rundet das Buch ab.

2007. XVIII, 395 S. 116 Abb. Geb.
ISBN 978-3-540-69079-5 ► **€ (D) 49,95 | € (A) 51,35 | *sFr 81,50**




## Softwareentwicklung im Offshoring

### Erfolgsfaktoren für die Praxis

**T. Steimle**, Wetzikon, Schweiz

Der Mangel an qualifizierten Softwareentwicklern im deutschsprachigen Raum verschärft sich weiter. Der entscheidende Wettbewerbsfaktor: die effektive Zusammenarbeit weltweit verteilter Teams. Daher wird Offshoring immer relevanter. Der Autor realisierte mehrere Jahre Offshore-Projekte für KMU's. Für diese Zielgruppe fasst er hier seine Erfahrungen aus Theorie und Praxis zusammen. Die Eintrittskarte für KMU's zur kostengünstigen Offshore-Softwareentwicklung: Was sie aus den Anfängerfehlern der „First Mover" lernen. Wie sie erfolgreich Offshore-Projekte umsetzen. Kompakt und praxisnah mit konkreten Fallstudien und Werkzeugen z.B. Checklisten.

2007. X, 132 S. 13 Abb. Geb.
ISBN 978-3-540-71045-5 ► **€ (D) 34,95 | € (A) 35,93 | *sFr 57,00**




## SOA?

### Serviceorientierung in Business und Software

**D. Masak**, Wiesbaden

Serviceorientierte Architektur, kurz SOA, das Zauberwort in modernen Unternehmen: Durch SOA hoffen Manager, dass Firmen flexibler, agiler, einfach schneller und unmittelbar auf Kundenwünsche eingehen können. Der Autor zeigt v.a. die Auswirkungen der Serviceorientierung auf Softwareentwicklung, Organisation und Prozesse. Denn die Einführung einer SOA hat massive Auswirkungen auf alle Teile eines Unternehmens und seine Wertschöpfungskette. Mittelfristig erfordert sie neue Kooperations- und Kommunikationsformen. Plus: die Grenzen von Services aus systemtheoretischer Sicht sowie mögliche Lösungen für sehr große Systeme und mobile Anwendungen.

2007. X, 392 S. 82 Abb. Geb.
ISBN 978-3-540-71871-0 ► **€ (D) 49,95 | € (A) 51,35 | *sFr 81,50**

**Bei Fragen oder Bestellung wenden Sie sich bitte an** ► Springer Distribution Center GmbH, Haberstr. 7, 69126 Heidelberg ► **Telefon:** +49 (0) 6221-345-4301 ► **Fax:** +49 (0) 6221-345-4229 ► **Email:** SDC-bookorder@springer.com ► € (D) sind gebundene Ladenpreise in Deutschland und enthalten 7% MwSt; € (A) sind gebundene Ladenpreise in Österreich und enthalten 10% MwSt. Die mit * gekennzeichneten Preise für Bücher und die mit ** gekennzeichneten Preise für elektronische Produkte sind unverbindliche Preisempfehlungen und enthalten die landesübliche MwSt. ► Preisänderungen und Irrtümer vorbehalten. ► Springer-Verlag GmbH, Handelsregistersitz: Berlin-Charlottenburg, HR B 91022. Geschäftsführer: Haank, Mos, Hendriks

Springer

springer.de




# Der Erfinder des Computers im Originalton!

## Der Computer – Mein Lebenswerk

**K. Zuse**

Geleitwort von: **F. Bauer, H. Zemanek**

Autobiografie des Computerpioniers Konrad Zuse

Konrad Zuse baute die erste vollautomatische, programmgesteuerte und frei programmierte, in binärer Gleitpunktrechnung arbeitende Rechenanlage. Sie war 1941 betriebsfähig. Mit anderen Worten: Zuse baute den ersten funktionsfähigen Computer.

Eindrucksvolle Darstellung der Anfänge des Computerzeitalters

In diesem Buch erzählt er die Geschichte seines Lebens, das wie kaum ein anderes mit der Geschichte der bedeutendsten technischen Entwicklung seines Jahrhunderts verbunden ist – einer Entwicklung, die mit der „Abneigung" des Bauingenieurstudenten Zuse gegen die statischen Rechnungen begonnen hat...

**Aus den Rezensionen**

► *Von der ersten bis zur letzten Seite ist diese Autobiographie eine faszinierende Lektüre, weil aus jeder Zeile das persönliche Erleben des Autors spricht. Diese Unmittelbarkeit macht Buch und Autor sympathisch.* ► **ntz Nachrichtentechnische Zeitschrift**

► *...Für entspannende und erholsame Stunden, informativ und allgemeinbildend, eigentlich genau das, was im Zusammenhang mit dem Thema Computer sehr oft gefragt und verlangt wird... Unser Tip: Sehr empfehlenswert* ► **PASCAL**

**4. AUFLAGE**

4., unveränd. Aufl. 2007. XVI, 220 S. 78 Abb. Geb.
ISBN 978-3-540-73138-2 ► **€ (D) 34,95 | € (A) 35,93 | *sFr 57,00**

**Bei Fragen oder Bestellung wenden Sie sich bitte an** ► Springer Distribution Center GmbH, Haberstr. 7, 69126 Heidelberg ► **Telefon:** +49 (0) 6221-345-4301 ► **Fax:** +49 (0) 6221-345-4229 ► **Email:** SDC-bookorder@springer.com ► € (D) sind gebundene Ladenpreise in Deutschland und enthalten 7% MwSt; € (A) sind gebundene Ladenpreise in Österreich und enthalten 10% MwSt. ► Preisänderungen und Irrtümer vorbehalten. ► Springer-Verlag GmbH, Handelsregistersitz: Berlin-Charlottenburg, HR B 91022. Geschäftsführer: Haank, Mos, Hendriks

013360a




## Das Google Web Toolkit: GWT

**Michael Seemann**
**272 Seiten, 34,90 €**
**ISBN 978-3-89721-719-5**

Eine besonders komfortable Möglichkeit für Java-Entwickler, am Web 2.0 mitzubauen, ist das Google Web Toolkit, kurz GWT. Das GWT ist ein Werkzeug, mit dem sich Webseiten analog zu Swing-Oberflächen programmieren lassen. Der eingebaute GWT-Compiler übersetzt den Java-Code in HTML und JavaScript, per Remote Procedure Calls und XML oder JSON wird die asynchrone Kommunikation mit dem Server bewerkstelligt. Die Vorteile sind immens: Der Entwickler kann sauber in seiner vertrauten Sprache programmieren, die Seiten werden ordentlich auf verschiedenen Plattformen dargestellt und der Endanwender kann problemlos per Zurückknopf zu vorherigen Seiten navigieren.

Michael Seemann zeigt die Funktionsweise des GWT anhand eines durchgängigen Projekts: In einem Fitnesstagebuch im Web sollen User ihre sportlichen Erfolge aufzeichnen können. Mit diesem praxisnahen Ansatz kommt er schnell zum Punkt und ermöglicht es dem Leser, selbst zügig loszulegen.

# Computerbücher von O'Reilly

## Lösungen vorprogrammiert!




**Michael Hüttermann**
**432 Seiten, 44,90 €**
**ISBN 978-3-89721-482-8**

Java Champion Michael Hüttermann erklärt, was agile Softwareentwicklung wirklich bedeutet, und zeigt, wie sie sich konkret in die Tat umsetzen lässt. Dabei geht er auch auf Lean Development, XP und Scrum ein, verdeutlicht den Nutzen für alle Beteiligten und berichtet von zahlreichen Fallstricken in der Praxis.



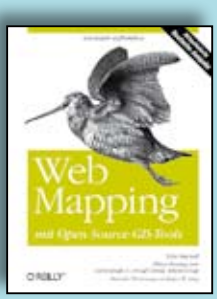

**Jörg Staudemeyer**
**352 Seiten, 39,90 €**
**ISBN 978-3-89721-483-5**

Groovy ist eine neue Open-Source-Programmiersprache, die mit dem Ziel erschaffen wurde, produktivitätssteigernde Features moderner Interpretersprachen wie Python, Ruby und Smalltalk nahtlos in die Java-Welt zu integrieren. Was Java-Entwickler wissen müssen, um diese aufregenden Eigenschaften für ihre Arbeit zu nutzen, erfahren Sie in diesem Buch.




**Steven Douglas Olson**
**248 Seiten, 29,90 €**
**ISBN 978-3-89721-718-8**

Dieses praktische, kompakte Handbuch zeigt Java-Entwicklern, wie sie ihre Webanwendungen dynamischer und interaktiver machen können, indem sie aktuelle Ajax-Features einbauen: Vorschlagslisten, Drag-and-Drop und vieles mehr. Es gibt ganz unterschiedliche Möglichkeiten, Ajax-Magie in die Java-Welt zu bringen!



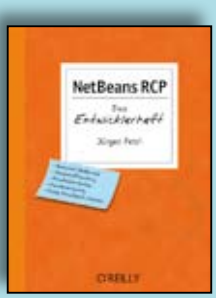

**Tyler Mitchell, Astrid Emde & Arnulf Christl**
**480 Seiten**
**inkl. CD-ROM, 49.90 €**
**ISBN 978-3-89721-723-2**

Web-Mapping mit Open Source-GIS-Tools zeigt Ihnen anschaulich, wie Sie mit frei erhältlichen Werkzeugen wie UMN MapServer, Quantum GIS, GDAL, OGR, Mapbender und PostGIS interaktive Karten erstellen können - und wie Sie sie gekonnt auf Ihrer Website präsentieren.




**Jeffrey E.F. Friedl**
**560 Seiten, 44,90 €**
**ISBN 978-3-89721-720-1**

Reguläre Ausdrücke sind standardmäßig in Perl, PHP, Java, Python, Ruby, MySQL, VB.NET und C# (und allen Sprachen des .NET-Frameworks) sowie anderen Programmiersprachen und Werkzeugen eingebaut. Dieses Buch geht detailliert auf die Unterschiede und Gemeinsamkeiten in diesen Sprachen ein.




**Jürgen Petri**
**288 Seiten, 29,90 €**
**ISBN 978-3-89721-724-9**

Jürgen Petri zeigt in diesem Entwicklerheft, wie man die moderne und mächtige NetBeans 6-Plattform nutzt, um umfangreiche Geschäftsanwendungen zu entwickeln. Anhand eines durchgängigen Projekts zeigt er, wie man schnell und effektiv zum Ziel kommt – ein Buch vom Profi für Profis!

O'REILLY®
**www.oreilly.de**

O'Reilly Verlag GmbH & Co. KG
Balthasarstr.81, 50670 Köln
Tel.: 0221-973160-91, Fax: 0221-973160-8
E-Mail: anfragen@oreilly.de

**3 für 2** ***Für Freunde englischsprachiger Fachbücher***

Wählen Sie drei Bücher aus, das günstigste schenken wir Ihnen. Unter anderem im Aktionspaket: *Head First Java*, *Java & XML* und *Java Generics*. Alle Bücher unter:

**www.oreilly.de/3fuer2**

# Das Allerletzte

*„That was the hardest day of my life. I felt like I was missing my arm. I never want to do that again. Please do not ask me to do that again.“*

Das war der schlimmste Tag meines Lebens. Es fühlte sich an als fehle mir mein Arm. Ich möchte das nie wieder machen. Bitte verlangt das nicht noch einmal von mir.

Aussage der 10-jährigen Pinky aus Deltona, Florida (USA)
nachdem sie bei einer Studie in 2005 für
einen Tag lang ihr Handy abgegeben hatte.

Chmielewski, Dawn C. *For Some Teenagers, Cell Phones Become 'an Extension of Me'*,
San Jose Mercury News, Californien (USA),
http://www.redorbit.com/news/technology/264475/for_some_teenagers_cell_phones_become_an_extension_of_me/

Der nächste KaffeeKlatsch erscheint am 4. Februar 2008

# Herbstcampus

## Wissenstransfer par excellence

in der Woche vom
15.–19. September 2008
in Nürnberg

http://www.herbstcampus.de